

代码随想录

# LeetCode 题目册

按原刷题顺序整理 · 题目内容 · 思路 · 考察范围

|      |      |      |
|------|------|------|
| 11   | 158  | 174  |
| 算法章节 | 唯一题目 | 章节题位 |

生成日期：2026-08-26

# 阅读与编排说明

本题册从本地 11 个章节中扫描 206 个语义化 LeetCode 链接，整理为 158 道唯一题。题目顺序来自本地资料；题号、难度和题干由题目链接对照 LeetCode 中文站公开数据。

同一章的总览、正文和总结重复链接已合并；跨章节复练仍在原章节出现，但完整题干只在首次出现处展示。PDF 中“我的题目思路与考察范围”均已展开。源文件中两处 637 错链已按题号修正。

| 章节 | 主题   | 题目位置 |
|----|------|------|
| 1  | 数组   | 14   |
| 2  | 链表   | 7    |
| 3  | 哈希表  | 9    |
| 4  | 字符串  | 7    |
| 5  | 双指针  | 14   |
| 6  | 栈与队列 | 7    |
| 7  | 二叉树  | 42   |
| 8  | 回溯算法 | 15   |
| 9  | 贪心算法 | 17   |
| 10 | 动态规划 | 37   |
| 11 | 单调栈  | 5    |

# 目录

|                      |           |
|----------------------|-----------|
| <b>第 1 章 数组</b>      | <b>8</b>  |
| 704. 二分查找            | 8         |
| 69. x 的平方根           | 8         |
| 367. 有效的完全平方数        | 9         |
| 27. 移除元素             | 10        |
| 26. 删除有序数组中的重复项      | 11        |
| 283. 移动零             | 12        |
| 844. 比较含退格的字符串       | 13        |
| 977. 有序数组的平方         | 13        |
| 209. 长度最小的子数组        | 14        |
| 904. 水果成篮            | 15        |
| 76. 最小覆盖子串           | 16        |
| 59. 螺旋矩阵 II          | 17        |
| 54. 螺旋矩阵             | 18        |
| 剑指 Offer 29. 顺时针打印矩阵 | 20        |
| <b>第 2 章 链表</b>      | <b>21</b> |
| 203. 移除链表元素          | 21        |
| 707. 设计链表            | 22        |
| 206. 反转链表            | 23        |
| 24. 两两交换链表中的节点       | 24        |
| 19. 删除链表的倒数第 N 个结点   | 25        |
| 面试题 02.07. 链表相交      | 26        |
| 142. 环形链表 II         | 29        |
| <b>第 3 章 哈希表</b>     | <b>31</b> |
| 242. 有效的字母异位词        | 31        |
| 349. 两个数组的交集         | 31        |
| 350. 两个数组的交集 II      | 32        |
| 202. 快乐数             | 33        |
| 1. 两数之和              | 34        |
| 454. 四数相加 II         | 34        |
| 383. 赎金信             | 35        |
| 15. 三数之和             | 36        |
| 18. 四数之和             | 37        |
| <b>第 4 章 字符串</b>     | <b>39</b> |
| 344. 反转字符串           | 39        |

|                         |           |
|-------------------------|-----------|
| 541. 反转字符串 II           | 39        |
| 剑指 Offer 05. 替换空格       | 40        |
| 151. 反转字符串中的单词          | 41        |
| 剑指 Offer 58-II. 左旋转字符串  | 42        |
| 28. 找出字符串中第一个匹配项的下标     | 42        |
| 459. 重复的子字符串            | 43        |
| <b>第 5 章 双指针</b>        | <b>45</b> |
| 27. 移除元素 (复练)           | 45        |
| 26. 删除有序数组中的重复项 (复练)    | 45        |
| 283. 移动零 (复练)           | 45        |
| 844. 比较含退格的字符串 (复练)     | 45        |
| 977. 有序数组的平方 (复练)       | 45        |
| 344. 反转字符串 (复练)         | 45        |
| 剑指 Offer 05. 替换空格 (复练)  | 46        |
| 151. 反转字符串中的单词 (复练)     | 46        |
| 206. 反转链表 (复练)          | 46        |
| 19. 删除链表的倒数第 N 个结点 (复练) | 46        |
| 面试题 02.07. 链表相交 (复练)    | 46        |
| 142. 环形链表 II (复练)       | 46        |
| 15. 三数之和 (复练)           | 47        |
| 18. 四数之和 (复练)           | 47        |
| <b>第 6 章 栈与队列</b>       | <b>48</b> |
| 232. 用栈实现队列             | 48        |
| 225. 用队列实现栈             | 49        |
| 20. 有效的括号               | 50        |
| 1047. 删除字符串中的所有相邻重复项    | 51        |
| 150. 逆波兰表达式求值           | 52        |
| 239. 滑动窗口最大值            | 53        |
| 347. 前 K 个高频元素          | 54        |
| <b>第 7 章 二叉树</b>        | <b>55</b> |
| 144. 二叉树的前序遍历           | 55        |
| 145. 二叉树的后序遍历           | 56        |
| 94. 二叉树的中序遍历            | 58        |
| 102. 二叉树的层序遍历           | 59        |
| 107. 二叉树的层序遍历 II        | 60        |
| 199. 二叉树的右视图            | 61        |
| 637. 二叉树的层平均值           | 63        |
| 429. N 叉树的层序遍历          | 64        |

|                                    |     |
|------------------------------------|-----|
| 515. 在每个树行中找最大值 . . . . .          | 66  |
| 116. 填充每个节点的下一个右侧节点指针 . . . . .    | 67  |
| 117. 填充每个节点的下一个右侧节点指针 II . . . . . | 68  |
| 104. 二叉树的最大深度 . . . . .            | 69  |
| 111. 二叉树的最小深度 . . . . .            | 70  |
| 226. 翻转二叉树 . . . . .               | 71  |
| 589. N 叉树的前序遍历 . . . . .           | 72  |
| 590. N 叉树的后序遍历 . . . . .           | 73  |
| 101. 对称二叉树 . . . . .               | 75  |
| 100. 相同的树 . . . . .                | 76  |
| 572. 另一棵树的子树 . . . . .             | 78  |
| 559. N 叉树的最大深度 . . . . .           | 79  |
| 222. 完全二叉树的节点个数 . . . . .          | 81  |
| 110. 平衡二叉树 . . . . .               | 82  |
| 257. 二叉树的所有路径 . . . . .            | 83  |
| 404. 左叶子之和 . . . . .               | 84  |
| 513. 找树左下角的值 . . . . .             | 85  |
| 112. 路径总和 . . . . .                | 87  |
| 113. 路径总和 II . . . . .             | 88  |
| 106. 从中序与后序遍历序列构造二叉树 . . . . .     | 90  |
| 105. 从前序与中序遍历序列构造二叉树 . . . . .     | 91  |
| 654. 最大二叉树 . . . . .               | 92  |
| 617. 合并二叉树 . . . . .               | 94  |
| 700. 二叉搜索树中的搜索 . . . . .           | 95  |
| 98. 验证二叉搜索树 . . . . .              | 96  |
| 530. 二叉搜索树的最小绝对差 . . . . .         | 98  |
| 501. 二叉搜索树中的众数 . . . . .           | 99  |
| 236. 二叉树的最近公共祖先 . . . . .          | 100 |
| 235. 二叉搜索树的最近公共祖先 . . . . .        | 102 |
| 701. 二叉搜索树中的插入操作 . . . . .         | 103 |
| 450. 删除二叉搜索树中的节点 . . . . .         | 104 |
| 669. 修剪二叉搜索树 . . . . .             | 105 |
| 108. 将有序数组转换为二叉搜索树 . . . . .       | 106 |
| 538. 把二叉搜索树转换为累加树 . . . . .        | 108 |

## 第 8 章 回溯算法

110

|                         |     |
|-------------------------|-----|
| 77. 组合 . . . . .        | 110 |
| 216. 组合总和 III . . . . . | 110 |
| 17. 电话号码的字母组合 . . . . . | 111 |
| 39. 组合总和 . . . . .      | 112 |
| 40. 组合总和 II . . . . .   | 113 |

|                     |            |
|---------------------|------------|
| 131. 分割回文串          | 114        |
| 93. 复原 IP 地址        | 115        |
| 78. 子集              | 116        |
| 90. 子集 II           | 116        |
| 491. 非递减子序列         | 117        |
| 46. 全排列             | 118        |
| 47. 全排列 II          | 118        |
| 332. 重新安排行程         | 119        |
| 51. N 皇后            | 121        |
| 37. 解数独             | 122        |
| <b>第 9 章 贪心算法</b>   | <b>124</b> |
| 455. 分发饼干           | 124        |
| 376. 摆动序列           | 125        |
| 53. 最大子数组和          | 126        |
| 122. 买卖股票的最佳时机 II   | 126        |
| 55. 跳跃游戏            | 127        |
| 45. 跳跃游戏 II         | 128        |
| 1005. K 次取反后最大化的数组和 | 129        |
| 134. 加油站            | 130        |
| 135. 分发糖果           | 131        |
| 860. 柠檬水找零          | 131        |
| 406. 根据身高重建队列       | 132        |
| 452. 用最少数量的箭引爆气球    | 133        |
| 435. 无重叠区间          | 134        |
| 763. 划分字母区间         | 135        |
| 56. 合并区间            | 136        |
| 738. 单调递增的数字        | 137        |
| 968. 监控二叉树          | 137        |
| <b>第 10 章 动态规划</b>  | <b>139</b> |
| 509. 斐波那契数          | 139        |
| 70. 爬楼梯             | 140        |
| 746. 使用最小花费爬楼梯      | 140        |
| 62. 不同路径            | 141        |
| 63. 不同路径 II         | 142        |
| 343. 整数拆分           | 144        |
| 96. 不同的二叉搜索树        | 145        |
| 416. 分割等和子集         | 145        |
| 1049. 最后一块石头的重量 II  | 146        |
| 494. 目标和            | 147        |

|                        |     |
|------------------------|-----|
| 474. 一和零               | 148 |
| 518. 零钱兑换 II           | 149 |
| 377. 组合总和 IV           | 150 |
| 322. 零钱兑换              | 151 |
| 279. 完全平方数             | 151 |
| 139. 单词拆分              | 152 |
| 198. 打家劫舍              | 153 |
| 213. 打家劫舍 II           | 154 |
| 337. 打家劫舍 III          | 155 |
| 121. 买卖股票的最佳时机         | 156 |
| 122. 买卖股票的最佳时机 II (复练) | 157 |
| 123. 买卖股票的最佳时机 III     | 157 |
| 188. 买卖股票的最佳时机 IV      | 158 |
| 309. 买卖股票的最佳时机含冷冻期     | 159 |
| 714. 买卖股票的最佳时机含手续费     | 160 |
| 300. 最长递增子序列           | 160 |
| 674. 最长连续递增序列          | 161 |
| 718. 最长重复子数组           | 162 |
| 1143. 最长公共子序列          | 163 |
| 1035. 不相交的线            | 163 |
| 53. 最大子数组和 (复练)        | 165 |
| 392. 判断子序列             | 165 |
| 115. 不同的子序列            | 166 |
| 583. 两个字符串的删除操作        | 166 |
| 72. 编辑距离               | 167 |
| 647. 回文子串              | 168 |
| 516. 最长回文子序列           | 169 |

## 第 11 章 单调栈 170

|                 |     |
|-----------------|-----|
| 739. 每日温度       | 170 |
| 496. 下一个更大元素 I  | 170 |
| 503. 下一个更大元素 II | 171 |
| 42. 接雨水         | 172 |
| 84. 柱状图中最大的矩形   | 173 |

## 附录：校验口径 175

# 第 1 章 数组

本章 14 道去重题目，按源文件首次出现顺序排列。

## 704. 二分查找

|      |        |      |                              |
|------|--------|------|------------------------------|
| 唯一题序 | 1/158  | 本章顺序 | 1/14                         |
| 难度   | 简单     | 题目类型 | 主讲题                          |
| 出现章节 | 第1章 数组 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:122 |

LeetCode : <https://leetcode.cn/problems/binary-search/>

### 题目内容

给定一个  $n$  个元素有序的（升序）整型数组 `nums` 和一个目标值 `target`，写一个函数搜索 `nums` 中的 `target`，如果 `target` 存在返回下标，否则返回 `-1`。

你必须编写一个具有  $O(\log n)$  时间复杂度的算法。

示例 1:

输入: `nums = [-1,0,3,5,9,12]`, `target = 9`  
 输出: 4  
 解释: 9 出现在 `nums` 中并且下标为 4

示例 2:

输入: `nums = [-1,0,3,5,9,12]`, `target = 2`  
 输出: -1  
 解释: 2 不存在 `nums` 中因此返回 -1

提示：

- 1 你可以假设 `nums` 中的所有元素是不重复的。
- 2 `n` 将在 `[1, 10000]` 之间。
- 3 `nums` 的每个元素都将在 `[-9999, 9999]` 之间。

我的题目思路与考察范围

核心思路：在有序数组上维护闭区间 `[left, right]`，比较中点后排除不可能包含 `target` 的半区，命中时返回下标。

关键不变量 / 状态：`target` 若存在就始终位于当前搜索区间；闭区间写法要求循环条件为 `left <= right`。

复杂度：时间  $O(\log n)$ ，额外空间  $O(1)$

考察范围：二分查找 · 有序数组 · 区间定义 · 边界处理

易错点：中点计算、循环条件和左右边界更新不配套会导致漏解或死循环。

## 69. x 的平方根

|      |        |      |                              |
|------|--------|------|------------------------------|
| 唯一题序 | 2/158  | 本章顺序 | 2/14                         |
| 难度   | 简单     | 题目类型 | 拓展/推荐                        |
| 出现章节 | 第1章 数组 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:266 |

LeetCode : <https://leetcode.cn/problems/sqrtx/>



## 题目内容

给你一个非负整数  $x$ ，计算并返回  $x$  的算术平方根。

由于返回类型是整数，结果只保留整数部分，小数部分将被舍去。

注意：不允许使用任何内置指数函数和算符，例如  $\text{pow}(x, 0.5)$  或者  $x^{**0.5}$ 。

示例 1：

输入： $x = 4$   
输出：2

示例 2：

输入： $x = 8$   
输出：2  
解释：8 的算术平方根是 2.82842...，由于返回类型是整数，小数部分将被舍去。

提示：

1  $0 \leq x \leq 2^{31} - 1$

我的题目思路与考察范围

核心思路：在整数区间  $[0, x]$  上二分寻找满足  $\text{mid}^2 \leq x$  的最大  $\text{mid}$ ，即向下取整的平方根；用  $\text{mid} \leq x / \text{mid}$  比较可规避乘法溢出。

关键不变量 / 状态：已排除区间不可能包含答案，并持续保留最后一个平方不超过  $x$  的候选值。

复杂度：时间  $O(\log x)$ ，额外空间  $O(1)$

考察范围：二分答案 · 整数平方根 · 溢出规避 · 边界处理

易错点：必须单独正确覆盖  $x$  为 0 或 1，并避免直接计算  $\text{mid} * \text{mid}$  造成溢出。

## 367. 有效的完全平方数

|      |        |      |                               |
|------|--------|------|-------------------------------|
| 唯一题序 | 3/158  | 本章顺序 | 3/14                          |
| 难度   | 简单     | 题目类型 | 拓展/推荐                         |
| 出现章节 | 第1章 数组 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:267 |

LeetCode：https://leetcode.cn/problems/valid-perfect-square/

## 题目内容

给你一个正整数  $\text{num}$ 。如果  $\text{num}$  是一个完全平方数，则返回  $\text{true}$ ，否则返回  $\text{false}$ 。

完全平方数 是一个可以写成某个整数的平方的整数。换句话说，它可以写成某个整数和自身的乘积。

不能使用任何内置的库函数，如  $\text{sqrt}$ 。

示例 1：

输入： $\text{num} = 16$   
输出： $\text{true}$   
解释：返回  $\text{true}$ ，因为  $4 * 4 = 16$  且 4 是一个整数。

示例 2：

输入： $\text{num} = 14$   
输出： $\text{false}$   
解释：返回  $\text{false}$ ，因为  $3.742 * 3.742 = 14$  但 3.742 不是一个整数。

提示：

1 1 <= num <= 2 ^31 - 1

我的题目思路与考察范围

核心思路：二分查找是否存在整数 mid 使  $mid^2$  恰好等于 num，而不是只求平方根的下界或向下取整值。

关键不变量 / 状态：小于 left 的候选平方都过小，大于 right 的候选平方都过大。

复杂度：时间  $O(\log \text{num})$ ，额外空间  $O(1)$

考察范围：二分查找 · 完全平方数 · 整数运算 · 溢出规避

易错点：得到  $\text{floor}(\text{sqrt}(\text{num}))$  后仍要检查平方是否精确相等，且乘法可能溢出。

## 27. 移除元素

|      |                |      |                               |
|------|----------------|------|-------------------------------|
| 唯一题序 | 4/158          | 本章顺序 | 4/14                          |
| 难度   | 简单             | 题目类型 | 主讲题                           |
| 出现章节 | 第1章 数组、第5章 双指针 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:271 |

LeetCode : <https://leetcode.cn/problems/remove-element/>

### 题目内容

给你一个数组 nums 和一个值 val，你需要原地移除所有数值等于 val 的元素。元素的顺序可能发生改变。然后返回 nums 中与 val 不同的元素的数量。

假设 nums 中不等于 val 的元素数量为 k，要通过此题，您需要执行以下操作：

- 1 更改 nums 数组，使 nums 的前 k 个元素包含不等于 val 的元素。nums 的其余元素和 nums 的大小并不重要。
- 1 返回 k。

用户评测：

评测机将使用以下代码测试您的解决方案：

```
int[] nums = [...]; // 输入数组
int val = ...; // 要移除的值
int[] expectedNums = [...]; // 长度正确的预期答案。
// 它以不等于 val 的值排序。

int k = removeElement(nums, val); // 调用你的实现

assert k == expectedNums.length;
sort(nums, 0, k); // 排序 nums 的前 k 个元素
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

如果所有的断言都通过，你的解决方案将会通过。

示例 1：

输入：nums = [3,2,2,3], val = 3  
 输出：2, nums = [2,2,\_,\_]   
 解释：你的函数应该返回 k = 2，并且 nums 中的前两个元素均为 2。  
 你在返回的 k 个元素之外留下了什么并不重要（因此它们并不计入评测）。

示例 2：

输入：nums = [0,1,2,2,3,0,4,2], val = 2  
 输出：5, nums = [0,1,4,0,3,\_,\_,\_]   
 解释：你的函数应该返回 k = 5，并且 nums 中的前五个元素为 0,0,1,3,4。  
 注意这五个元素可以任意顺序返回。

你在返回的  $k$  个元素之外留下了什么并不重要（因此它们并不计入评测）。

提示：

```
1 0 <= nums.length <= 100
```

```
1 0 <= nums[i] <= 50
```

```
1 0 <= val <= 100
```

我的题目思路与考察范围

核心思路：用快指针扫描所有元素，慢指针只写入不等于  $val$  的元素，从而原地压缩有效前缀并返回其长度。

关键不变量 / 状态：任意时刻区间  $[0, slow)$  恰好是已扫描部分中应保留元素的稳定序列。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：快慢指针 · 原地修改 · 数组压缩 · 循环不变量

易错点：返回后只有前  $k$  个位置有语义，不能要求其后的数组内容保持原样。

## 26. 删除有序数组中的重复项

|      |                |      |                              |
|------|----------------|------|------------------------------|
| 唯一题序 | 5/158          | 本章顺序 | 5/14                         |
| 难度   | 简单             | 题目类型 | 拓展/推荐                        |
| 出现章节 | 第1章 数组、第5章 双指针 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:408 |

LeetCode：https://leetcode.cn/problems/remove-duplicates-from-sorted-array/

### 题目内容

给你一个 非严格递增排列 的数组  $nums$ ，请你 原地 删除重复出现的元素，使每个元素 只出现一次，返回删除后数组的新长度。元素的 相对顺序 应该保持一致。然后返回  $nums$  中唯一元素的个数。

考虑  $nums$  的唯一元素的数量为  $k$ 。去重后，返回唯一元素的数量  $k$ 。

$nums$  的前  $k$  个元素应包含 排序后 的唯一数字。下标  $k - 1$  之后的剩余元素可以忽略。

判题标准：

系统会用下面的代码来测试你的题解：

```
int[] nums = [...]; // 输入数组
int[] expectedNums = [...]; // 长度正确的期望答案

int k = removeDuplicates(nums); // 调用

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

如果所有断言都通过，那么您的题解将被 通过。

示例 1：

```
输入：nums = [1,1,2]
输出：2, nums = [1,2,...]
解释：函数应该返回新的长度 2，并且原数组 nums 的前两个元素被修改为 1, 2。不需要考虑数组中超出新长度后面的元素。
```

示例 2：

```
输入：nums = [0,0,1,1,1,2,2,3,3,4]
输出：5, nums = [0,1,2,3,4,...]
解释：函数应该返回新的长度 5，并且原数组 nums 的前五个元素被修改为 0, 1, 2, 3, 4。不需要考虑数组中超出新长度后面的元素。
```

提示：

1 1 <= nums.length <= 3 \* 10 ^4

1 -10 0 <= nums[i] <= 10 0

1 nums 已按 非递减 顺序排列。

我的题目思路与考察范围

核心思路：利用数组有序性，让快指针发现新值后写到慢指针的下一位，使每个不同元素只保留一次。

关键不变量 / 状态：区间 [0, slow] 始终是已扫描前缀去重后的完整有序结果。

复杂度：时间 O(n)，额外空间 O(1)

考察范围：快慢指针 · 有序数组 · 原地去重 · 边界处理

易错点：空数组需要返回 0，非空时应与 nums[slow] 比较而不是固定与前一个快指针元素比较。

## 283. 移动零

|      |                |      |                              |
|------|----------------|------|------------------------------|
| 唯一题序 | 6/158          | 本章顺序 | 6/14                         |
| 难度   | 简单             | 题目类型 | 拓展/推荐                        |
| 出现章节 | 第1章 数组、第5章 双指针 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:409 |

LeetCode：https://leetcode.cn/problems/move-zeroes/

### 题目内容

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

输入: nums = [0,1,0,3,12]  
输出: [1,3,12,0,0]

示例 2:

输入: nums = [0]  
输出: [0]

提示：

1 1 <= nums.length <= 10 ^4

1 -2 ^31 <= nums[i] <= 2 ^31 - 1

进阶：你能尽量减少完成的操作次数吗？

我的题目思路与考察范围

核心思路：先按原相对顺序压缩所有非零元素，再补零，或在扫描时把新发现的非零元素交换到慢指针位置。

关键不变量 / 状态：区间 [0, slow) 始终是已扫描部分全部非零元素的稳定序列，slow 指向下一个可写位置。

复杂度：时间 O(n)，额外空间 O(1)

考察范围：快慢指针 · 稳定划分 · 原地数组 · 写指针

易错点：覆盖式压缩后必须把剩余位置补成零，同时不能改变非零元素的相对顺序。

## 844. 比较含退格的字符串

|      |                |      |                               |
|------|----------------|------|-------------------------------|
| 唯一题序 | 7/158          | 本章顺序 | 7/14                          |
| 难度   | 简单             | 题目类型 | 拓展/推荐                         |
| 出现章节 | 第1章 数组、第5章 双指针 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:410 |

LeetCode : <https://leetcode.cn/problems/backspace-string-compare/>

### 题目内容

给定  $s$  和  $t$  两个字符串，当它们分别被输入到空白的文本编辑器后，如果两者相等，返回 `true`。# 代表退格字符。

注意：如果对空文本输入退格字符，文本继续为空。

示例 1：

输入： $s = "ab\#c"$ ,  $t = "ad\#c"$   
 输出：`true`  
 解释： $s$  和  $t$  都会变成 `"ac"`。

示例 2：

输入： $s = "ab\#\#"$ ,  $t = "c\#d\#"$   
 输出：`true`  
 解释： $s$  和  $t$  都会变成 `""`。

示例 3：

输入： $s = "a\#c"$ ,  $t = "b"$   
 输出：`false`  
 解释： $s$  会变成 `"c"`，但  $t$  仍然是 `"b"`。

提示：

- 1  $1 \leq s.length, t.length \leq 200$
- 1  $s$  和  $t$  只含有小写字母以及字符 '#'

进阶：

- 1 你可以用  $O(n)$  的时间复杂度和  $O(1)$  的空间复杂度解决该问题吗？

我的题目思路与考察范围

核心思路：从两个字符串末尾逆向扫描，用各自的 `skip` 计数跳过被退格删除的字符，再逐个比较下一枚实际可见字符。

关键不变量 / 状态：每轮比较前，两个指针都停在各自尚未比较的下一枚有效字符上，或已越过字符串开头。

复杂度：时间  $O(m + n)$ ，额外空间  $O(1)$

考察范围：双指针 · 逆向扫描 · 退格模拟 · 常数空间

易错点：连续退格以及退格数超过已有字符数时，`skip` 的更新顺序最容易写错。

## 977. 有序数组的平方

|      |                |      |                               |
|------|----------------|------|-------------------------------|
| 唯一题序 | 8/158          | 本章顺序 | 8/14                          |
| 难度   | 简单             | 题目类型 | 拓展/推荐、主讲题                     |
| 出现章节 | 第1章 数组、第5章 双指针 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:411 |

LeetCode : <https://leetcode.cn/problems/squares-of-a-sorted-array/>

## 题目内容

给你一个按 非递减顺序 排序的整数数组 `nums`，返回 每个数字的平方 组成的新数组，要求也按 非递减顺序 排序。

示例 1：

输入：nums = [-4,-1,0,3,10]  
输出：[0,1,9,16,100]  
解释：平方后，数组变为 [16,1,0,9,100]  
排序后，数组变为 [0,1,9,16,100]

示例 2：

输入：nums = [-7,-3,2,3,11]  
输出：[4,9,9,49,121]

提示：

- 1 1 <= nums.length <= 10 ^4
- 1 -10 ^4 <= nums[i] <= 10 ^4
- 1 nums 已按 非递减顺序 排序

进阶：

- 1 请你设计时间复杂度为  $O(n)$  的算法解决本问题

我的题目思路与考察范围

核心思路：有序数组平方后的最大值一定来自左右端点；比较两端绝对值，并从结果数组末尾向前写入较大平方。

关键不变量 / 状态：写指针右侧已经放好当前尚未处理元素中最大的若干平方，且顺序正确。

复杂度：时间  $O(n)$ ，输出空间  $O(n)$ ，除结果外额外空间  $O(1)$

考察范围：双指针 · 有序数组 · 逆向填充 · 绝对值比较

易错点：结果必须从后向前填充，否则先写入的大平方会破坏非递减顺序。

## 209. 长度最小的子数组

|      |        |      |                              |
|------|--------|------|------------------------------|
| 唯一题序 | 9/158  | 本章顺序 | 9/14                         |
| 难度   | 中等     | 题目类型 | 主讲题                          |
| 出现章节 | 第1章 数组 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:507 |

LeetCode：https://leetcode.cn/problems/minimum-size-subarray-sum/

## 题目内容

给定一个含有  $n$  个正整数的数组和一个正整数 `target`。

找出该数组中满足其总和大于等于 `target` 的长度最小的 子数组 `[nums_l, nums_l+1, ..., nums_r-1, nums_r]`，并返回其长度。如果不存在符合条件的子数组，返回 0。

示例 1：

输入：target = 7, nums = [2,3,1,2,4,3]  
输出：2  
解释：子数组 [4,3] 是该条件下的长度最小的子数组。

示例 2：

输入：target = 4, nums = [1,4,4]  
输出：1

示例 3：

输入：target = 11, nums = [1,1,1,1,1,1,1]  
输出：0

提示：

- 1 1 <= target <= 10 ^9
- 1 1 <= nums.length <= 10 ^5
- 1 1 <= nums[i] <= 10 ^4

进阶：

- 1 如果你已经实现 O(n) 时间复杂度的解法, 请尝试设计一个 O(n log(n)) 时间复杂度的解法。

我的题目思路与考察范围

核心思路：数组元素均为正数，因此右端扩张只会增大窗口和；当和达到 target 时反复移动左端，记录所有可行窗口中的最短长度。

关键不变量 / 状态：窗口和始终对应 [left, right]，每个 right 下所有能通过收缩得到的可行候选都会被检查。

复杂度：时间 O(n)，额外空间 O(1)

考察范围：滑动窗口 · 正数数组 · 最短子数组 · 双指针

易错点：该收缩逻辑依赖元素为正，且不存在可行窗口时必须返回 0。

## 904. 水果成篮

|      |        |      |                              |
|------|--------|------|------------------------------|
| 唯一题序 | 10/158 | 本章顺序 | 10/14                        |
| 难度   | 中等     | 题目类型 | 拓展/推荐                        |
| 出现章节 | 第1章 数组 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:642 |

LeetCode：https://leetcode.cn/problems/fruit-into-baskets/

### 题目内容

你正在探访一家农场，农场从左到右种植了一排果树。这些树用一个整数数组 fruits 表示，其中 fruits[i] 是第 i 棵树上的水果 种类。

你想要尽可能多地收集水果。然而，农场的主人设定了一些严格的规矩，你必须按照要求采摘水果：

- 1 你只有 两个 篮子，并且每个篮子只能装 单一类型 的水果。每个篮子能够装的水果总量没有限制。
- 1 你可以选择任意一棵树开始采摘，你必须从 每棵 树 ( 包括开始采摘的树 ) 上 恰好摘一个水果 。采摘的水果应当符合篮子中的水果类型。每采摘一次，你将会向右移动到下一棵树，并继续采摘。
- 1 一旦你走到某棵树前，但水果不符合篮子的水果类型，那么就必须停止采摘。

给你一个整数数组 fruits ，返回你可以收集的水果的 最大 数目。

示例 1：

输入：fruits = [1,2,1]  
输出：3  
解释：可以采摘全部 3 棵树。

示例 2：

输入：fruits = [0,1,2,2]  
输出：3  
解释：可以采摘 [1,2,2] 这三棵树。

如果从第一棵树开始采摘，则只能采摘 [0,1] 这两棵树。

示例 3：

输入：fruits = [1,2,3,2,2]

输出：4

解释：可以采摘 [2,3,2,2] 这四棵树。

如果从第一棵树开始采摘，则只能采摘 [1,2] 这两棵树。

示例 4：

输入：fruits = [3,3,3,1,2,1,1,2,3,3,4]

输出：5

解释：可以采摘 [1,2,1,1,2] 这五棵树。

提示：

1 1 <= fruits.length <= 10 ^5

1 0 <= fruits[i] < fruits.length

我的题目思路与考察范围

核心思路：把问题转化为最长的至多包含两种值的连续子数组，用频次表扩张右端，并在种类超过两种时收缩左端。

关键不变量 / 状态：收缩结束后窗口内至多有二种水果，频次表与当前窗口完全一致。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$  (窗口中种类数有常数上限)

考察范围：滑动窗口 · 频次哈希 · 最长子数组 · 至多 K 种

易错点：某种水果计数降为零时要删除键，否则会错误判断窗口仍包含三种水果。

## 76. 最小覆盖子串

|      |        |      |                               |
|------|--------|------|-------------------------------|
| 唯一题序 | 11/158 | 本章顺序 | 11/14                         |
| 难度   | 困难     | 题目类型 | 拓展/推荐                         |
| 出现章节 | 第1章 数组 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:643 |

LeetCode：https://leetcode.cn/problems/minimum-window-substring/

### 题目内容

给定两个字符串  $s$  和  $t$ ，长度分别是  $m$  和  $n$ ，返回  $s$  中的最短窗口子串，使得该子串包含  $t$  中的每一个字符（包括重复字符）。如果没有这样的子串，返回空字符串 ""。

测试用例保证答案唯一。

示例 1：

输入：s = "ADOBECODEBANC", t = "ABC"

输出："BANC"

解释：最小覆盖子串 "BANC" 包含来自字符串  $t$  的 'A'、'B' 和 'C'。

示例 2：

输入：s = "a", t = "a"

输出："a"

解释：整个字符串  $s$  是最小覆盖子串。

示例 3:

输入: s = "a", t = "aa"

输出: ""

解释:  $t$  中两个字符 'a' 均应包含在  $s$  的子串中，因此没有符合条件的子字符串，返回空字符串。



提示：

```
1 m == s.length
1 n == t.length
1 1 <= m, n <= 10 ^5
1 s 和 t 由英文字母组成
```

进阶：

你能设计一个在

$O(m + n)$

时间内解决此问题的算法吗？

我的题目思路与考察范围

核心思路：统计  $t$  的字符需求及重数，右指针扩张直到窗口覆盖全部需求，再收缩左端寻找以当前右端结尾的最短覆盖窗口。

关键不变量 / 状态：窗口计数与  $[left, right]$  一致，并用 `missing` 或 `formed` 精确表示需求是否全部满足。

复杂度：时间  $O(|s| + |t|)$ ，空间  $O(|\Sigma|)$

考察范围：滑动窗口 · 字符计数 · 最小覆盖 · 多重集合 · 双指针

易错点：必须按字符重数判断覆盖，并在移出左端字符的正确时机更新有效状态。

## 59. 螺旋矩阵 II

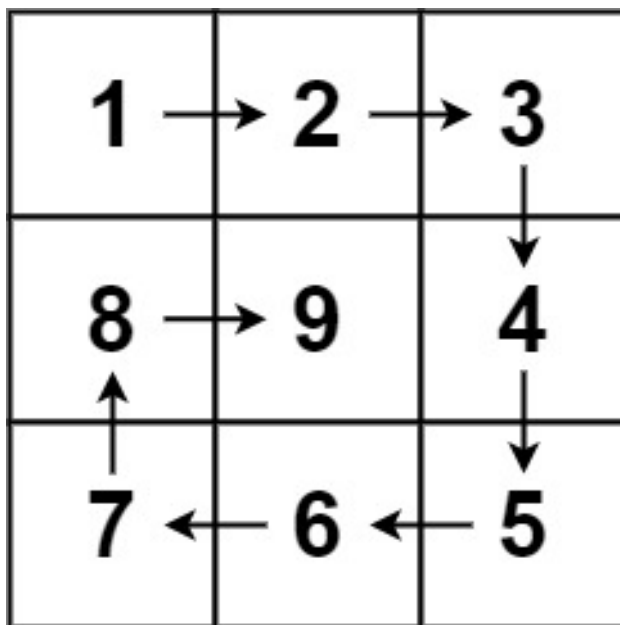
|      |        |      |                               |
|------|--------|------|-------------------------------|
| 唯一题序 | 12/158 | 本章顺序 | 12/14                         |
| 难度   | 中等     | 题目类型 | 主讲题                           |
| 出现章节 | 第1章 数组 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:647 |

LeetCode：https://leetcode.cn/problems/spiral-matrix-ii/

### 题目内容

给你一个正整数  $n$ ，生成一个包含 1 到  $n^2$  所有元素，且元素按顺时针顺序螺旋排列的  $n \times n$  正方形矩阵 `matrix`。

示例 1：



输入：n = 3

输出：[[1,2,3],[8,9,4],[7,6,5]]

示例 2：

输入：n = 1

输出：[[1]]

提示：

1  $1 \leq n \leq 20$

我的题目思路与考察范围

核心思路：按层维护上、下、左、右边界，依次填充四条边并向内收缩，直到把 1 到  $n^2$  全部写入矩阵。

关键不变量 / 状态：每层四段采用一致的左闭右开规则，角点只填一次，边界外位置均已完成。

复杂度：时间  $O(n^2)$ ，除输出矩阵外额外空间  $O(1)$

考察范围：矩阵模拟 · 分层遍历 · 边界收缩 · 循环不变量

易错点：奇数阶矩阵的中心格以及四条边的端点归属容易产生遗漏或重复。

## 54. 螺旋矩阵

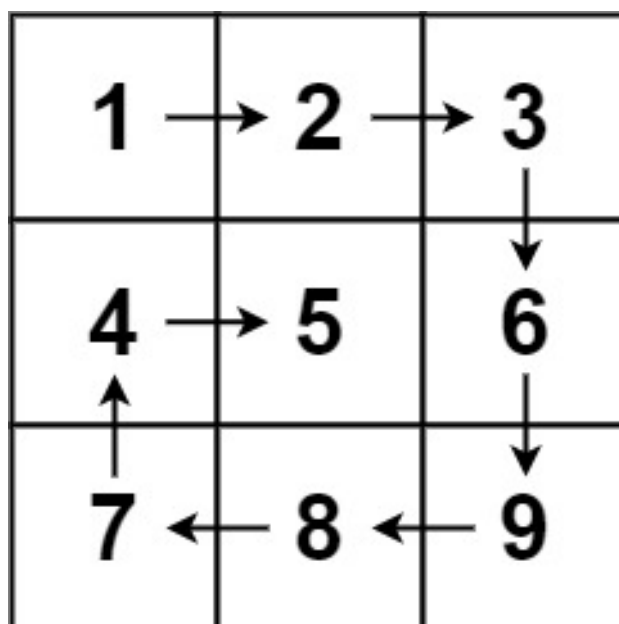
|      |        |      |                              |
|------|--------|------|------------------------------|
| 唯一题序 | 13/158 | 本章顺序 | 13/14                        |
| 难度   | 中等     | 题目类型 | 拓展/推荐                        |
| 出现章节 | 第1章 数组 | 源位置  | 1.《代码随想录》数组 ( V3.0 ) .md:758 |

LeetCode：https://leetcode.cn/problems/spiral-matrix/

### 题目内容

给你一个  $m$  行  $n$  列的矩阵 `matrix`，请按照 顺时针螺旋顺序，返回矩阵中的所有元素。

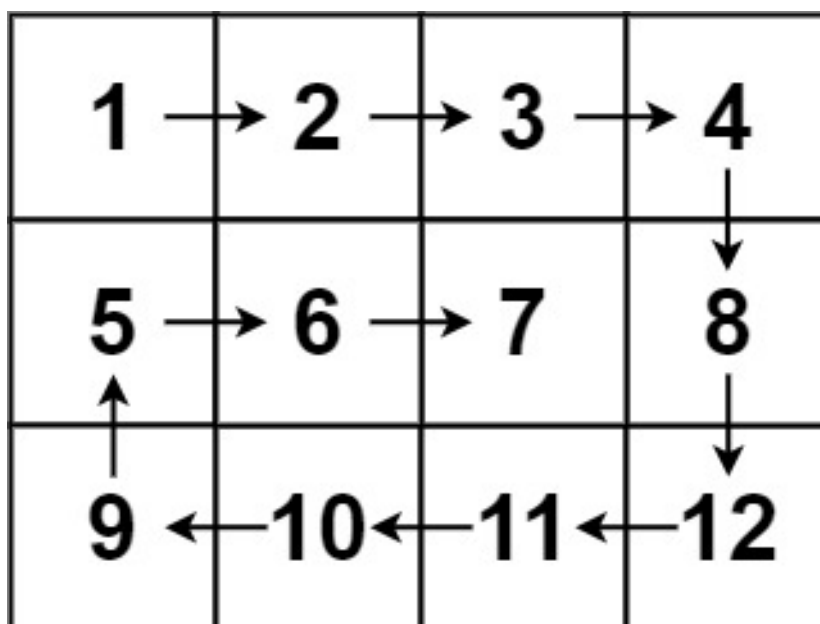
示例 1：



输入 : matrix = [[1,2,3],[4,5,6],[7,8,9]]

输出 : [1,2,3,6,9,8,7,4,5]

示例 2 :



输入 : matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]

输出 : [1,2,3,4,8,12,11,10,9,5,6,7]

提示 :

```
1 m == matrix.length
1 n == matrix[i].length
1 1 <= m, n <= 10
1 -100 <= matrix[i][j] <= 100
```

## 我的题目思路与考察范围

核心思路：用四个边界顺时针读取矩形矩阵，每完成一条边就收缩相应边界，并在读取下边或左边前检查边界是否仍有效。

关键不变量 / 状态：边界外的元素都已且仅已输出一次，边界内恰是尚未访问的子矩阵。

复杂度：时间  $O(mn)$ ，除返回结果外额外空间  $O(1)$

考察范围：矩阵遍历 · 边界收缩 · 螺旋顺序 · 退化矩阵

易错点：剩余单行或单列时若不再次检查边界，会重复输出同一批元素。

## 剑指 Offer 29. 顺时针打印矩阵

|      |        |      |                               |
|------|--------|------|-------------------------------|
| 唯一题序 | 14/158 | 本章顺序 | 14/14                         |
| 难度   | 简单     | 题目类型 | 拓展/推荐                         |
| 出现章节 | 第1章 数组 | 源位置  | 1. 《代码随想录》数组 ( V3.0 ) .md:759 |

LeetCode：https://leetcode.cn/problems/shun-shi-zhen-da-yin-ju-zhen-lcof/

## 题目内容

给定一个二维数组 `array`，请返回「螺旋遍历」该数组的结果。

螺旋遍历：从左上角开始，按照 向右、向下、向左、向上 的顺序依次提取元素，然后再进入内部一层重复相同的步骤，直到提取完所有元素。

示例 1：

输入：array = [[1,2,3],[8,9,4],[7,6,5]]  
输出：[1,2,3,4,5,6,7,8,9]

示例 2：

输入：array = [[1,2,3,4],[12,13,14,5],[11,16,15,6],[10,9,8,7]]  
输出：[1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16]

限制：

1  $0 \leq \text{array.length} \leq 100$

1  $0 \leq \text{array}[i].\text{length} \leq 100$

注意：本题与主站 54 题相同：https://leetcode.cn/problems/spiral-matrix/

## 我的题目思路与考察范围

核心思路：将矩阵外圈按上边、右边、下边、左边依次加入一维结果，再逐层缩小边界，得到顺时针打印序列。

关键不变量 / 状态：每轮开始时边界框住全部未打印元素，框外元素已按螺旋顺序加入结果。

复杂度：时间  $O(mn)$ ，除返回结果外额外空间  $O(1)$

考察范围：剑指 Offer · 矩阵模拟 · 螺旋遍历 · 边界判断

易错点：空矩阵和收缩后只剩单行或单列的情况必须避免二次访问。

# 第 2 章 链表

本章 7 道去重题目，按源文件首次出现顺序排列。

## 203. 移除链表元素

|      |        |      |                                |
|------|--------|------|--------------------------------|
| 唯一题序 | 15/158 | 本章顺序 | 1/7                            |
| 难度   | 简单     | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:153 |

LeetCode：https://leetcode.cn/problems/remove-linked-list-elements/

### 题目内容

给你一个链表的头节点

head

和一个整数

val

，请你删除链表中所有满足

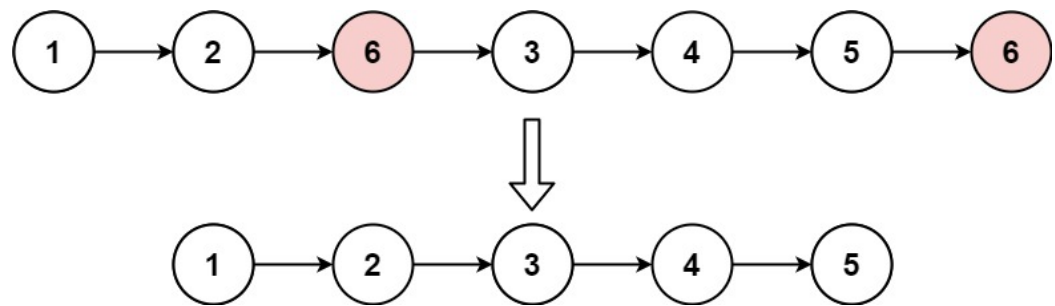
Node.val == val

的节点，并返回

新的头节点

。

示例 1：



输入：head = [1,2,6,3,4,5,6], val = 6  
输出：[1,2,3,4,5]

示例 2：

输入：head = [], val = 1  
输出：[]

示例 3：

输入：head = [7,7,7,7], val = 7  
输出：[]

提示：

1 列表中的节点数目在范围 [0, 10<sup>4</sup>] 内

```
1 1 <= Node.val <= 50
```

```
1 0 <= val <= 50
```

我的题目思路与考察范围

核心思路：在头节点前设置虚拟节点，始终检查 `cur.next`；命中 `val` 就跨过该节点，否则推进 `cur`，从而统一删除头部和中间节点。

关键不变量 / 状态：cur 之前的链表已清除所有目标值，`cur.next` 是下一待检查节点。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：单链表 · 虚拟头节点 · 节点删除 · 指针重连

易错点：删除后不能立即推进 `cur`，否则连续出现的目标节点会被跳过。

## 707. 设计链表

|      |        |      |                                |
|------|--------|------|--------------------------------|
| 唯一题序 | 16/158 | 本章顺序 | 2/7                            |
| 难度   | 中等     | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:293 |

LeetCode：https://leetcode.cn/problems/design-linked-list/

### 题目内容

你可以选择使用单链表或者双链表，设计并实现自己的链表。

单链表中的节点应该具备两个属性：`val` 和 `next`。`val` 是当前节点的值，`next` 是指向下一个节点的指针/引用。

如果是双向链表，则还需要属性 `prev` 以指示链表中的上一个节点。假设链表中的所有节点下标从 0 开始。

实现 `MyLinkedList` 类：

- 1 `MyLinkedList()` 初始化 `MyLinkedList` 对象。
- 1 `int get(int index)` 获取链表中下标为 `index` 的节点的值。如果下标无效，则返回 -1。
- 1 `void addAtHead(int val)` 将一个值为 `val` 的节点插入到链表中第一个元素之前。在插入完成后，新节点会成为链表的第一个节点。
- 1 `void addAtTail(int val)` 将一个值为 `val` 的节点追加到链表中作为链表的最后一个元素。
- 1 `void addAtIndex(int index, int val)` 将一个值为 `val` 的节点插入到链表中下标为 `index` 的节点之前。如果 `index` 等于链表的长度，那么该节点会被追加到链表的末尾。如果 `index` 比长度更大，该节点将不会插入到链表中。
- 1 `void deleteAtIndex(int index)` 如果下标有效，则删除链表中下标为 `index` 的节点。

示例：

输入

```
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]
```

```
[[], [1], [3], [1, 2], [1], [1], [1]]
```

输出

```
[null, null, null, null, 2, null, 3]
```

解释

```
MyLinkedList myLinkedList = new MyLinkedList();
```

```
myLinkedList.addAtHead(1);
```

```
myLinkedList.addAtTail(3);
```

```
myLinkedList.addAtIndex(1, 2); // 链表变为 1->2->3
```

```
myLinkedList.get(1); // 返回 2
```

```
myLinkedList.deleteAtIndex(1); // 现在，链表变为 1->3
```

```
myLinkedList.get(1); // 返回 3
```

提示：

1 0 &lt;= index, val &lt;= 1000

1 请不要使用内置的 LinkedList 库。

1 调用 get、addAtHead、addAtTail、addAtIndex 和 deleteAtIndex 的次数不超过 2000。

我的题目思路与考察范围

核心思路：用虚拟头节点和 size 维护链表；所有按下标访问、插入和删除先校验范围，再从哨兵节点走到对应前驱完成重连。

关键不变量 / 状态：size 始终等于可达的数据节点数，哨兵节点不计入下标。

复杂度：get 和按索引增删最坏时间  $O(n)$ ，头插时间  $O(1)$ ，空间  $O(n)$

考察范围：链表设计 · 哨兵节点 · 索引校验 · 封装 · 数据结构实现

易错点：addAtIndex 允许 index == size 追加，负下标按题意视为头插，而 index > size 应直接忽略。

## 206. 反转链表

|      |                |      |                                |
|------|----------------|------|--------------------------------|
| 唯一题序 | 17/158         | 本章顺序 | 3/7                            |
| 难度   | 简单             | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表、第5章 双指针 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:445 |

LeetCode：https://leetcode.cn/problems/reverse-linked-list/

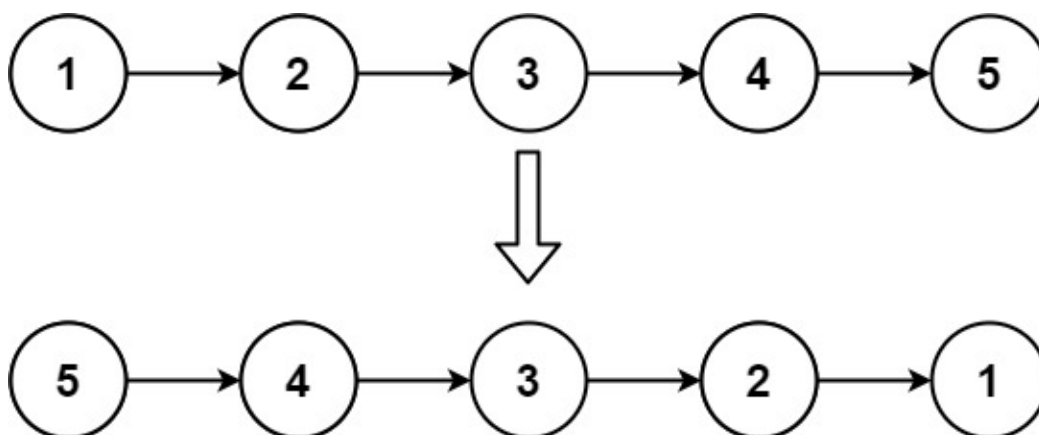
### 题目内容

给你单链表的头节点

head

，请你反转链表，并返回反转后的链表。

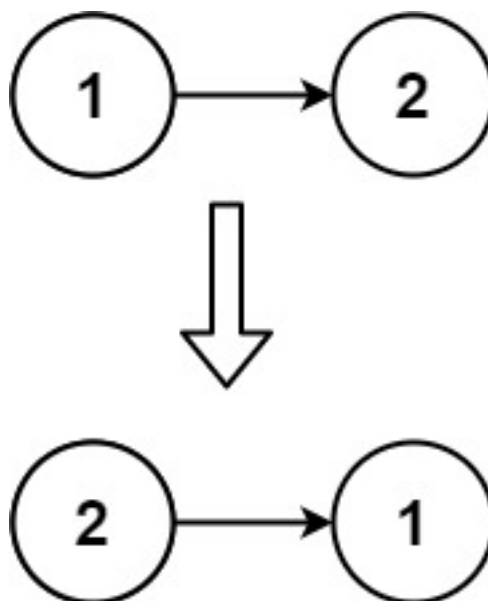
示例 1：



输入：head = [1,2,3,4,5]

输出：[5,4,3,2,1]

示例 2：



输入：head = [1,2]  
输出：[2,1]

示例 3：

输入：head = []  
输出：[]

提示：

1 链表中节点的数目范围是 [0, 5000]

1  $-5000 \leq \text{Node.val} \leq 5000$

进阶：链表可以选用迭代或递归方式完成反转。你能否用两种方法解决这道题？

我的题目思路与考察范围

核心思路：迭代保存 curr.next，再把 curr.next 指向 prev，最后同步推进 prev 与 curr，逐条反转链表方向。

关键不变量 / 状态：prev 是已反转前缀的头，curr 是尚未处理后缀的头，两部分节点不丢失。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：单链表 · 指针反转 · 迭代 · 循环不变量

易错点：改写 curr.next 前若未保存原 next，尚未处理的整段链表会丢失。

## 24. 两两交换链表中的节点

|      |        |      |                                |
|------|--------|------|--------------------------------|
| 唯一题序 | 18/158 | 本章顺序 | 4/7                            |
| 难度   | 中等     | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:631 |

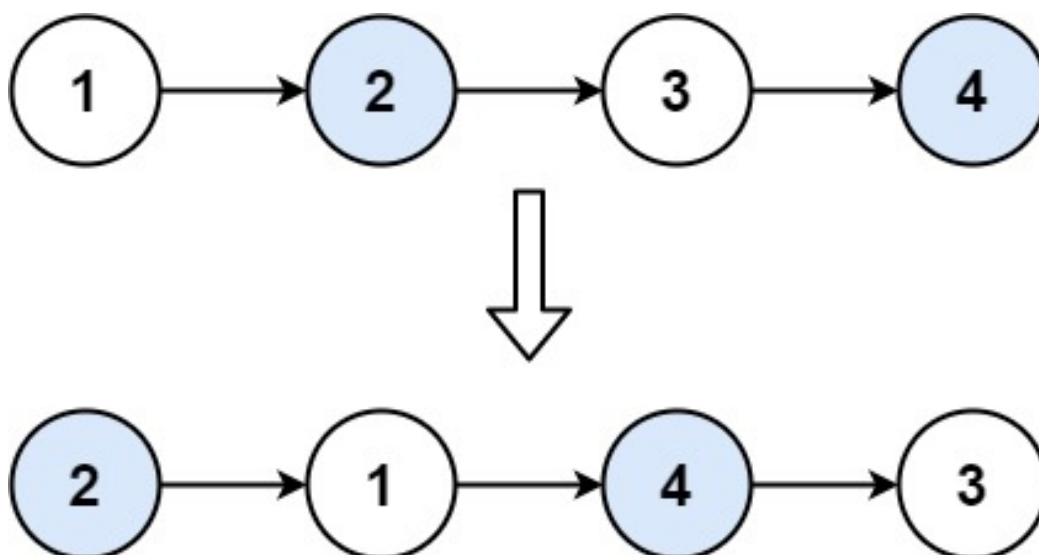
LeetCode：https://leetcode.cn/problems/swap-nodes-in-pairs/

### 题目内容

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在\*\*不修改节点内部的值\*\*的情况下完成本题（即，只能进行节点交换）。

示例 1：





输入：head = [1,2,3,4]  
输出：[2,1,4,3]

示例 2：

输入：head = []  
输出：[]

示例 3：

输入：head = [1]  
输出：[1]

提示：

1 链表中节点的数目在范围 [0, 100] 内

1  $0 \leq \text{Node.val} \leq 100$

我的题目思路与考察范围

核心思路：从虚拟头节点出发，每次取得相邻的 first、second 和后继节点，通过三次重连交换这一对，再把前驱移动到交换后的尾部。

关键不变量 / 状态：前驱节点之前的所有节点已经两两交换完成，前驱的 next 指向下一对的首节点。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：链表重连 · 虚拟头节点 · 成对处理 · 指针顺序

易错点：必须先保存第二个节点及其后继，奇数个节点时最后一个节点保持原位。

## 19. 删除链表的倒数第 N 个结点

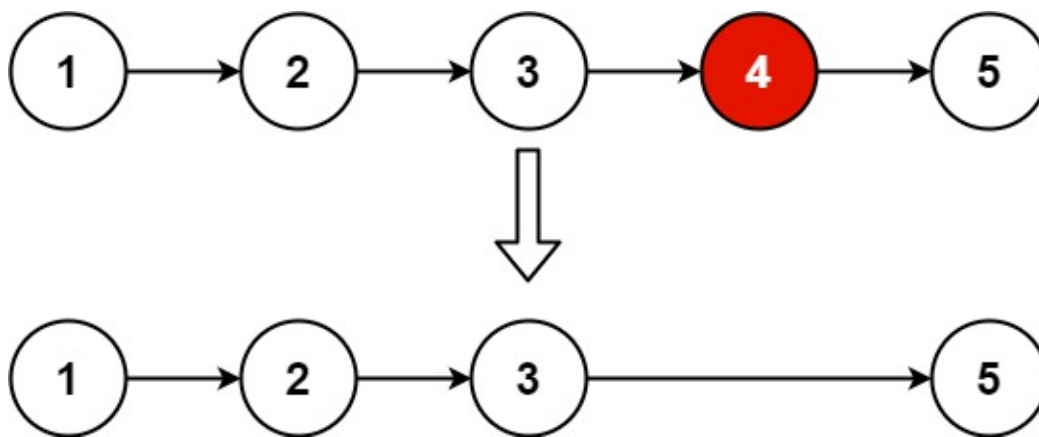
|      |                |      |                                |
|------|----------------|------|--------------------------------|
| 唯一题序 | 19/158         | 本章顺序 | 5/7                            |
| 难度   | 中等             | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表、第5章 双指针 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:708 |

LeetCode：https://leetcode.cn/problems/remove-nth-node-from-end-of-list/

### 题目内容

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1：



输入：head = [1,2,3,4,5], n = 2  
输出：[1,2,3,5]

示例 2：

输入：head = [1], n = 1  
输出：[]

示例 3：

输入：head = [1,2], n = 1  
输出：[1]

提示：

- 1 链表中结点的数目为 sz
- 1  $1 \leq sz \leq 30$
- 1  $0 \leq \text{Node.val} \leq 100$
- 1  $1 \leq n \leq sz$

进阶：你能尝试使用一趟扫描实现吗？

我的题目思路与考察范围

核心思路：从虚拟头节点让快指针先走  $n + 1$  步，再与慢指针同步前进；快指针到空时，慢指针正好位于待删节点的前驱。

关键不变量 / 状态：同步阶段快慢指针间始终保持  $n + 1$  条 next 边的距离。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：快慢指针 · 链表删除 · 倒数第  $N$  个 · 虚拟头节点

易错点：少走或多走一步都会让慢指针错位，删除头节点时尤其需要虚拟头节点。

## 面试题 02.07. 链表相交

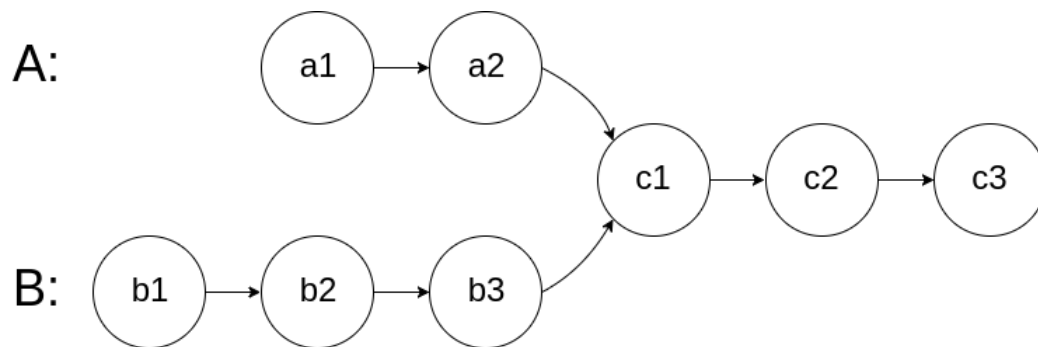
|      |                |      |                                |
|------|----------------|------|--------------------------------|
| 唯一题序 | 20/158         | 本章顺序 | 6/7                            |
| 难度   | 简单             | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表、第5章 双指针 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:792 |

LeetCode：https://leetcode.cn/problems/intersection-of-two-linked-lists-lcci/

### 题目内容

给你两个单链表的头节点 headA 和 headB，请你找出并返回两个单链表相交的起始节点。如果两个链表没有交点，返回 null。

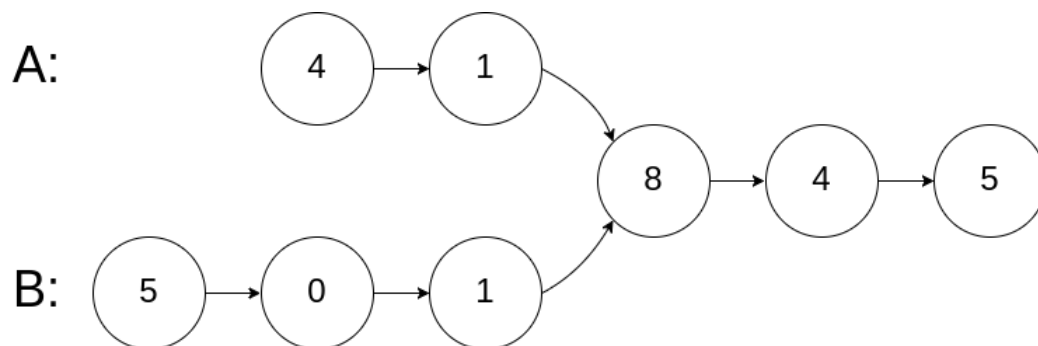
图示两个链表在节点 c1 开始相交：



题目数据 保证 整个链式结构中不存在环。

注意，函数返回结果后，链表必须 保持其原始结构。

示例 1：



输入：intersectVal = 8, listA = [4,1,8,4,5], listB = [5,0,1,8,4,5], skipA = 2, skipB = 3

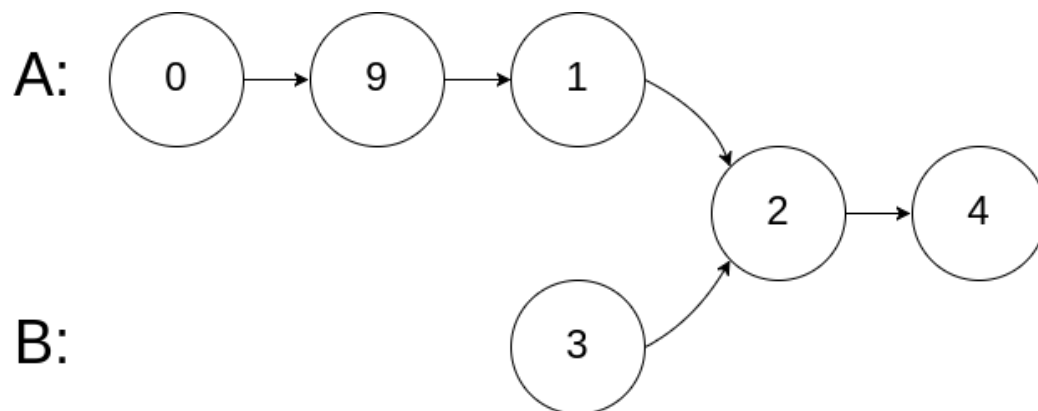
输出：Intersected at '8'

解释：相交节点的值为 8（注意，如果两个链表相交则不能为 0）。

从各自的表头开始算起，链表 A 为 [4,1,8,4,5]，链表 B 为 [5,0,1,8,4,5]。

在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。

示例 2：



输入：intersectVal = 2, listA = [0,9,1,2,4], listB = [3,2,4], skipA = 3, skipB = 1

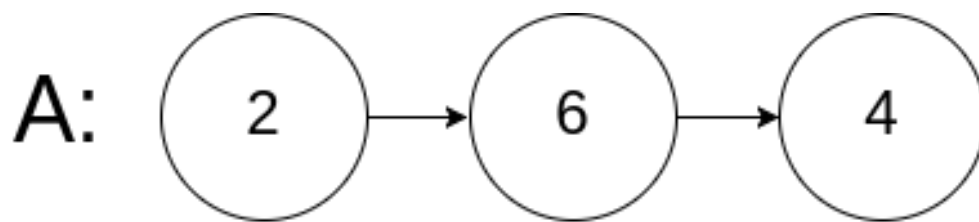
输出：Intersected at '2'

解释：相交节点的值为 2（注意，如果两个链表相交则不能为 0）。

从各自的表头开始算起，链表 A 为 [0,9,1,2,4]，链表 B 为 [3,2,4]。

在 A 中，相交节点前有 3 个节点；在 B 中，相交节点前有 1 个节点。

示例 3：



输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2

输出：null

解释：从各自的表头开始算起，链表 A 为 [2,6,4]，链表 B 为 [1,5]。

由于这两个链表不相交，所以 intersectVal 必须为 0，而 skipA 和 skipB 可以是任意值。  
这两个链表不相交，因此返回 null。

提示：

- 1 listA 中节点数目为 m
- 1 listB 中节点数目为 n
- 1  $0 \leq m, n \leq 3 \times 10^4$
- 1  $1 \leq \text{Node.val} \leq 10^5$
- 1  $0 \leq \text{skipA} \leq m$
- 1  $0 \leq \text{skipB} \leq n$
- 1 如果 listA 和 listB 没有交点，intersectVal 为 0
- 1 如果 listA 和 listB 有交点， $\text{intersectVal} == \text{listA}[\text{skipA} + 1] == \text{listB}[\text{skipB} + 1]$

进阶：你能否设计一个时间复杂度  $O(n)$ 、仅用  $O(1)$  内存的解决方案？

我的题目思路与考察范围

核心思路：两个指针分别遍历 A、B，走到末尾后切换到另一条链表头；它们走过相同总路程后会在公共节点相遇，或同时为空。

关键不变量 / 状态：切换后两指针需要走的剩余总长度相同，因此消除了两条链表的长度差。

复杂度：时间  $O(m + n)$ ，额外空间  $O(1)$

考察范围：链表相交 · 双指针 · 长度对齐 · 节点身份

易错点：应比较节点对象是否相同而不是节点值是否相等。

## 142. 环形链表 II

|      |                |      |                                |
|------|----------------|------|--------------------------------|
| 唯一题序 | 21/158         | 本章顺序 | 7/7                            |
| 难度   | 中等             | 题目类型 | 主讲题                            |
| 出现章节 | 第2章 链表、第5章 双指针 | 源位置  | 2. 《代码随想录》链表. ( V3.0 ) .md:888 |

LeetCode : <https://leetcode.cn/problems/linked-list-cycle-ii/>

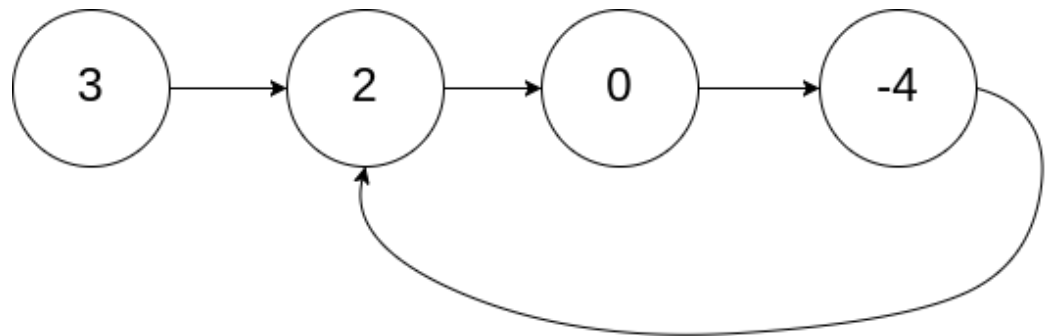
### 题目内容

给定一个链表的头节点 head ，返回链表开始入环的第一个节点。如果链表无环，则返回 null 。

如果链表中有某个节点，可以通过连续跟踪 next 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。如果 pos 是 -1 ，则在该链表中没有环。注意：pos 不作为参数进行传递，仅仅是为了标识链表的实际情况。

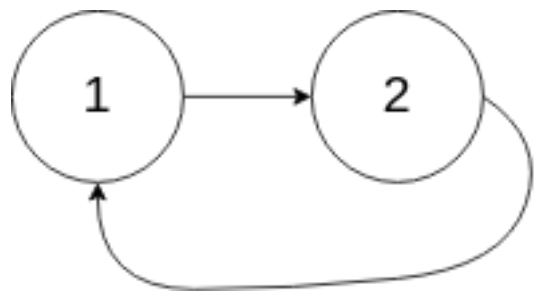
不允许修改 链表。

示例 1：



输入：head = [3,2,0,-4], pos = 1  
输出：返回索引为 1 的链表节点  
解释：链表中有一个环，其尾部连接到第二个节点。

示例 2：



输入：head = [1,2], pos = 0  
输出：返回索引为 0 的链表节点  
解释：链表中有一个环，其尾部连接到第一个节点。

示例 3：



输入：head = [1], pos = -1  
输出：返回 null  
解释：链表中没有环。

提示：

- 1 链表中节点的数目范围在范围  $[0, 10^4]$  内
- 1  $-10^5 \leq \text{Node.val} \leq 10^5$
- 1 pos 的值为 -1 或者链表中的一个有效索引

进阶：你是否可以使用  $O(1)$  空间解决此题？

我的题目思路与考察范围

核心思路：先用 Floyd 快慢指针判断并定位环内相遇点；随后一个指针回到头节点，两者同速前进，再次相遇处就是入环节点。

关键不变量 / 状态：第一次相遇后，头到入口的距离等于相遇点沿环前进到入口的等价距离。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：Floyd 判圈 · 快慢指针 · 链表环入口 · 距离推导

易错点：第一次快慢指针相遇的位置通常不是环入口，不能直接返回。

## 第 3 章 哈希表

本章 9 道去重题目，按源文件首次出现顺序排列。

### 242. 有效的字母异位词

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 22/158  | 本章顺序 | 1/9                            |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:141 |

LeetCode : <https://leetcode.cn/problems/valid-anagram/>

#### 题目内容

给定两个字符串  $s$  和  $t$ ，编写一个函数来判断  $t$  是否是  $s$  的字母异位词。

示例 1:

输入:  $s = \text{"anagram"}, t = \text{"nagaram"}$   
输出: true

示例 2:

输入:  $s = \text{"rat"}, t = \text{"car"}$   
输出: false

提示:

1  $1 \leq s.length, t.length \leq 5 * 10^4$

1  $s$  和  $t$  仅包含小写字母

进阶: 如果输入字符串包含 unicode 字符怎么办? 你能否调整你的解法来应对这种情况?

我的题目思路与考察范围

核心思路: 对  $s$  的字符频次加一、对  $t$  的字符频次减一, 最终所有计数为零才说明两个字符串含有完全相同的字符多重集合。

关键不变量 / 状态: 计数表始终表示已处理前缀中  $s$  相对  $t$  的字符频次差。

复杂度: 时间  $O(|s| + |t|)$ , 空间  $O(|\Sigma|)$ ; 固定小写字母表时为  $O(1)$

考察范围: 哈希计数 · 字符频次 · 多重集合 · 字符串

易错点: 长度相同并不代表是异位词, 且不能只比较出现过哪些字符而忽略次数。

### 349. 两个数组的交集

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 23/158  | 本章顺序 | 2/9                            |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:224 |

LeetCode : <https://leetcode.cn/problems/intersection-of-two-arrays/>

#### 题目内容

给定两个数组  $nums1$  和  $nums2$ , 返回 它们的 交集。输出结果中的每个元素一定是 唯一 的。我们可以 不考虑输出结果的顺序。

示例 1：

输入：nums1 = [1,2,2,1], nums2 = [2,2]  
输出：[2]

示例 2：

输入：nums1 = [4,9,5], nums2 = [9,4,9,8,4]  
输出：[9,4]  
解释：[4,9] 也是可通过的

提示：

1 1 <= nums1.length, nums2.length <= 1000

1 0 <= nums1[i], nums2[i] <= 1000

我的题目思路与考察范围

核心思路：把一个数组放入集合，扫描另一个数组收集存在于集合中的值，并用结果集合保证每个交集元素只出现一次。

关键不变量 / 状态：结果中恰好包含已扫描部分与第一个数组共有的不同值。

复杂度：期望时间  $O(m + n)$ ，空间  $O(m + n)$

考察范围：哈希集合 · 数组交集 · 去重 · 集合运算

易错点：题目要求的是不同元素的集合，不能按输入中的重复次数重复输出。

## 350. 两个数组的交集 II

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 24/158  | 本章顺序 | 3/9                           |
| 难度   | 简单      | 题目类型 | 拓展/推荐                         |
| 出现章节 | 第3章 哈希表 | 源位置  | 3.《代码随想录》哈希表 ( V3.0 ) .md:330 |

LeetCode：https://leetcode.cn/problems/intersection-of-two-arrays-ii/

### 题目内容

给你两个整数数组 nums1 和 nums2，请你以数组形式返回两数组的交集。返回结果中每个元素出现的次数，应与元素在两个数组中都出现的次数一致（如果出现次数不一致，则考虑取较小值）。可以不考虑输出结果的顺序。

示例 1：

输入：nums1 = [1,2,2,1], nums2 = [2,2]  
输出：[2,2]

示例 2:

输入：nums1 = [4,9,5], nums2 = [9,4,9,8,4]  
输出：[4,9]

提示：

1 1 <= nums1.length, nums2.length <= 1000

1 0 <= nums1[i], nums2[i] <= 1000

进阶：

1 如果给定的数组已经排好序呢？你将如何优化你的算法？

1 如果 nums1 的大小比 nums2 小，哪种方法更优？

1 如果 nums2 的元素存储在磁盘上，内存是有限的，并且你不能一次加载所有的元素到内存中，你该怎么办？



## 我的题目思路与考察范围

核心思路：统计较短数组中每个值的剩余次数，扫描另一数组时若计数为正就加入结果并减一，从而保留交集重复次数。

关键不变量 / 状态：频次表记录每个值尚可匹配的次数，结果是已扫描部分的正确多重集合交集。

复杂度：期望时间  $O(m + n)$ ，空间  $O(\min(m, n))$

考察范围：哈希计数 · 多重集合 · 数组交集 · 频次消耗

易错点：与无重复版本不同，匹配后必须递减计数，不能只使用普通集合。

## 202. 快乐数

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 25/158  | 本章顺序 | 4/9                            |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:336 |

LeetCode : <https://leetcode.cn/problems/happy-number/>

## 题目内容

编写一个算法来判断一个数  $n$  是不是快乐数。

「快乐数」定义为：

- 1 对于一个正整数，每一次将该数替换为它每个位置上的数字的平方和。
- 1 然后重复这个过程直到这个数变为 1，也可能是 无限循环 但始终变不到 1。
- 1 如果这个过程 结果为 1，那么这个数就是快乐数。

如果  $n$  是 快乐数 就返回 true ；不是，则返回 false 。

示例 1：

输入：n = 19  
输出：true  
解释：  
 $1^2 + 9^2 = 82$   
 $8^2 + 2^2 = 68$   
 $6^2 + 8^2 = 100$   
 $1^2 + 0^2 + 0^2 = 1$

示例 2：

输入：n = 2  
输出：false

提示：

- 1  $1 \leq n \leq 2^{31} - 1$

## 我的题目思路与考察范围

核心思路：把“各位数字平方和”视为确定的状态转移，用 Floyd 快慢指针判断序列最终到达 1，还是进入不含 1 的循环。

关键不变量 / 状态：slow 每次走一步、fast 每次走两步；若二者相遇，则已进入同一状态环。

复杂度：时间  $O(\log n)$  量级，额外空间  $O(1)$

考察范围：状态循环 · Floyd 判圈 · 数位运算 · 哈希集合替代方案

易错点：每轮计算平方和都要从零开始，并把到达 1 与普通环相遇区分开。

## 1. 两数之和

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 26/158  | 本章顺序 | 5/9                            |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:402 |

LeetCode : <https://leetcode.cn/problems/two-sum/>

### 题目内容

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 和为目标值 `target` 的那 两个 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

示例 1：

输入：nums = [2,7,11,15], target = 9  
输出：[0,1]  
解释：因为 nums[0] + nums[1] == 9，返回 [0, 1]。

示例 2：

输入：nums = [3,2,4], target = 6  
输出：[1,2]

示例 3：

输入：nums = [3,3], target = 6  
输出：[0,1]

提示：

- 1 2 <= nums.length <= 10 ^4
- 1 -10 ^9 <= nums[i] <= 10 ^9
- 1 -10 ^9 <= target <= 10 ^9
- 1 只会存在一个有效答案

进阶：你可以想出一个时间复杂度小于  $O(n^2)$  的算法吗？

我的题目思路与考察范围

核心思路：单次扫描数组，在哈希表中查询 `target - nums[i]` 是否已出现；命中就返回其旧下标和当前下标，否则记录当前值。

关键不变量 / 状态：处理下标 `i` 时，哈希表只包含 `i` 之前元素到其下标的映射。

复杂度：期望时间  $O(n)$ ，空间  $O(n)$

考察范围：哈希表 · 补数查找 · 单遍扫描 · 下标返回

易错点：应先查询补数再插入当前元素，避免把同一个数组位置使用两次。

## 454. 四数相加 II

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 27/158  | 本章顺序 | 6/9                            |
| 难度   | 中等      | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:524 |

LeetCode : <https://leetcode.cn/problems/4sum-ii/>

## 题目内容

给你四个整数数组 `nums1`、`nums2`、`nums3` 和 `nums4`，数组长度都是 `n`，请你计算有多少个元组  $(i, j, k, l)$  能满足：

1 `0 ≤ i, j, k, l < n`

1 `nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0`

示例 1：

输入：`nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]`

输出：2

解释：

两个元组如下：

1.  $(0, 0, 0, 1) \rightarrow \text{nums1}[0] + \text{nums2}[0] + \text{nums3}[0] + \text{nums4}[1] = 1 + (-2) + (-1) + 2 = 0$

2.  $(1, 1, 0, 0) \rightarrow \text{nums1}[1] + \text{nums2}[1] + \text{nums3}[0] + \text{nums4}[0] = 2 + (-1) + (-1) + 0 = 0$

示例 2：

输入：`nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]`

输出：1

提示：

1 `n == nums1.length`

1 `n == nums2.length`

1 `n == nums3.length`

1 `n == nums4.length`

1 `1 ≤ n ≤ 200`

1 `-228 ≤ nums1[i], nums2[i], nums3[i], nums4[i] ≤ 228`

我的题目思路与考察范围

核心思路：枚举前两个数组的所有数对和并记录出现次数，再枚举后两个数组的数对和，把其相反数在表中的次数累加到答案。

关键不变量 / 状态：哈希表保存 A、B 每一种和的完整组合数，后半每个数对都贡献所有可配对的前半组合。

复杂度：时间  $O(n^2)$ ，空间  $O(n^2)$

考察范围：分组枚举 · 数对和 · 哈希计数 · 组合数

易错点：必须累计相同数对和的出现次数而不是只记录是否存在。

## 383. 赎金信

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 28/158  | 本章顺序 | 7/9                           |
| 难度   | 简单      | 题目类型 | 主讲题                           |
| 出现章节 | 第3章 哈希表 | 源位置  | 3.《代码随想录》哈希表 ( V3.0 ) .md:605 |

LeetCode : <https://leetcode.cn/problems/ransom-note/>

## 题目内容

给你两个字符串：`ransomNote` 和 `magazine`，判断 `ransomNote` 能不能由 `magazine` 里面的字符构成。

如果可以，返回 `true`；否则返回 `false`。

`magazine` 中的每个字符只能在 `ransomNote` 中使用一次。

示例 1：

输入：ransomNote = "a", magazine = "b"  
输出：false

示例 2：

输入：ransomNote = "aa", magazine = "ab"  
输出：false

示例 3：

输入：ransomNote = "aa", magazine = "aab"  
输出：true

提示：

1 1 <= ransomNote.length, magazine.length <= 10 ^5

1 ransomNote 和 magazine 由小写英文字母组成

我的题目思路与考察范围

核心思路：先统计 magazine 可提供的字符数量，再逐个消耗 ransomNote 的需求；任何字符库存变为负数就无法构造。

关键不变量 / 状态：计数表表示处理当前前缀后，杂志中每个字符尚可使用的数量。

复杂度：时间  $O(m + n)$ ，空间  $O(|\Sigma|)$ ；固定小写字母表时为  $O(1)$

考察范围：字符计数 · 哈希表 · 资源消耗 · 字符串

易错点：计数方向不能写反，杂志中每个字符实例最多只能使用一次。

## 15. 三数之和

|      |                 |      |                                |
|------|-----------------|------|--------------------------------|
| 唯一题序 | 29/158          | 本章顺序 | 8/9                            |
| 难度   | 中等              | 题目类型 | 主讲题                            |
| 出现章节 | 第3章 哈希表、第5章 双指针 | 源位置  | 3. 《代码随想录》哈希表 ( V3.0 ) .md:703 |

LeetCode：https://leetcode.cn/problems/3sum/

### 题目内容

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足  $i \neq j$ 、 $i \neq k$  且  $j \neq k$ ，同时还满足  $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组。

示例 1：

输入：nums = [-1,0,1,2,-1,-4]  
输出：[[-1,-1,2],[-1,0,1]]  
解释：  
 $nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0$ 。  
 $nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0$ 。  
 $nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0$ 。  
不同的三元组是  $[-1,0,1]$  和  $[-1,-1,2]$ 。  
注意，输出的顺序和三元组的顺序并不重要。

示例 2：

输入：nums = [0,1,1]  
输出：[]  
解释：唯一可能的三元组和不为 0。

示例 3：

输入：nums = [0,0,0]  
输出：[[0,0,0]]  
解释：唯一可能的三元组和为 0。

提示：

1 3 <= nums.length <= 3000

1 -10 ^5 <= nums[i] <= 10 ^5

我的题目思路与考察范围

核心思路：排序后固定第一个数，在其右侧用左右指针寻找和为相反数的数对；命中后跳过重复值，得到不重复的三元组。

关键不变量 / 状态：对固定 i，左右区间按和的大小单调收缩，所有已跨过位置都不可能再形成新的有效三元组。

复杂度：时间  $O(n^2)$ ，排序辅助空间通常  $O(\log n)$ ，输出空间不计

考察范围：排序 · 双指针 · 三元组 · 去重 · 剪枝

易错点：去重应按值跳过相邻重复项，同时不能因指针移动顺序错误漏掉合法组合。

## 18. 四数之和

|      |                 |      |                               |
|------|-----------------|------|-------------------------------|
| 唯一题序 | 30/158          | 本章顺序 | 9/9                           |
| 难度   | 中等              | 题目类型 | 主讲题                           |
| 出现章节 | 第3章 哈希表、第5章 双指针 | 源位置  | 3.《代码随想录》哈希表 ( V3.0 ) .md:944 |

LeetCode：https://leetcode.cn/problems/4sum/

### 题目内容

给你一个由 n 个整数组成的数组 nums，和一个目标值 target。请你找出并返回满足下述全部条件且 不重复 的四元组 [nums[a], nums[b], nums[c], nums[d]]（若两个四元组元素一一对应，则认为两个四元组重复）：

1 0 <= a, b, c, d < n

1 a、b、c 和 d 互不相同

1 nums[a] + nums[b] + nums[c] + nums[d] == target

你可以按 任意顺序 返回答案。

示例 1：

输入：nums = [1,0,-1,0,-2,2], target = 0  
输出：[[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

示例 2：

输入：nums = [2,2,2,2,2], target = 8  
输出：[[2,2,2,2]]

提示：

1 1 <= nums.length <= 200

1 -10 ^9 <= nums[i] <= 10 ^9

1 -10 ^9 <= target <= 10 ^9

#### 我的题目思路与考察范围

核心思路：排序后固定前两个数，再用左右指针搜索剩余两数；每一层都按值去重，并用宽整数计算四数和及安全剪枝。

关键不变量 / 状态：固定  $i$ 、 $j$  后，左右指针依据当前和与  $target$  的比较单调排除不可能区间。

复杂度：时间  $O(n^3)$ ，排序辅助空间通常  $O(\log n)$ ，输出空间不计

考察范围：排序 · 双指针 · 四元组 · 分层去重 · 溢出处理

易错点：整型溢出和未考虑  $target$  符号的过度剪枝都会导致错误结果。

## 第 4 章 字符串

本章 7 道去重题目，按源文件首次出现顺序排列。

### 344. 反转字符串

|      |                 |      |                              |
|------|-----------------|------|------------------------------|
| 唯一题序 | 31/158          | 本章顺序 | 1/7                          |
| 难度   | 简单              | 题目类型 | 主讲题                          |
| 出现章节 | 第4章 字符串、第5章 双指针 | 源位置  | 4.《代码随想录》字符串 ( V3.0 ) .md:21 |

LeetCode : <https://leetcode.cn/problems/reverse-string/>

#### 题目内容

编写一个函数，其作用是将输入的字符串反转过来。输入字符串以字符数组 `s` 的形式给出。

不要给另外的数组分配额外的空间，你必须 原地 修改输入数组、使用  $O(1)$  的额外空间解决这一问题。

示例 1：

输入：s = ["h","e","l","l","o"]  
输出：["o","l","l","e","h"]

示例 2：

输入：s = ["H","a","n","n","a","h"]  
输出：["h","a","n","n","a","H"]

提示：

1 `1 <= s.length <= 10 ^5`

1 `s[i]` 都是 ASCII 码表中的可打印字符

我的题目思路与考察范围

核心思路：从字符数组两端同时向中间移动，每轮交换左右字符，直接在原数组中完成反转。

关键不变量 / 状态：左右指针外侧的字符都已位于最终位置，指针之间仍待处理。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$

考察范围：双指针 · 原地交换 · 字符数组 · 对称位置

易错点：题目要求原地修改输入数组，创建完整反转副本不符合常数额外空间要求。

### 541. 反转字符串 II

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 32/158  | 本章顺序 | 2/7                           |
| 难度   | 简单      | 题目类型 | 主讲题                           |
| 出现章节 | 第4章 字符串 | 源位置  | 4.《代码随想录》字符串 ( V3.0 ) .md:149 |

LeetCode : <https://leetcode.cn/problems/reverse-string-ii/>

#### 题目内容

给定一个字符串 `s` 和一个整数 `k`，从字符串开头算起，每计数至 `2k` 个字符，就反转这 `2k` 字符中的前 `k` 个字符，再重新计数。

- 1 如果剩余字符少于  $k$  个，则将剩余字符全部反转。
- 1 如果剩余字符小于  $2k$  但大于或等于  $k$  个，则反转前  $k$  个字符，其余字符保持原样。

示例 1：

输入：s = "abcdefg", k = 2  
输出："bacdfeg"

示例 2：

输入：s = "abcd", k = 2  
输出："bacd"

提示：

- 1  $1 \leq s.length \leq 10^4$
- 1 s 仅由小写英文组成
- 1  $1 \leq k \leq 10^4$

我的题目思路与考察范围

核心思路：以  $2k$  为步长分块，每块只反转起点后的前  $k$  个字符；末尾不足  $k$  个时全部反转，介于  $k$  与  $2k$  时仍只反转前  $k$  个。

关键不变量 / 状态：进入下一块前，当前块已严格按题目规则处理，之前的块不会再改变。

复杂度：时间  $O(n)$ ，可变字符数组上额外空间  $O(1)$

考察范围：分段反转 · 双指针 · 字符串模拟 · 尾段边界

易错点：最后一段的右端应取  $\min(\text{start} + k, n)$ ，不能假定长度总是  $2k$ 。

## 剑指 Offer 05. 替换空格

|      |                 |      |                               |
|------|-----------------|------|-------------------------------|
| 唯一题序 | 33/158          | 本章顺序 | 3/7                           |
| 难度   | 简单              | 题目类型 | 主讲题                           |
| 出现章节 | 第4章 字符串、第5章 双指针 | 源位置  | 4.《代码随想录》字符串 ( V3.0 ) .md:263 |

LeetCode：https://leetcode.cn/problems/ti-huan-kong-ge-lcof/

### 题目内容

请实现一个函数，把字符串 s 中的每个空格替换成"%20"。

示例：

输入：s = "We are happy."  
输出："We%20are%20happy."

限制：

- 1  $0 \leq s.length \leq 10000$

我的题目思路与考察范围

核心思路：统计空格后扩展目标缓冲区，再从后向前把普通字符原样复制、把空格写成 %20，可避免前向移动造成反复搬移。

关键不变量 / 状态：逆向写入时，写指针之后已经是最终结果，读指针尚未处理的内容不会被覆盖。

复杂度：时间  $O(n)$ ，输出空间  $O(n)$ ；允许原缓冲区扩容时额外空间  $O(1)$

考察范围：剑指 Offer · 字符串替换 · 逆向填充 · 数组扩容

易错点：每个空格会从一个字符扩成三个字符，目标长度和逆向指针必须据此计算。



## 151. 反转字符串中的单词

|      |                 |      |                                |
|------|-----------------|------|--------------------------------|
| 唯一题序 | 34/158          | 本章顺序 | 4/7                            |
| 难度   | 中等              | 题目类型 | 主讲题                            |
| 出现章节 | 第4章 字符串、第5章 双指针 | 源位置  | 4. 《代码随想录》字符串 ( V3.0 ) .md:378 |

LeetCode : <https://leetcode.cn/problems/reverse-words-in-a-string/>

### 题目内容

给你一个字符串  $s$ ，请你反转字符串中 单词 的顺序。

单词 是由非空格字符组成的字符串。 $s$  中使用至少一个空格将字符串中的 单词 分隔开。

返回 单词 顺序颠倒且 单词 之间用单个空格连接的结果字符串。

注意：输入字符串  $s$  中可能会存在前导空格、尾随空格或者单词间的多个空格。返回的结果字符串中，单词间应当仅用单个空格分隔，且不包含任何额外的空格。

示例 1：

输入： $s = \text{"the sky is blue"}$   
输出： $\text{"blue is sky the"}$

示例 2：

输入： $s = \text{"hello world "}$   
输出： $\text{"world hello"}$   
解释：反转后的字符串中不能存在前导空格和尾随空格。

示例 3：

输入： $s = \text{"a good example"}$   
输出： $\text{"example good a"}$   
解释：如果两个单词间有多余的空格，反转后的字符串需要将单词间的空格减少到仅有一个。

提示：

- 1  $1 \leq s.length \leq 10^4$
- 1  $s$  包含英文大小写字母、数字和空格 ''
- 1  $s$  中 至少存在一个 单词

进阶：如果字符串在你使用的编程语言中是一种可变数据类型，请尝试使用  $O(1)$  额外空间复杂度的 原地 解法。

我的题目思路与考察范围

核心思路：先去除首尾空格并压缩单词间空格，再整体反转字符串，最后逐个反转单词，即可在保持单词内部顺序的同时翻转单词次序。

关键不变量 / 状态：逐词反转阶段，已处理前缀中的单词顺序和字符方向均已最终确定，且单词间只有一个空格。

复杂度：时间  $O(n)$ ，不可变字符串实现空间  $O(n)$ ；可变字符数组可做到额外  $O(1)$

考察范围：字符串规范化 · 整体与局部反转 · 双指针 · 空格处理

易错点：必须同时处理前导、尾随和连续空格，不能只按单个空格简单切分后原样拼接。

## 剑指 Offer 58-II. 左旋转字符串

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 35/158  | 本章顺序 | 5/7                           |
| 难度   | 简单      | 题目类型 | 主讲题                           |
| 出现章节 | 第4章 字符串 | 源位置  | 4.《代码随想录》字符串 ( V3.0 ) .md:572 |

LeetCode : <https://leetcode.cn/problems/zuo-xuan-zhuan-zi-fu-chuan-lcof/>

### 题目内容

字符串的左旋转操作是把字符串前面的若干个字符转移到字符串的尾部。

请定义一个函数实现字符串左旋转操作。给定字符串  $s$  和数字  $n$ ，返回左旋转  $n$  位得到的字符串。

示例 1：

输入： $s = \text{"abcdefg"}, n = 2$   
输出： $\text{"cdefgab"}$

示例 2：

输入： $s = \text{"lrloseumgh"}, n = 6$   
输出： $\text{"umghlrlose"}$

限制：

1  $1 \leq n < s.length \leq 10000$

我的题目思路与考察范围

核心思路：将前  $k$  个字符、后  $n-k$  个字符分别反转，再整体反转，就能原地把前缀移动到字符串末尾。

关键不变量 / 状态：两段各自逆序后，整体反转会交换两段位置并恢复每段内部的原顺序。

复杂度：时间  $O(n)$ ，可变字符数组上额外空间  $O(1)$

考察范围：剑指 Offer · 三次反转 · 字符串旋转 · 模运算

易错点：通用实现应先令  $k$  对字符串长度取模，并处理空字符串以避免除零。

## 28. 找出字符串中第一个匹配项的下标

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 36/158  | 本章顺序 | 6/7                           |
| 难度   | 简单      | 题目类型 | 主讲题                           |
| 出现章节 | 第4章 字符串 | 源位置  | 4.《代码随想录》字符串 ( V3.0 ) .md:653 |

LeetCode : <https://leetcode.cn/problems/find-the-index-of-the-first-occurrence-in-a-string/>

### 题目内容

给你两个字符串  $haystack$  和  $needle$ ，请在  $haystack$  字符串中找出  $needle$  字符串的第一个匹配项的下标（下标从 0 开始）。如果  $needle$  不是  $haystack$  的一部分，则返回 -1。

示例 1：

输入： $haystack = \text{"sadbutsad"}, needle = \text{"sad"}$   
输出：0  
解释： $\text{"sad"}$  在下标 0 和 6 处匹配。  
第一个匹配项的下标是 0，所以返回 0。

示例 2：

输入: haystack = "leetcode", needle = "leeto"  
 输出: -1  
 解释: "leeto" 没有在 "leetcode" 中出现, 所以返回 -1。

提示:

1  $1 \leq \text{haystack.length}, \text{needle.length} \leq 10^4$

1 haystack 和 needle 仅由小写英文字母组成

我的题目思路与考察范围

核心思路: 先为 needle 构造 KMP 前缀函数, 再扫描 haystack; 失配时跳到最长可复用前后缀, 完整匹配后返回起始下标。

关键不变量 / 状态: 扫描到文本位置  $i$  时,  $j$  始终等于当前文本后缀与模式前缀的最长匹配长度。

复杂度: 时间  $O(n + m)$ , 空间  $O(m)$

考察范围: KMP · 前缀函数 · 字符串匹配 · 失配回退

易错点: 失配时要反复令  $j = \text{lps}[j - 1]$ , 并按题意让空 needle 返回 0。

## 459. 重复的子字符串

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 37/158  | 本章顺序 | 7/7                             |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第4章 字符串 | 源位置  | 4. 《代码随想录》字符串 ( V3.0 ) .md:1207 |

LeetCode: <https://leetcode.cn/problems/repeated-substring-pattern/>

### 题目内容

给定一个非空的字符串  $s$ , 检查是否可以通过由它的一个子串重复多次构成。

示例 1:

输入:  $s = \text{"abab"}$   
 输出: true  
 解释: 可由子串 "ab" 重复两次构成。

示例 2:

输入:  $s = \text{"aba"}$   
 输出: false

示例 3:

输入:  $s = \text{"abcabcabcabc"}$   
 输出: true  
 解释: 可由子串 "abc" 重复四次构成。(或子串 "abcabc" 重复两次构成。)

提示:

1  $1 \leq s.length \leq 10^4$

1  $s$  由小写英文字母组成

#### 我的题目思路与考察范围

核心思路：计算整个字符串的最长相等真前后缀长度  $L$ ；当  $L > 0$  且  $n$  能被  $n - L$  整除时，字符串可由长度  $n - L$  的子串重复构成。

关键不变量 / 状态：前缀函数在每个位置记录该前缀的最长相等真前后缀，为周期长度提供可复用边界。

复杂度：时间  $O(n)$ ，空间  $O(n)$

考察范围：KMP · 字符串周期 · 最长前后缀 · 整除判定

易错点：必须同时检查  $L$  大于零和  $n \% (n - L) == 0$ ，只有长前后缀并不足够。

## 第 5 章 双指针

本章 14 道去重题目，按源文件首次出现顺序排列。

### 27. 移除元素 (复练)

本章顺序：1/14

LeetCode：https://leetcode.cn/problems/remove-element/

完整收录：第 1 章 数组

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

### 26. 删除有序数组中的重复项 (复练)

本章顺序：2/14

LeetCode：https://leetcode.cn/problems/remove-duplicates-from-sorted-array/

完整收录：第 1 章 数组

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

### 283. 移动零 (复练)

本章顺序：3/14

LeetCode：https://leetcode.cn/problems/move-zeroes/

完整收录：第 1 章 数组

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

### 844. 比较含退格的字符串 (复练)

本章顺序：4/14

LeetCode：https://leetcode.cn/problems/backspace-string-compare/

完整收录：第 1 章 数组

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

### 977. 有序数组的平方 (复练)

本章顺序：5/14

LeetCode：https://leetcode.cn/problems/squares-of-a-sorted-array/

完整收录：第 1 章 数组

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

### 344. 反转字符串 (复练)

本章顺序：6/14

LeetCode：https://leetcode.cn/problems/reverse-string/

完整收录：第 4 章 字符串

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 剑指 Offer 05. 替换空格 ( 复练 )

本章顺序：7/14

LeetCode：https://leetcode.cn/problems/ti-huan-kong-ge-lcof/

完整收录：第 4 章 字符串

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 151. 反转字符串中的单词 ( 复练 )

本章顺序：8/14

LeetCode：https://leetcode.cn/problems/reverse-words-in-a-string/

完整收录：第 4 章 字符串

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 206. 反转链表 ( 复练 )

本章顺序：9/14

LeetCode：https://leetcode.cn/problems/reverse-linked-list/

完整收录：第 2 章 链表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 19. 删除链表的倒数第 N 个结点 ( 复练 )

本章顺序：10/14

LeetCode：https://leetcode.cn/problems/remove-nth-node-from-end-of-list/

完整收录：第 2 章 链表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 面试题 02.07. 链表相交 ( 复练 )

本章顺序：11/14

LeetCode：https://leetcode.cn/problems/intersection-of-two-linked-lists-lcci/

完整收录：第 2 章 链表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 142. 环形链表 II ( 复练 )

本章顺序：12/14

LeetCode：https://leetcode.cn/problems/linked-list-cycle-ii/

完整收录：第 2 章 链表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 15. 三数之和 ( 复练 )

本章顺序：13/14

LeetCode：https://leetcode.cn/problems/3sum/

完整收录：第 3 章 哈希表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

## 18. 四数之和 ( 复练 )

本章顺序：14/14

LeetCode：https://leetcode.cn/problems/4sum/

完整收录：第 3 章 哈希表

复练重点：本章把它作为双指针复练：明确两个指针各自表示什么，并用指针关系维持不变量。

# 第 6 章 栈与队列

本章 7 道去重题目，按源文件首次出现顺序排列。

## 232. 用栈实现队列

|      |          |      |                               |
|------|----------|------|-------------------------------|
| 唯一题序 | 38/158   | 本章顺序 | 1/7                           |
| 难度   | 简单       | 题目类型 | 主讲题                           |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:95 |

LeetCode：https://leetcode.cn/problems/implement-queue-using-stacks/

### 题目内容

请你仅使用两个栈实现先入先出队列。队列应当支持一般队列支持的所有操作（ push 、 pop 、 peek 、 empty ）：

实现 MyQueue 类：

- 1 void push(int x) 将元素 x 推到队列的末尾
- 1 int pop() 从队列的开头移除并返回元素
- 1 int peek() 返回队列开头的元素
- 1 boolean empty() 如果队列为空，返回 true ；否则，返回 false

说明：

- 1 你 只能 使用标准的栈操作 —— 也就是只有 push to top , peek/pop from top , size , 和 is empty 操作是合法的。
- 1 你所使用的语言也许不支持栈。你可以使用 list 或者 deque（双端队列）来模拟一个栈，只要是标准的栈操作即可。

示例 1：

输入：  
["MyQueue", "push", "push", "peek", "pop", "empty"]  
[[], [1], [2], [], [], []]  
输出：  
[null, null, null, 1, 1, false]

解释：  
MyQueue myQueue = new MyQueue();  
myQueue.push(1); // queue is: [1]  
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)  
myQueue.peek(); // return 1  
myQueue.pop(); // return 1, queue is [2]  
myQueue.empty(); // return false

提示：

- 1 1 <= x <= 9
- 1 最多调用 100 次 push 、 pop 、 peek 和 empty
- 1 假设所有操作都是有效的（例如，一个空的队列不会调用 pop 或者 peek 操作）

进阶：

- 1 你能否实现每个操作均摊时间复杂度为 O(1) 的队列？换句话说，执行 n 个操作的总时间复杂度为 O(n) ，即使其中一个操作可能花费较长时间。



## 我的题目思路与考察范围

核心思路：用输入栈接收 push，用输出栈提供队首；仅当输出栈为空时才把输入栈全部倒入，使最早进入的元素翻转到栈顶。

关键不变量 / 状态：逻辑队列顺序由输出栈自顶向下接输入栈自底向上组成，每个元素至多跨栈一次。

复杂度：push、pop、peek 摊还时间  $O(1)$ ，单次转移最坏  $O(n)$ ，空间  $O(n)$

考察范围：栈与队列 · 摊还分析 · 双栈 · 数据结构设计

易错点：输出栈非空时不能再次倒入输入栈，否则会破坏已有元素的出队顺序。

## 225. 用队列实现栈

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 39/158   | 本章顺序 | 2/7                            |
| 难度   | 简单       | 题目类型 | 主讲题                            |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:200 |

LeetCode : <https://leetcode.cn/problems/implement-stack-using-queues/>

## 题目内容

请你仅使用两个队列实现一个后入先出 ( LIFO ) 的栈，并支持普通栈的全部四种操作 ( push 、 top 、 pop 和 empty )。

实现 MyStack 类：

- 1 void push(int x) 将元素 x 压入栈顶。
- 1 int pop() 移除并返回栈顶元素。
- 1 int top() 返回栈顶元素。
- 1 boolean empty() 如果栈是空的，返回 true ；否则，返回 false 。

注意：

- 1 你只能使用队列的标准操作 —— 也就是 push to back 、 peek/pop from front 、 size 和 is empty 这些操作。
- 1 你所使用的语言也许不支持队列。 你可以使用 list ( 列表 ) 或者 deque ( 双端队列 ) 来模拟一个队列，只要是标准的队列操作即可。

示例：

输入：  
["MyStack", "push", "push", "top", "pop", "empty"]  
[[], [1], [2], [], [], []]  
输出：  
[null, null, null, 2, 2, false]

解释：  
MyStack myStack = new MyStack();  
myStack.push(1);  
myStack.push(2);  
myStack.top(); // 返回 2  
myStack.pop(); // 返回 2  
myStack.empty(); // 返回 False

提示：

- 1  $1 \leq x \leq 9$
- 1 最多调用 100 次 push 、 pop 、 top 和 empty
- 1 每次调用 pop 和 top 都保证栈不为空

进阶：你能否仅用一个队列来实现栈。

我的题目思路与考察范围

核心思路：使用一个队列保存栈元素；执行 pop 或 top 前把队首的 size - 1 个元素依次移到队尾，使最后入队元素转到队首。

关键不变量 / 状态：常态下队列按栈底到栈顶排列，旋转 size - 1 次后原栈顶恰位于队首。

复杂度：push 时间  $O(1)$ ，pop 和 top 时间  $O(n)$ ，空间  $O(n)$

考察范围：队列模拟栈 · 队列旋转 · 数据结构设计 · 操作复杂度

易错点：旋转次数必须依据操作开始时的队列长度，top 查看后还要把该元素放回队列。

## 20. 有效的括号

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 40/158   | 本章顺序 | 3/7                             |
| 难度   | 简单       | 题目类型 | 主讲题                             |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6. 《代码随想录》栈与队列 ( V3.0 ) .md:344 |

LeetCode：https://leetcode.cn/problems/valid-parentheses/

### 题目内容

给定一个只包括 '('，')'，'{'，'}'，'['，']' 的字符串 s，判断字符串是否有效。

有效字符串需满足：

- 1 左括号必须用相同类型的右括号闭合。
- 2 左括号必须以正确的顺序闭合。
- 3 每个右括号都有一个对应的相同类型的左括号。

示例 1：

输入：s = "()"

输出：true

示例 2：

输入：s = "()[]{}"

输出：true

示例 3：

输入：s = "()["

输出：false

示例 4：

输入：s = "([)]"

输出：true

示例 5：

输入：s = "([)]"

输出：false

提示：

1 1 <= s.length <= 10 ^4

1 s 仅由括号 '(){}' 组成

我的题目思路与考察范围

核心思路：遇到左括号时把对应的期望右括号压栈，遇到右括号时必须与栈顶一致；扫描结束且栈空才是有效括号串。

关键不变量 / 状态：栈自底向顶保存所有尚未闭合左括号所期望的右括号顺序。

复杂度：时间  $O(n)$ ，空间  $O(n)$

考察范围：栈 · 括号匹配 · 嵌套结构 · 字符串扫描

易错点：读取右括号前要先检查栈是否为空，扫描完成后也必须确认没有剩余左括号。

## 1047. 删除字符串中的所有相邻重复项

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 41/158   | 本章顺序 | 4/7                            |
| 难度   | 简单       | 题目类型 | 主讲题                            |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:485 |

LeetCode : <https://leetcode.cn/problems/remove-all-adjacent-duplicates-in-string/>

### 题目内容

给出由小写字母组成的字符串  $s$ ，重复项删除操作 会选择两个相邻且相同的字母，并删除它们。

在  $s$  上反复执行重复项删除操作，直到无法继续删除。

在完成所有重复项删除操作后返回最终的字符串。答案保证唯一。

示例：

输入："abbaca"

输出："ca"

解释：

例如，在 "abbaca" 中，我们可以删除 "bb" 由于两字母相邻且相同，这是此时唯一可以执行删除操作的重复项。之后我们得到字符串 "aaca"，其中又只有 "aa" 可以执行重复项删除操作，所以最后的字符串为 "ca"。

提示：

1 1 <= s.length <= 10 ^5

2 s 仅由小写英文字母组成。

我的题目思路与考察范围

核心思路：把结果缓冲区当作栈：当前字符等于栈顶就弹出，否则压入；弹出后新形成的相邻重复也会在后续比较中继续消除。

关键不变量 / 状态：栈始终是已扫描前缀完全消除相邻重复后的唯一简化结果，内部不存在相邻相等字符。

复杂度：时间  $O(n)$ ，空间  $O(n)$

考察范围：栈 · 字符串消除 · 级联反应 · 线性扫描

易错点：只删除原字符串中一轮相邻对会漏掉删除后新产生的级联重复。

## 150. 逆波兰表达式求值

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 42/158   | 本章顺序 | 5/7                            |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:606 |

LeetCode : <https://leetcode.cn/problems/evaluate-reverse-polish-notation/>

### 题目内容

给你一个字符串数组 `tokens`，表示一个根据 逆波兰表示法 表示的算术表达式。

请你计算该表达式。返回一个表示表达式值的整数。

注意：

- 1 有效的算符为 '+'、 '-'、 '\*' 和 '/'。
- 1 每个操作数（运算对象）都可以是一个整数或者另一个表达式。
- 1 两个整数之间的除法总是 向零截断。
- 1 表达式中不含除零运算。
- 1 输入是一个根据逆波兰表示法表示的算术表达式。
- 1 答案及所有中间计算结果可以用 32 位 整数表示。

示例 1：

输入：tokens = ["2","1","+","3","\*"]  
 输出：9  
 解释：该算式转化为常见的中缀算术表达式为：((2 + 1) \* 3) = 9

示例 2：

输入：tokens = ["4","13","5","/","+"]  
 输出：6  
 解释：该算式转化为常见的中缀算术表达式为：(4 + (13 / 5)) = 6

示例 3：

输入：tokens = ["10","6","9","3","+","-11","\*","/", "17","+","5","+"]  
 输出：22  
 解释：该算式转化为常见的中缀算术表达式为：  

$$((10 * (6 / ((9 + 3) * -11))) + 17) + 5$$

$$= ((10 * (6 / (12 * -11))) + 17) + 5$$

$$= ((10 * (6 / -132)) + 17) + 5$$

$$= ((10 * 0) + 17) + 5$$

$$= (0 + 17) + 5$$

$$= 17 + 5$$

$$= 22$$

提示：

- 1 1 <= tokens.length <= 10 ^4
- 1 tokens[i] 是一个算符（ "+", "-", "\*" 或 "/" ），或是在范围 [-200, 200] 内的一个整数

逆波兰表达式：

逆波兰表达式是一种后缀表达式，所谓后缀就是指算符写在后面。

- 1 平常使用的算式则是一种中缀表达式，如 ( 1 + 2 ) \* ( 3 + 4 )。

1 该算式的逆波兰表达式写法为  $((12 +)(34 +)*)$ 。

逆波兰表达式主要有以下两个优点：

1 去掉括号后表达式无歧义，上式即便写成  $12 + 34 + *$  也可以依据次序计算出正确结果。

1 适合用栈操作运算：遇到数字则入栈；遇到算符则取出栈顶两个数字进行计算，并将结果压入栈中

我的题目思路与考察范围

核心思路：顺序读取 token，数字压栈，运算符则先弹出右操作数再弹出左操作数，计算后把结果压回栈。

关键不变量 / 状态：栈中保存当前已读前缀内所有尚未被更外层运算消费的子表达式结果。

复杂度：时间  $O(n)$ ，空间  $O(n)$

考察范围：栈 · 逆波兰表达式 · 表达式求值 · 整数除法

易错点：减法和除法的两个操作数顺序不能颠倒，除法还必须按题意向零截断。

## 239. 滑动窗口最大值

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 43/158   | 本章顺序 | 6/7                            |
| 难度   | 困难       | 题目类型 | 主讲题                            |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:733 |

LeetCode：https://leetcode.cn/problems/sliding-window-maximum/

### 题目内容

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。

返回 滑动窗口中的最大值。

示例 1：

输入：nums = [1,3,-1,-3,5,3,6,7], k = 3

输出：[3,3,5,5,6,7]

解释：

滑动窗口的位置 最大值

-----

[1 3 -1] -3 5 3 6 7 3

1 [3 -1 -3] 5 3 6 7 3

1 3 [-1 -3 5] 3 6 7 5

1 3 -1 [-3 5 3] 6 7 5

1 3 -1 -3 [5 3 6] 7 6

1 3 -1 -3 5 [3 6 7] 7

示例 2：

输入：nums = [1], k = 1

输出：[1]

提示：

1  $1 \leq \text{nums.length} \leq 10^5$

1  $-10^4 \leq \text{nums}[i] \leq 10^4$

1  $1 \leq k \leq \text{nums.length}$

## 我的题目思路与考察范围

核心思路：维护存放下标的单调递减双端队列：先移除窗口外下标，再从队尾删除不大于当前值的下标，队首始终给出窗口最大值。

关键不变量 / 状态：队列下标递增、对应值严格递减，且所有下标都位于当前窗口内。

复杂度：时间  $O(n)$ ，空间  $O(k)$

考察范围：单调队列 · 滑动窗口 · 双端队列 · 最大值维护

易错点：必须保存下标而非只保存值，才能准确淘汰已经离开窗口的元素。

## 347. 前 K 个高频元素

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 44/158   | 本章顺序 | 7/7                            |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第6章 栈与队列 | 源位置  | 6.《代码随想录》栈与队列 ( V3.0 ) .md:925 |

LeetCode : <https://leetcode.cn/problems/top-k-frequent-elements/>

## 题目内容

给你一个整数数组 `nums` 和一个整数 `k`，请你返回其中出现频率前 `k` 高的元素。你可以按 任意顺序 返回答案。

示例 1：

输入：`nums = [1,1,1,2,2,3], k = 2`

输出：`[1,2]`

示例 2：

输入：`nums = [1], k = 1`

输出：`[1]`

示例 3：

输入：`nums = [1,2,1,2,1,2,3,1,3,2], k = 2`

输出：`[1,2]`

提示：

1  $1 \leq \text{nums.length} \leq 10^5$

1  $-10^4 \leq \text{nums}[i] \leq 10^4$

1 `k` 的取值范围是  $[1, \text{数组中不相同的元素的个数}]$

1 题目数据保证答案唯一，换句话说，数组中前 `k` 个高频元素的集合是唯一的

进阶：你所设计算法的时间复杂度 必须 优于  $O(n \log n)$ ，其中 `n` 是数组大小。

## 我的题目思路与考察范围

核心思路：先用哈希表统计频次，再按频次把元素放入桶并从高频向低频收集前 `k` 个；数据流或内存受限场景也可维护大小为 `k` 的小顶堆。

关键不变量 / 状态：频次桶 `f` 中只含出现 `f` 次的元素，逆序扫描桶时已收集元素的频次不低于后续候选。

复杂度：桶排序时间  $O(n)$ 、空间  $O(n)$ ；小顶堆方案时间  $O(n \log k)$ 、空间  $O(n + k)$

考察范围：哈希计数 · 桶排序 · 优先队列 · Top K · 频率统计

易错点：`k` 表示不同元素的数量，收集到 `k` 个后要立即停止，不能把同一频次桶全部越界加入。

## 第 7 章 二叉树

本章 42 道去重题目，按源文件首次出现顺序排列。

### 144. 二叉树的前序遍历

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 45/158  | 本章顺序 | 1/42                           |
| 难度   | 简单      | 题目类型 | 章节题单                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:291 |

LeetCode : <https://leetcode.cn/problems/binary-tree-preorder-traversal/>

#### 题目内容

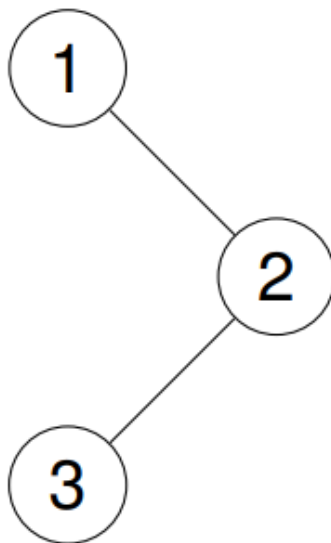
给你二叉树的根节点 `root`，返回它节点值的 前序 遍历。

示例 1：

输入：`root = [1,null,2,3]`

输出：`[1,2,3]`

解释：

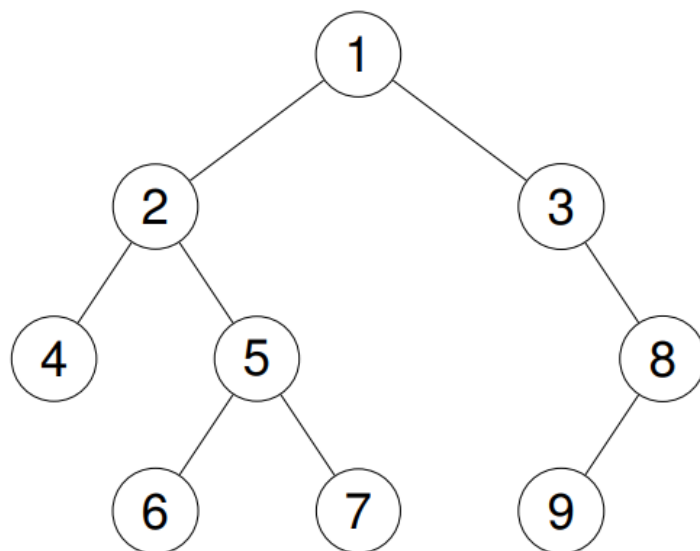


示例 2：

输入：`root = [1,2,3,4,5,null,8,null,null,6,7,9]`

输出：`[1,2,4,5,6,7,3,8,9]`

解释：



示例 3：

输入：root = []

输出：[]

示例 4：

输入：root = [1]

输出：[1]

提示：

1 树中节点数目在范围 [0, 100] 内

1 -100 <= Node.val <= 100

进阶：递归算法很简单，你可以通过迭代算法完成吗？

我的题目思路与考察范围

核心思路：递归按根-左-右访问；迭代时弹出即记录，并先压右子树再压左子树。

关键不变量 / 状态：每次处理栈顶时，它都是尚未输出节点中按前序顺序最靠前的节点。

复杂度：时间  $O(n)$ ，递归栈或显式栈最坏  $O(n)$ 。

考察范围：二叉树 · DFS · 递归 · 栈

易错点：迭代压栈顺序若写成先左后右，输出会变成根-右-左。

## 145. 二叉树的后序遍历

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 46/158  | 本章顺序 | 2/42                           |
| 难度   | 简单      | 题目类型 | 章节题单                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:292 |

LeetCode：https://leetcode.cn/problems/binary-tree-postorder-traversal/

### 题目内容

给你一棵二叉树的根节点 root，返回其节点值的 后序遍历。

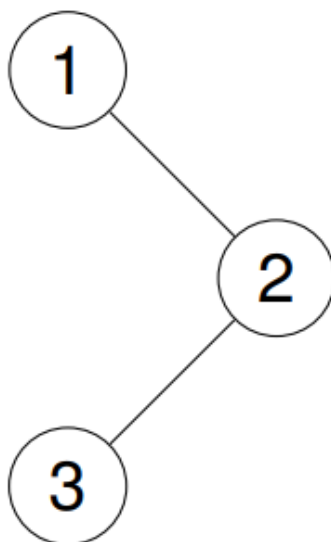


示例 1：

输入：root = [1,null,2,3]

输出：[3,2,1]

解释：

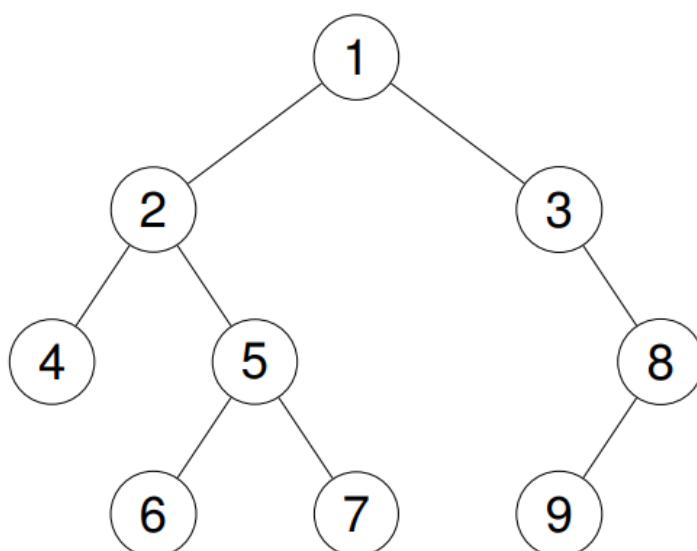


示例 2：

输入：root = [1,2,3,4,5,null,8,null,null,6,7,9]

输出：[4,6,7,5,2,9,8,3,1]

解释：



示例 3：

输入：root = []

输出：[]

示例 4：

输入：root = [1]

输出：[1]

提示：

1 树中节点的数目在范围 [0, 100] 内

1  $-100 \leq \text{Node.val} \leq 100$

进阶：递归算法很简单，你可以通过迭代算法完成吗？

我的题目思路与考察范围

核心思路：递归按左-右-根访问；迭代可先得到根-右-左，再整体反转。

关键不变量 / 状态：反转前序列中的每个根都先于其左右子树，反转后恰为左-右-根。

复杂度：时间  $O(n)$ ，辅助空间最坏  $O(n)$ 。

考察范围：二叉树 · DFS · 后序遍历 · 栈

易错点：只交换左右入栈顺序而忘记最终反转，得不到后序结果。

## 94. 二叉树的中序遍历

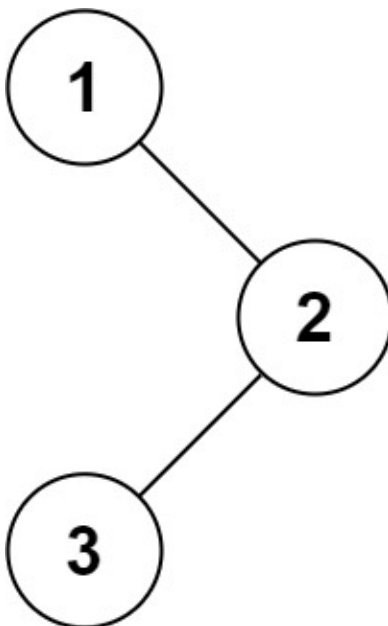
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 47/158  | 本章顺序 | 3/42                           |
| 难度   | 简单      | 题目类型 | 章节题单                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:293 |

LeetCode：https://leetcode.cn/problems/binary-tree-inorder-traversal/

### 题目内容

给定一个二叉树的根节点 root，返回它的中序遍历。

示例 1：



输入：root = [1,null,2,3]

输出：[1,3,2]

示例 2：

输入：root = []  
输出：[]

示例 3：

输入：root = [1]  
输出：[1]

提示：

1 树中节点数目在范围 [0, 100] 内

1  $-100 \leq \text{Node.val} \leq 100$

进阶: 递归算法很简单，你可以通过迭代算法完成吗？

我的题目思路与考察范围

核心思路：指针持续向左下探并压栈；无左路可走时弹栈记录，再转向右子树。

关键不变量 / 状态：栈保存已访问但尚未输出的祖先，栈顶是下一个应输出的节点。

复杂度：时间  $O(n)$ ，辅助空间  $O(h)$ ，最坏  $O(n)$ 。

考察范围：二叉树 · DFS · 中序遍历 · 栈

易错点：循环条件必须是 cur 非空或栈非空，不能只检查其中一个。

## 102. 二叉树的层序遍历

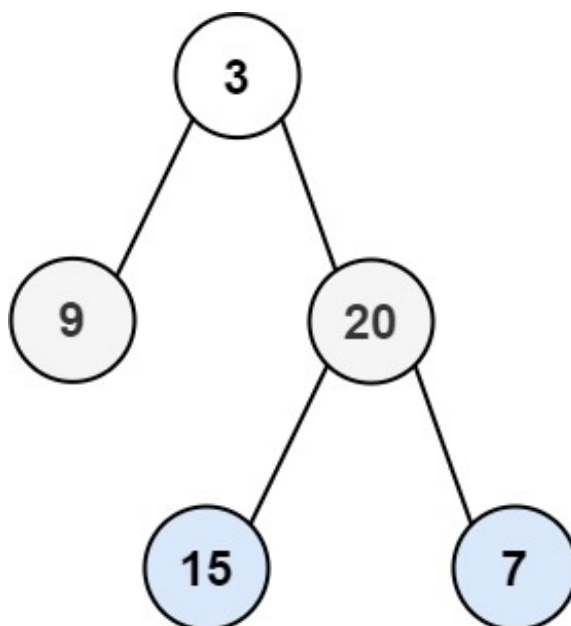
|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 48/158  | 本章顺序 | 4/42                          |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                      |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:589 |

LeetCode：https://leetcode.cn/problems/binary-tree-level-order-traversal/

### 题目内容

给你二叉树的根节点 root，返回其节点值的层序遍历。（即逐层地，从左到右访问所有节点）。

示例 1：



输入：root = [3,9,20,null,null,15,7]  
输出：[[3],[9,20],[15,7]]

示例 2：

输入：root = [1]  
输出：[[1]]

示例 3：

输入：root = []  
输出：[]

提示：

1 树中节点数目在范围 [0, 2000] 内

1 -1000 <= Node.val <= 1000

我的题目思路与考察范围

核心思路：用队列做 BFS，每轮先固定当前队列长度，只处理这一层。

关键不变量 / 状态：一轮开始时队列中恰好按从左到右顺序保存当前层全部节点。

复杂度：时间  $O(n)$ ，空间  $O(w)$ ， $w$  为最大层宽。

考察范围：二叉树 · BFS · 队列 · 分层遍历

易错点：层内循环不能动态使用变化中的 `queue.size()`，应先保存 `size`。

## 107. 二叉树的层序遍历 II

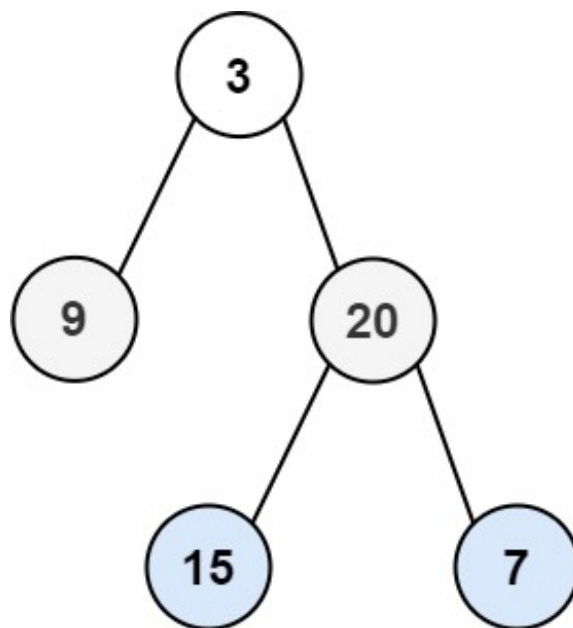
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 49/158  | 本章顺序 | 5/42                           |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:590 |

LeetCode：https://leetcode.cn/problems/binary-tree-level-order-traversal-ii/

### 题目内容

给你二叉树的根节点 root，返回其节点值 自底向上的层序遍历。（即按从叶子节点所在层到根节点所在的层，逐层从左向右遍历）

示例 1：



输入：root = [3,9,20,null,null,15,7]  
输出：[[15,7],[9,20],[3]]

示例 2：

输入：root = [1]  
输出：[[1]]

示例 3：

输入：root = []  
输出：[]

提示：

1 树中节点数目在范围 [0, 2000] 内

1  $-1000 \leq \text{Node.val} \leq 1000$

我的题目思路与考察范围

核心思路：照常从上到下 BFS 收集各层，结束后反转层数组。

关键不变量 / 状态：BFS 阶段每层内部始终保持从左到右，最终只反转层的先后。

复杂度：时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：二叉树 · BFS · 队列 · 数组反转

易错点：不要反转每层节点顺序；目标只要求层次自底向上。

## 199. 二叉树的右视图

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 50/158  | 本章顺序 | 6/42                           |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:591 |

LeetCode：https://leetcode.cn/problems/binary-tree-right-side-view/

### 题目内容

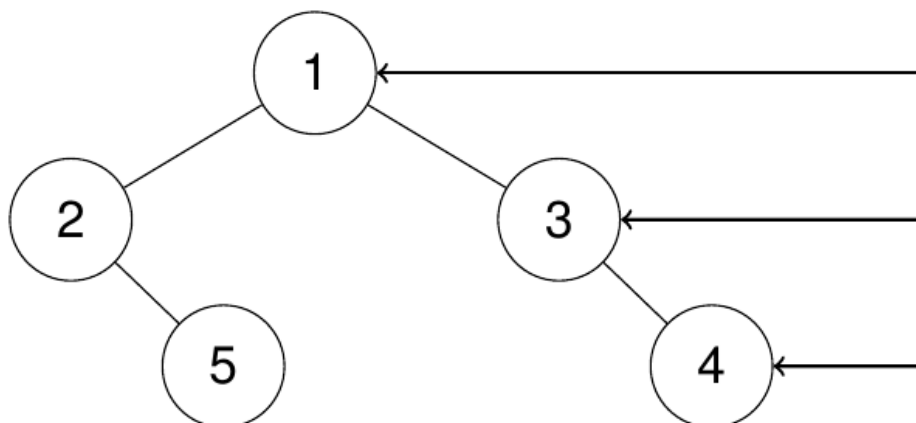
给定一个二叉树的根节点 root，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例 1：

输入：root = [1,2,3,null,5,null,4]

输出：[1,3,4]

解释：

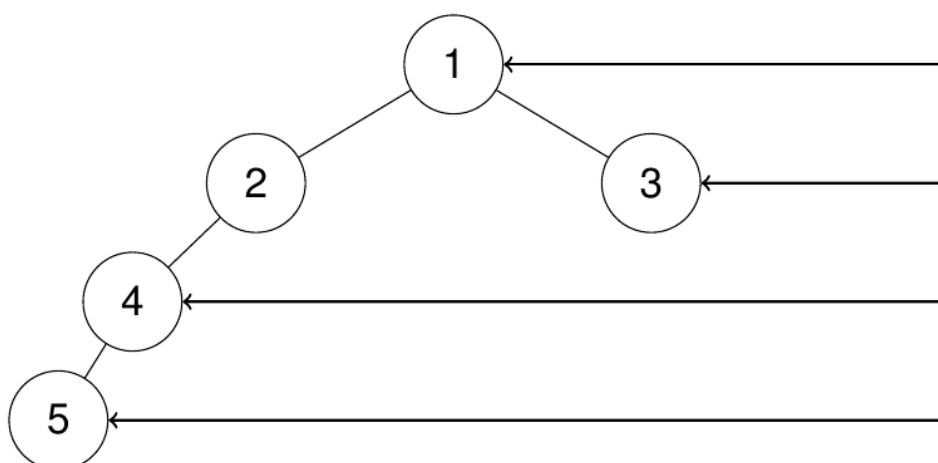


示例 2：

输入：root = [1,2,3,4,null,null,null,5]

输出：[1,3,4,5]

解释：



示例 3：

输入：root = [1,null,3]

输出：[1,3]

示例 4：

输入：root = []

输出：[]

提示:

1 二叉树的节点个数的范围是  $[0,100]$

1  $-100 \leq \text{Node.val} \leq 100$

我的题目思路与考察范围

核心思路：层序遍历时只记录每层最后出队的节点值。

关键不变量 / 状态：同层按从左到右入队，因此索引  $\text{size}-1$  的节点就是该层最右可见节点。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：二叉树 · BFS · 层序遍历 · 视图问题

易错点：若改成先右后左入队，应记录每层第一个节点，规则要配套。

## 637. 二叉树的层平均值

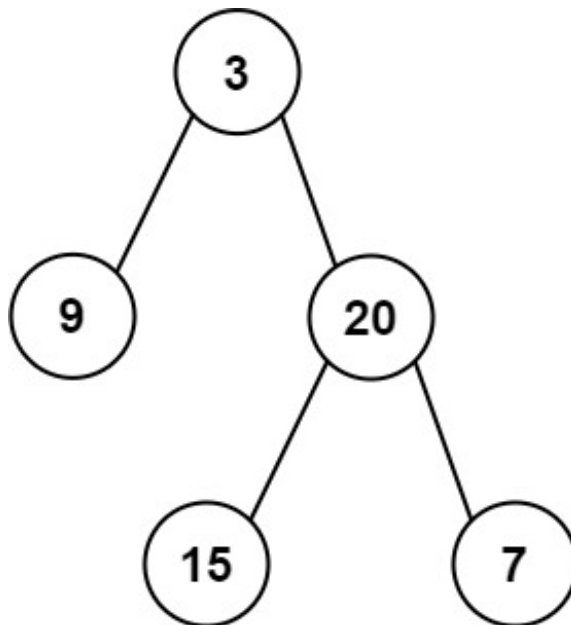
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 51/158  | 本章顺序 | 7/42                           |
| 难度   | 简单      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:592 |

LeetCode : <https://leetcode.cn/problems/average-of-levels-in-binary-tree/>

### 题目内容

给定一个非空二叉树的根节点  $\text{root}$ ，以数组的形式返回每一层节点的平均值。与实际答案相差  $10^{-5}$  以内的答案可以被接受。

示例 1：



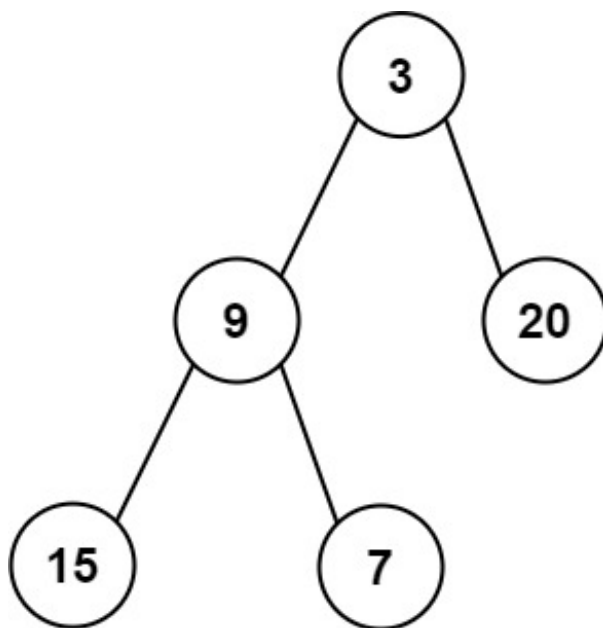
输入： $\text{root} = [3,9,20,\text{null},\text{null},15,7]$

输出： $[3.00000,14.50000,11.00000]$

解释：第 0 层的平均值为 3,第 1 层的平均值为 14.5,第 2 层的平均值为 11。

因此返回  $[3, 14.5, 11]$ 。

示例 2:



输入：root = [3,9,20,15,7]

输出：[3.00000,14.50000,11.00000]

提示：

1 树中节点数量在  $[1, 10^4]$  范围内

1  $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

我的题目思路与考察范围

核心思路：逐层 BFS 累加节点值，用该层总和除以固定层大小。

关键不变量 / 状态：每轮的 sum 和 size 只对应当前层，不混入下一层新入队节点。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：二叉树 · BFS · 分层统计 · 数值精度

易错点：总和应使用 double 或更宽整数，避免大节点值累加溢出。

## 429. N 叉树的层序遍历

|      |         |      |                               |
|------|---------|------|-------------------------------|
| 唯一题序 | 52/158  | 本章顺序 | 8/42                          |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                      |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:593 |

LeetCode：https://leetcode.cn/problems/n-ary-tree-level-order-traversal/

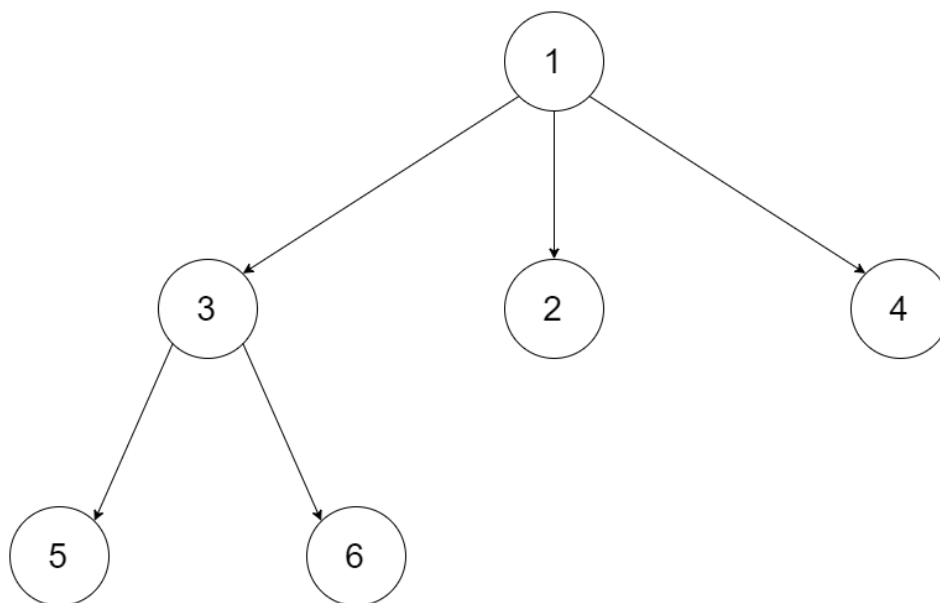
### 题目内容

给定一个 N 叉树，返回其节点值的 层序遍历。（即从左到右，逐层遍历）。

树的序列化输入是用层序遍历，每组子节点都由 null 值分隔（参见示例）。

示例 1：

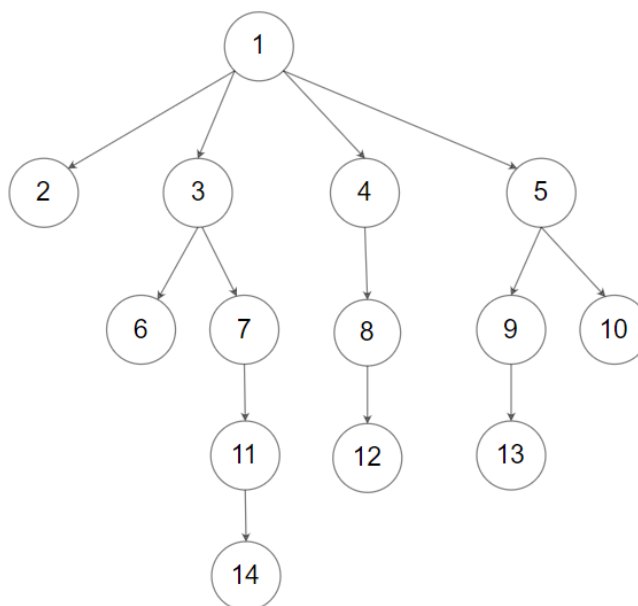




输入：root = [1,null,3,2,4,null,5,6]

输出：[[1],[3,2,4],[5,6]]

示例 2：



输入：root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

输出：[[1],[2,3,4,5],[6,7,8,9,10],[11,12,13],[14]]

提示：

- 1 树的高度不会超过 1000
- 1 树的节点总数在  $[0, 10^4]$  之间

我的题目思路与考察范围

核心思路：按层弹出 N 叉树节点，并把其 children 从左到右全部入队。

关键不变量 / 状态：一轮开始时队列只含当前层节点，处理后队列只含下一层节点。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：N 叉树 · BFS · 队列 · 分层遍历

易错点：节点的 children 可能为空，且不能假设每个节点孩子数相同。

## 515. 在每个树行中找最大值

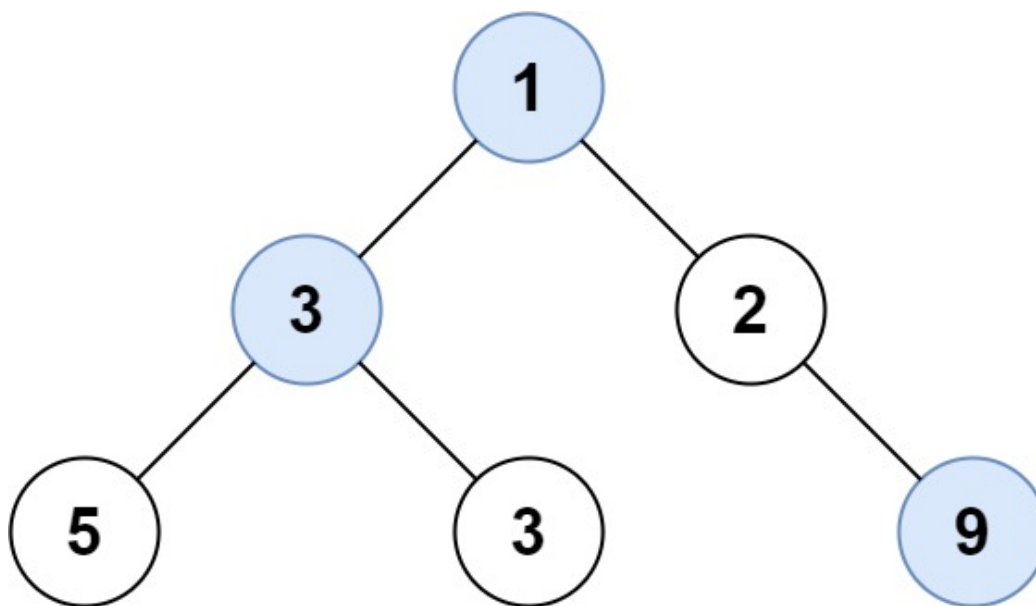
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 53/158  | 本章顺序 | 9/42                           |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:594 |

LeetCode : <https://leetcode.cn/problems/find-largest-value-in-each-tree-row/>

### 题目内容

给定一棵二叉树的根节点 `root`，请找出该二叉树中每一层的最大值。

示例1：



输入: `root = [1,3,2,5,3,null,9]`  
输出: `[1,3,9]`

示例2：

输入: `root = [1,2,3]`  
输出: `[1,3]`

提示：

1 二叉树的节点个数的范围是  $[0, 10^4]$

1  $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

我的题目思路与考察范围

核心思路：层序遍历，每层用第一个节点初始化最大值，再扫描该层更新。

关键不变量 / 状态：处理当前层前  $k$  个节点后，`max` 是这  $k$  个节点的最大值。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：二叉树 · BFS · 分层统计 · 最大值

易错点：最大值不能初始化为 0，否则全负数层会得到错误答案。

# 116. 填充每个节点的下一个右侧节点指针

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 54/158  | 本章顺序 | 10/42                          |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:595 |

LeetCode : <https://leetcode.cn/problems/populating-next-right-pointers-in-each-node/>

## 题目内容

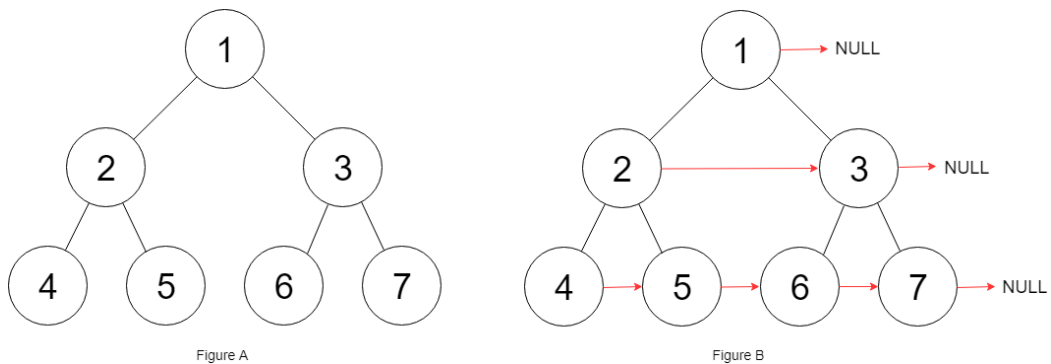
给定一个 完美二叉树 ，其所有叶子节点都在同一层，每个父节点都有两个子节点。二叉树定义如下：

```
struct Node {
    int val;
    Node *left;
    Node *right;
    Node *next;
}
```

填充它的每个 next 指针，让这个指针指向其下一个右侧节点。如果找不到下一个右侧节点，则将 next 指针设置为 NULL。

初始状态下，所有 next 指针都被设置为 NULL。

示例 1：



```
输入：root = [1,2,3,4,5,6,7]
输出：[1,#,2,3,#,4,5,6,7,#]
解释：给定二叉树如图 A 所示，你的函数应该填充它的每个 next 指针，以指向其下一个右侧节点，如图 B 所示。序列化的输出按层序遍历排列，同一层节点由 next 指针连接，'#' 标志着每一层的结束。
```

示例 2:

```
输入：root = []
输出：[]
```

提示：

1 树中节点的数量在  $[0, 2^{12} - 1]$  范围内

1  $-1000 \leq \text{node.val} \leq 1000$

进阶：

1 你只能使用常量级额外空间。

1 使用递归解题也符合要求，本题中递归程序占用的栈空间不算做额外的空间复杂度。

## 我的题目思路与考察范围

核心思路：利用完美二叉树结构，在一层 next 链上横移，连接 left->right 与 right->next->left。

关键不变量 / 状态：开始构造下一层时，当前层 next 链已经完整可遍历。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$ 。

考察范围：完美二叉树 · 层间连接 · 指针 ·  $O(1)$  空间

易错点：跨父节点连接前必须确认 parent.next 存在；该公式依赖完美二叉树。

## 117. 填充每个节点的下一个右侧节点指针 II

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 55/158  | 本章顺序 | 11/42                          |
| 难度   | 中等      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:596 |

LeetCode : <https://leetcode.cn/problems/populating-next-right-pointers-in-each-node-ii/>

## 题目内容

给定一个二叉树：

```
struct Node {  
    int val;  
    Node *left;  
    Node *right;  
    Node *next;  
}
```

填充它的每个 next 指针，让这个指针指向其下一个右侧节点。如果找不到下一个右侧节点，则将 next 指针设置为 NULL。

初始状态下，所有 next 指针都被设置为 NULL。

示例 1：

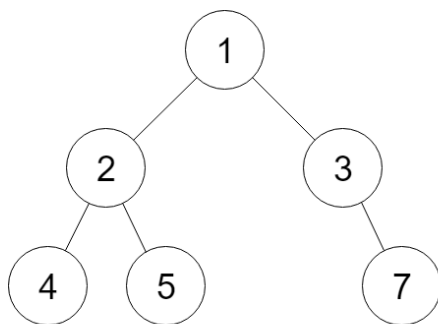


Figure A

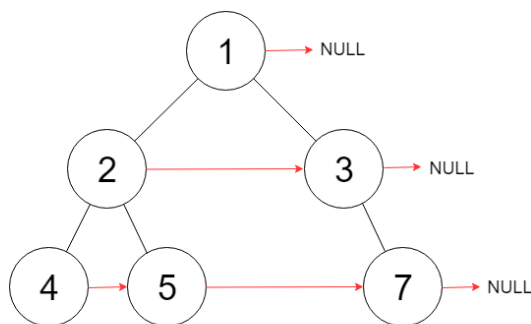


Figure B

输入：root = [1,2,3,4,5,null,7]

输出：[1,#,2,3,#,4,5,7,#]

解释：给定二叉树如图 A 所示，你的函数应该填充它的每个 next 指针，以指向其下一个右侧节点，如图 B 所示。序列化输出按层序遍历顺序（由 next 指针连接），'#' 表示每层的末尾。

示例 2：

输入：root = []

输出：[]

提示：

1 树中的节点数在范围 [0, 6000] 内

1     $-100 \leq \text{Node.val} \leq 100$

进阶：

1    你只能使用常量级额外空间。

1    使用递归解题也符合要求，本题中递归程序的隐式栈空间不计入额外空间复杂度。

我的题目思路与考察范围

核心思路：沿当前层 next 链扫描，用 dummy 和 tail 串起所有非空孩子形成下一层。

关键不变量 / 状态：扫描到当前节点时，dummy.next 到 tail 已是下一层的正确前缀。

复杂度：时间  $O(n)$ ，额外空间  $O(1)$ 。

考察范围：一般二叉树 · 层间连接 · 虚拟头节点 · 指针

易错点：孩子可能任意缺失，不能套用完美二叉树的固定左右连接公式。

## 104. 二叉树的最大深度

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 56/158  | 本章顺序 | 12/42                          |
| 难度   | 简单      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:597 |

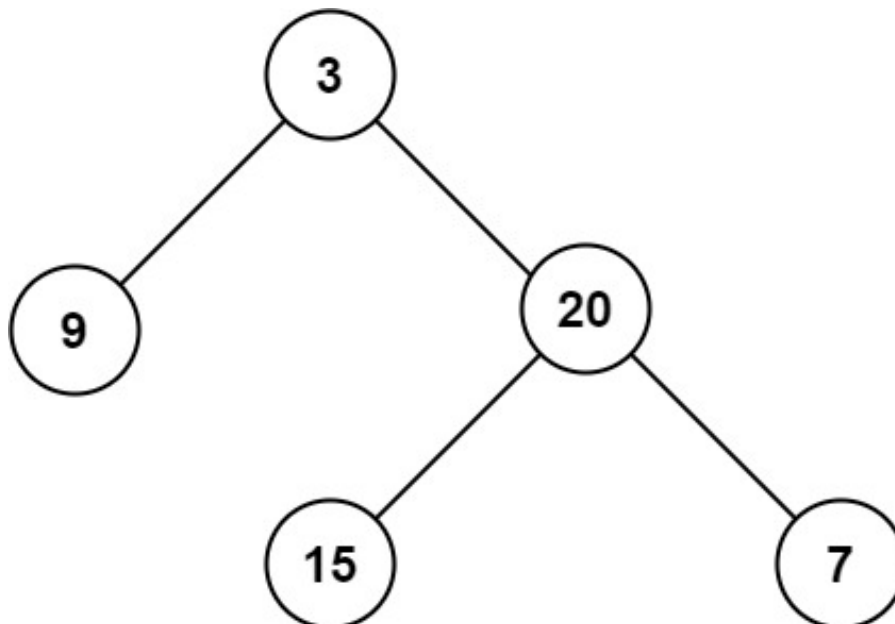
LeetCode：https://leetcode.cn/problems/maximum-depth-of-binary-tree/

### 题目内容

给定一个二叉树 root，返回其最大深度。

二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1：



输入：root = [3,9,20,null,null,15,7]

输出：3

示例 2：

输入：root = [1,null,2]

输出：2

提示：

1 树中节点的数量在  $[0, 10^4]$  区间内。

1  $-100 \leq \text{Node.val} \leq 100$

我的题目思路与考察范围  
核心思路：后序计算高度：当前深度等于左右子树最大深度加一。  
关键不变量 / 状态：递归返回时，返回值准确表示以当前节点为根的子树最大深度。  
复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。  
考察范围：二叉树 · 后序遍历 · 树高 · 递归  
易错点：空树深度为 0；不要混淆节点深度与节点高度的定义。

111. 二叉树的最小深度

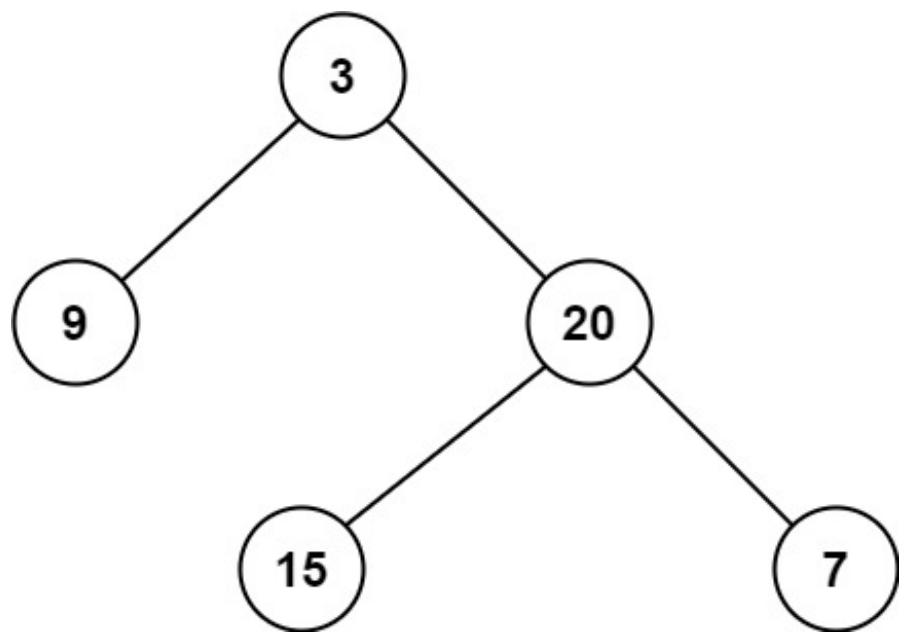
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 57/158  | 本章顺序 | 13/42                          |
| 难度   | 简单      | 题目类型 | 章节题单、主讲题                       |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:598 |

LeetCode：https://leetcode.cn/problems/minimum-depth-of-binary-tree/

题目内容

给定一个二叉树，找出其最小深度。  
最小深度是从根节点到最近叶子节点的最短路径上的节点数量。  
说明：叶子节点是指没有子节点的节点。

示例 1：



输入：root = [3,9,20,null,null,15,7]  
输出：2

示例 2：

输入：root = [2,null,3,null,4,null,5,null,6]  
输出：5

提示：

1 树中节点数的范围在  $[0, 10^5]$  内

1  $-1000 \leq \text{Node.val} \leq 1000$

我的题目思路与考察范围

核心思路：BFS 遇到第一个叶子即可返回层数；递归时单子树节点不能取缺失侧的 0。

关键不变量 / 状态：BFS 按深度递增访问，因此首个左右孩子都为空的节点属于最浅叶层。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：二叉树 · BFS · 叶子判定 · 最短路径

易错点：叶子必须左右孩子都为空；只空一侧的节点不是叶子。

## 226. 翻转二叉树

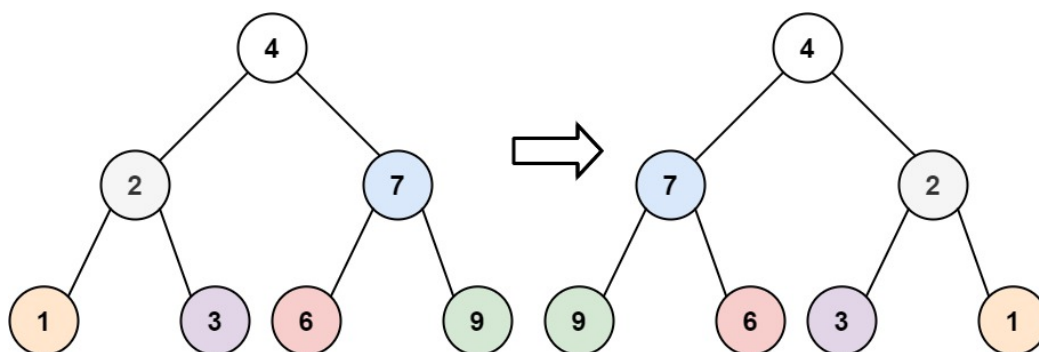
|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 58/158  | 本章顺序 | 14/42                          |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:1091 |

LeetCode：https://leetcode.cn/problems/invert-binary-tree/

### 题目内容

给你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

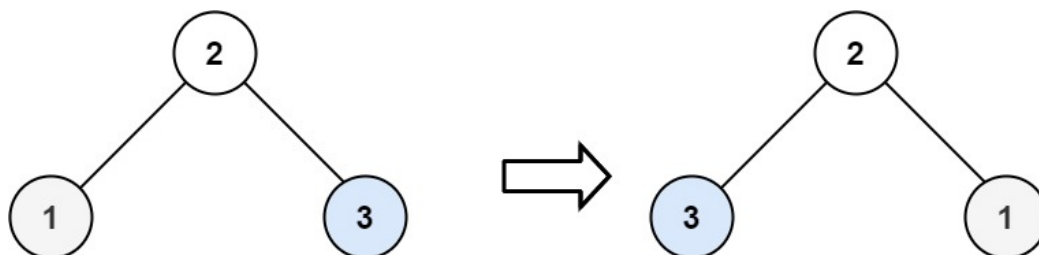
示例 1：



输入：root = [4,2,7,1,3,6,9]

输出：[4,7,2,9,6,3,1]

示例 2：



输入：root = [2,1,3]

输出：[2,3,1]

示例 3：

输入：root = []

输出：[]

提示：

1 树中节点数目范围在  $[0, 100]$  内

1  $-100 \leq \text{Node.val} \leq 100$

我的题目思路与考察范围

核心思路：访问每个节点并交换左右孩子，递归或 BFS/DFS 都可。

关键不变量 / 状态：处理完一个节点后，它的左右入口已交换，随后分别翻转两棵子树。

复杂度：时间  $O(n)$ ，辅助空间  $O(h)$  或  $O(w)$ 。

考察范围：二叉树 · 递归 · 结构修改 · 遍历

易错点：交换后递归变量的含义要保持一致，避免同一子树被处理两次。

## 589. N 叉树的前序遍历

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 59/158  | 本章顺序 | 15/42                           |
| 难度   | 简单      | 题目类型 | 章节题单                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:1393 |

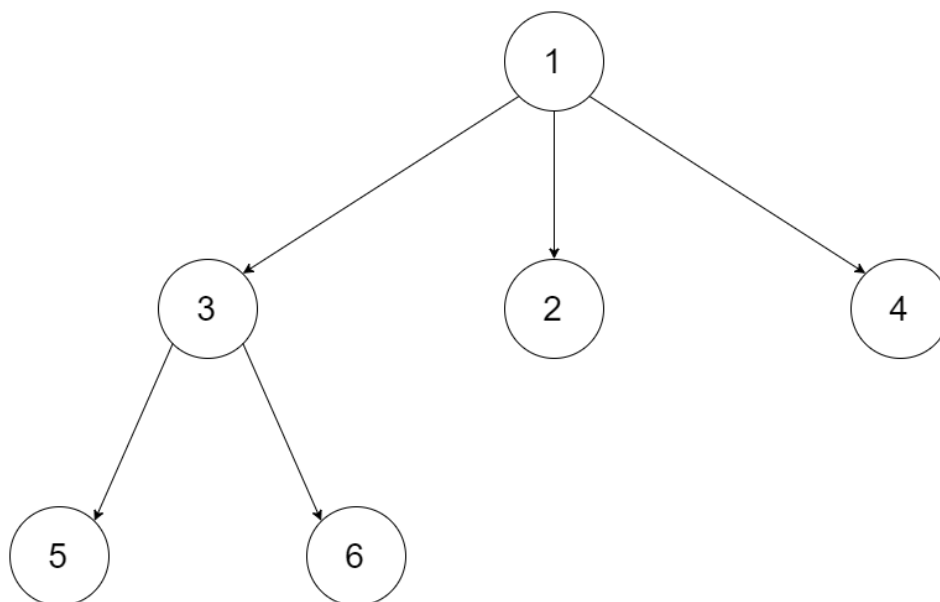
LeetCode： <https://leetcode.cn/problems/n-ary-tree-preorder-traversal/>

### 题目内容

给定一个  $n$  叉树的根节点  $\text{root}$ ，返回 其节点值的 前序遍历。

$n$  叉树 在输入中按层序遍历进行序列化表示，每组子节点由空值  $\text{null}$  分隔（请参见示例）。

示例 1：

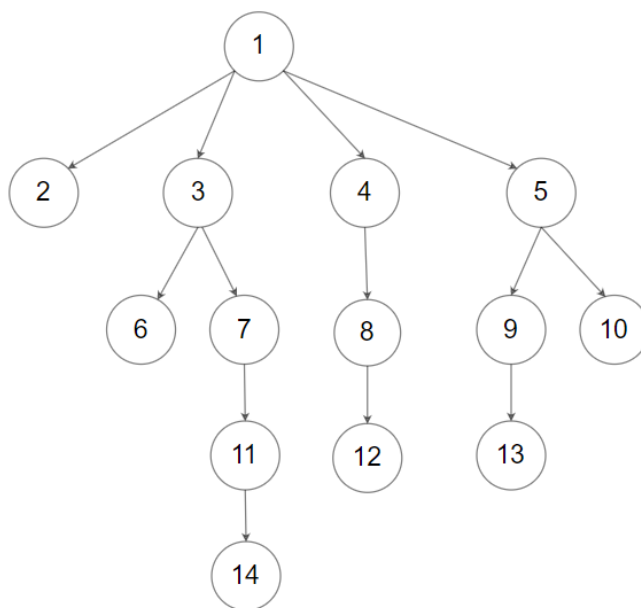


输入： $\text{root} = [1, \text{null}, 3, 2, 4, \text{null}, 5, 6]$

输出： $[1, 3, 5, 6, 2, 4]$

示例 2：





输入：root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

输出：[1,2,3,6,7,11,14,4,8,12,5,9,13,10]

提示：

- 1 节点总数在范围  $[0, 10^4]$  内
- 1  $0 \leq \text{Node.val} \leq 10^4$
- 1 n 叉树的高度小于或等于 1000

进阶：递归法很简单，你可以使用迭代法完成此题吗？

我的题目思路与考察范围

核心思路：先记录当前节点，再从左到右递归孩子；迭代栈需按孩子的逆序压入。

关键不变量 / 状态：节点被记录时，其所有祖先已记录，而其孩子尚未记录。

复杂度：时间  $O(n)$ ，辅助空间最坏  $O(n)$ 。

考察范围：N叉树 · DFS · 前序遍历 · 栈

易错点：按 children 原顺序压栈会导致兄弟节点逆序输出。

## 590. N 叉树的后序遍历

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 60/158  | 本章顺序 | 16/42                           |
| 难度   | 简单      | 题目类型 | 章节题单                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:1394 |

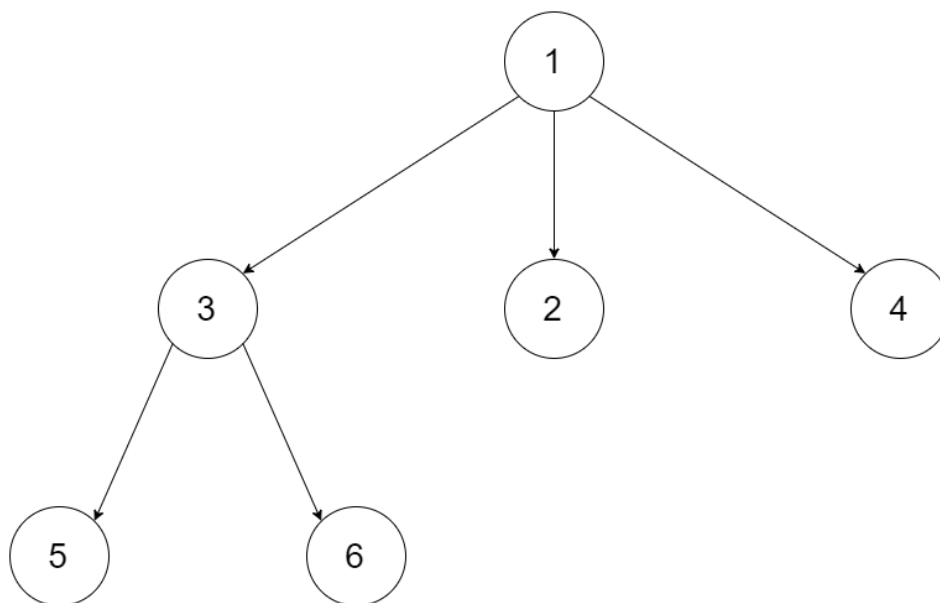
LeetCode：https://leetcode.cn/problems/n-ary-tree-postorder-traversal/

### 题目内容

给定一个 n 叉树的根节点 root，返回其节点值的后序遍历。

n 叉树在输入中按层序遍历进行序列化表示，每组子节点由空值 null 分隔（请参见示例）。

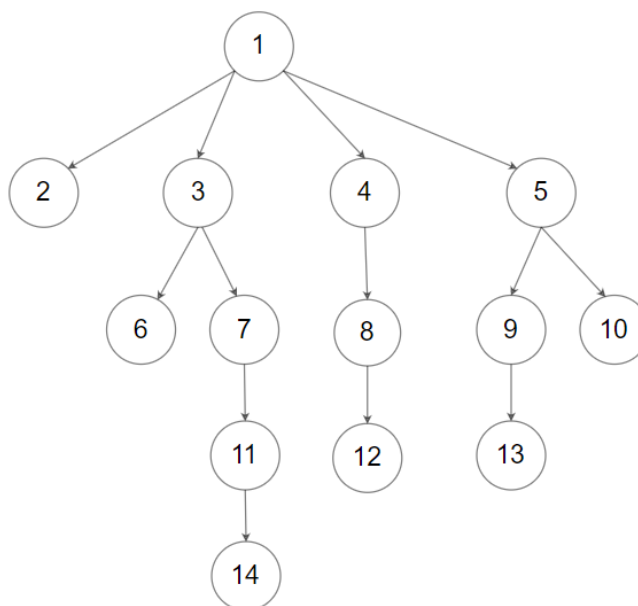
示例 1：



输入：root = [1,null,3,2,4,null,5,6]

输出：[5,6,3,2,4,1]

示例 2：



输入：root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

输出：[2,6,14,11,7,3,12,8,4,13,9,10,5,1]

提示：

- 1 节点总数在范围  $[0, 10^4]$  内
- 1  $0 \leq \text{Node.val} \leq 10^4$
- 1 n 叉树的高度小于或等于 1000

进阶：递归法很简单，你可以使用迭代法完成此题吗？

我的题目思路与考察范围  
核心思路：先递归处理全部孩子再记录根；迭代可构造根-逆孩子子序并反转。  
关键不变量 / 状态：一个节点输出前，它的每棵孩子子树都已完整输出。  
复杂度：时间  $O(n)$ ，辅助空间最坏  $O(n)$ 。  
考察范围：N叉树 · DFS · 后序遍历 · 栈  
易错点：N 叉树孩子有固定顺序，迭代与反转时要同时校准顺序。

101. 对称二叉树

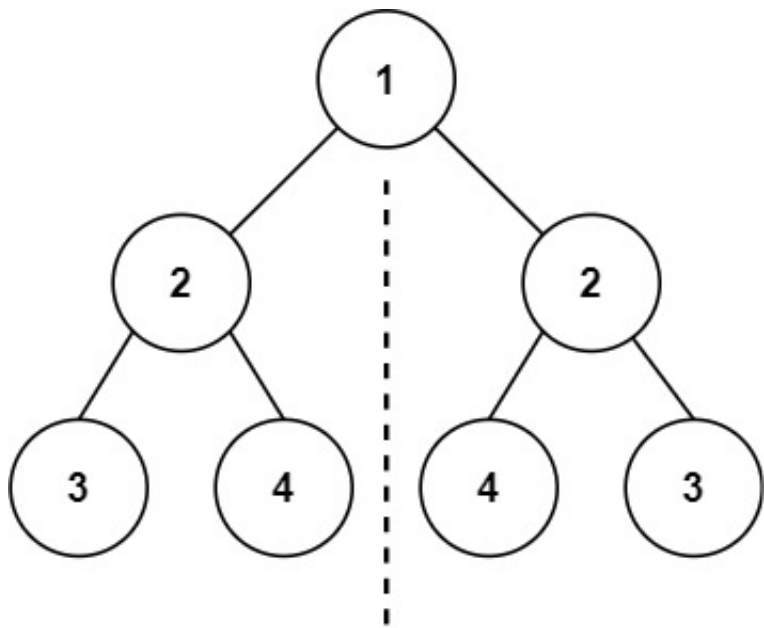
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 61/158  | 本章顺序 | 17/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:1541 |

LeetCode：https://leetcode.cn/problems/symmetric-tree/

题目内容

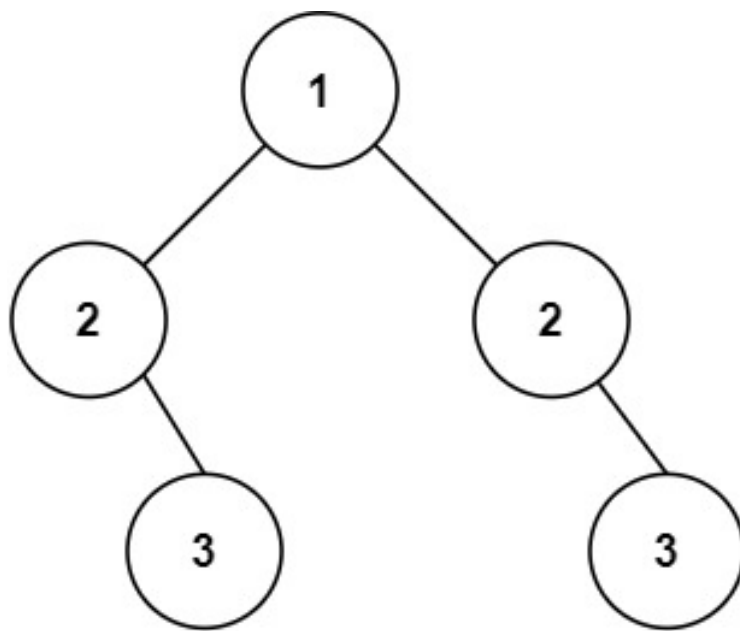
给你一个二叉树的根节点 root，检查它是否轴对称。

示例 1：



输入：root = [1,2,2,3,4,4,3]  
输出：true

示例 2：



输入：root = [1,2,2,null,3,null,3]  
输出：false

提示：

1 树中节点数目在范围 [1, 1000] 内

1  $-100 \leq \text{Node.val} \leq 100$

进阶：你可以运用递归和迭代两种方法解决这个问题吗？

我的题目思路与考察范围

核心思路：成对比较镜像位置：值相等，并递归比较外侧与内侧孩子。

关键不变量 / 状态：队列或递归中的每一对节点都应位于对称轴两侧的镜像位置。

复杂度：时间  $O(n)$ ，辅助空间最坏  $O(n)$ 。

考察范围：二叉树 · 镜像递归 · 成对遍历 · 结构比较

易错点：空节点必须成对判断；比较方向应为 left.left 对 right.right。

## 100. 相同的树

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 62/158  | 本章顺序 | 18/42                           |
| 难度   | 简单      | 题目类型 | 拓展/推荐                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:1787 |

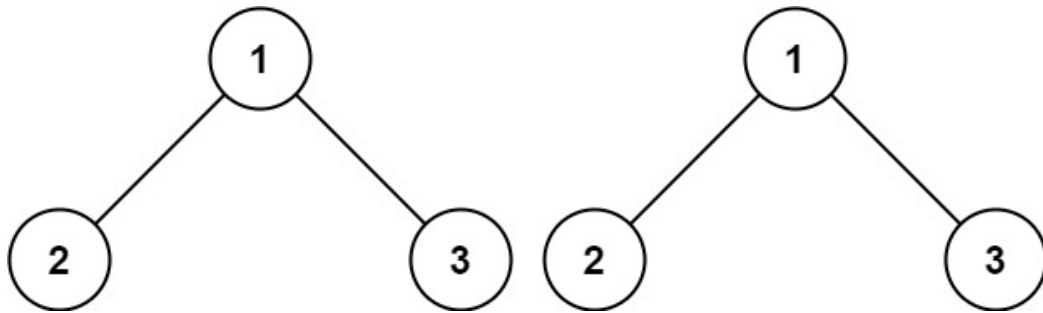
LeetCode：https://leetcode.cn/problems/same-tree/

### 题目内容

给你两棵二叉树的根节点 p 和 q，编写一个函数来检验这两棵树是否相同。

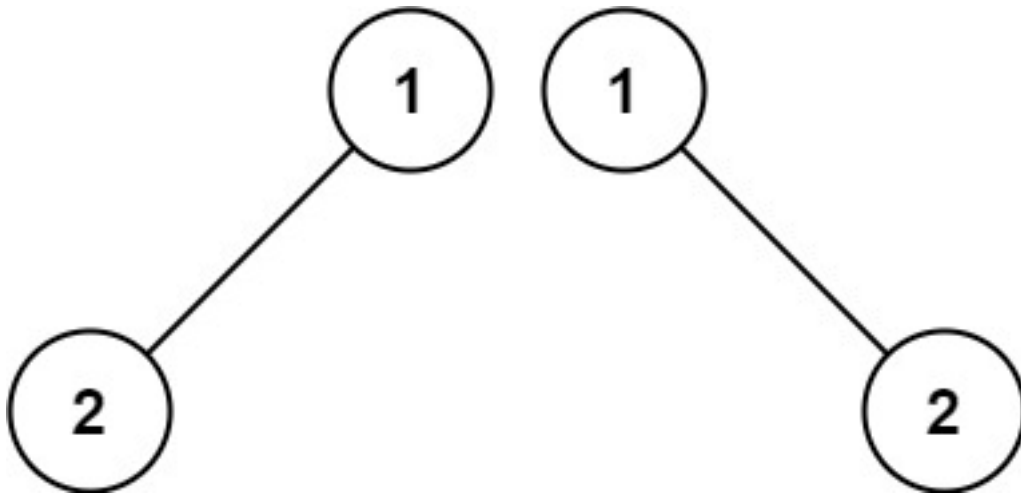
如果两个树在结构上相同，并且节点具有相同的值，则认为它们是相同的。

示例 1：



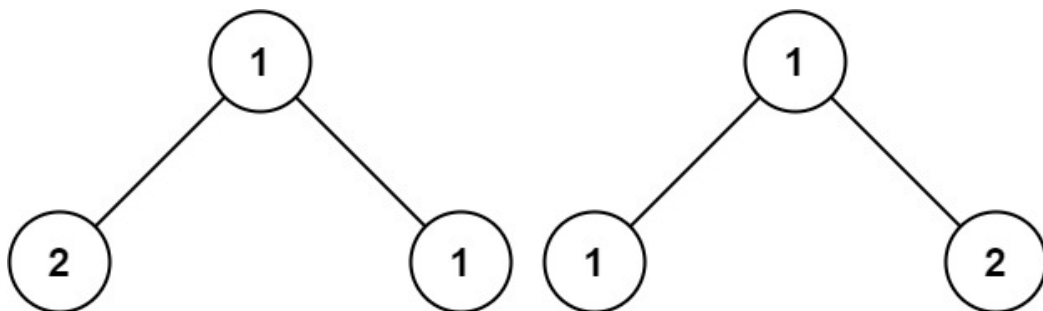
输入 :  $p = [1,2,3]$ ,  $q = [1,2,3]$   
 输出 : true

示例 2 :



输入 :  $p = [1,2]$ ,  $q = [1,null,2]$   
 输出 : false

示例 3 :



输入 :  $p = [1,2,1]$ ,  $q = [1,1,2]$   
 输出 : false

提示 :

- 1 两棵树上的节点数目都在范围  $[0, 100]$  内
- 1  $-10^4 \leq \text{Node.val} \leq 10^4$

我的题目思路与考察范围

核心思路：同步遍历两棵树，逐对检查空性、节点值以及左右子树。

关键不变量 / 状态：每次递归参数代表两棵树中应占据同一结构位置的节点。

复杂度：时间  $O(\min(n,m))$ ，辅助空间  $O(\min(h1,h2))$ 。

考察范围：二叉树 · 递归 · 结构比较 · 同步遍历

易错点：只比较遍历值序列会丢失空节点结构，不能证明两树相同。

## 572. 另一棵树的子树

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 63/158  | 本章顺序 | 19/42                           |
| 难度   | 简单      | 题目类型 | 拓展/推荐                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:1788 |

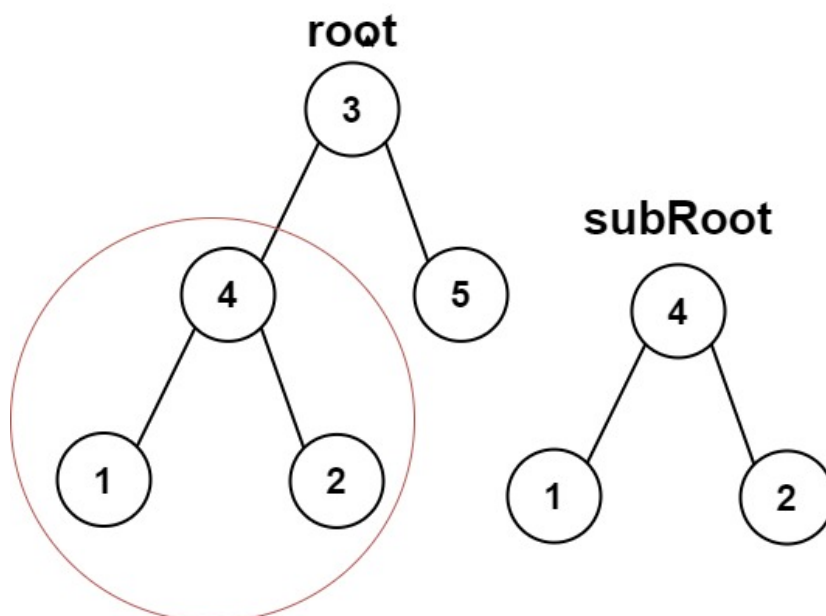
LeetCode : <https://leetcode.cn/problems/subtree-of-another-tree/>

### 题目内容

给你两棵二叉树 `root` 和 `subRoot`。检验 `root` 中是否包含和 `subRoot` 具有相同结构和节点值的子树。如果存在，返回 `true`；否则，返回 `false`。

二叉树 `tree` 的一棵子树包括 `tree` 的某个节点和这个节点的所有后代节点。`tree` 也可以看做它自身的一棵子树。

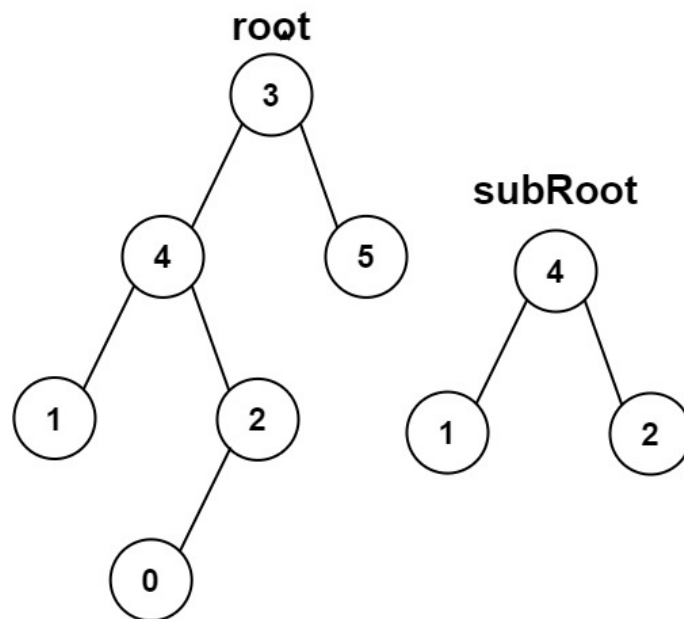
示例 1：



输入：root = [3,4,5,1,2], subRoot = [4,1,2]

输出：true

示例 2：



输入：root = [3,4,5,1,2,null,null,null,0], subRoot = [4,1,2]  
 输出：false

提示：

- 1 root 树上的节点数量范围是 [1, 2000]
- 1 subRoot 树上的节点数量范围是 [1, 1000]
- 1  $-10^4 \leq \text{root.val} \leq 10^4$
- 1  $-10^4 \leq \text{subRoot.val} \leq 10^4$

我的题目思路与考察范围

核心思路：遍历主树的每个节点作为候选根，并用相同树判断验证整棵结构。

关键不变量 / 状态：候选根匹配成功仅当从该点起的值和左右结构全部与 subRoot 相同。

复杂度：朴素时间  $O(nm)$ ，递归栈  $O(h)$ 。

考察范围：二叉树 · 递归 · 子树匹配 · 结构比较

易错点：匹配到根值相同还不够，必须继续验证完整左右结构。

## 559. N 叉树的最大深度

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 64/158  | 本章顺序 | 20/42                          |
| 难度   | 简单      | 题目类型 | 章节题单、拓展/推荐                     |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:1815 |

LeetCode：https://leetcode.cn/problems/maximum-depth-of-n-ary-tree/

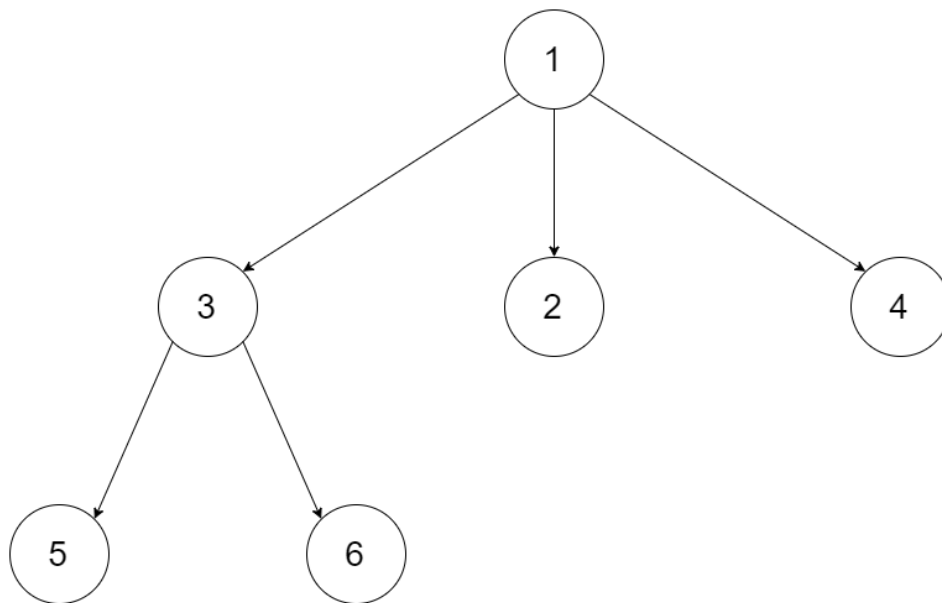
### 题目内容

给定一个 N 叉树，找到其最大深度。

最大深度是指从根节点到最远叶子节点的最长路径上的节点总数。

N 叉树输入按层序遍历序列化表示，每组子节点由空值分隔（请参见示例）。

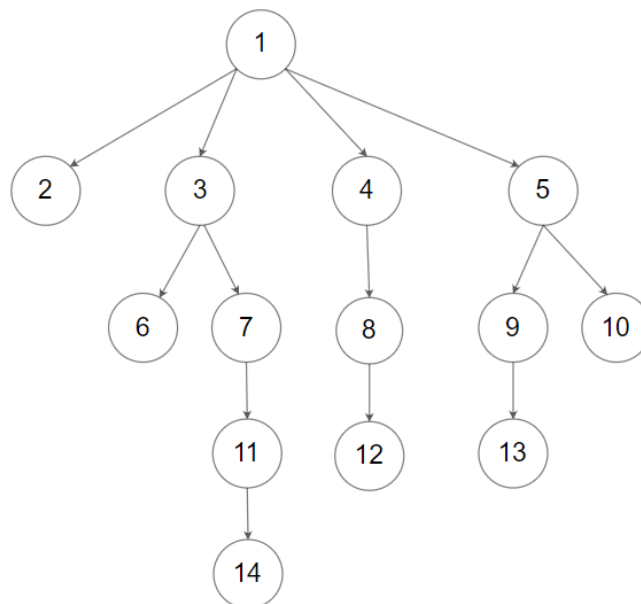
示例 1：



输入：root = [1,null,3,2,4,null,5,6]

输出：3

示例 2：



输入：root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]

输出：5

提示：

- 1 树的深度不会超过 1000。
- 1 树的节点数目位于  $[0, 10^4]$  之间。

我的题目思路与考察范围

核心思路：递归求所有孩子深度的最大值，再加一得到当前节点深度。

关键不变量 / 状态：当前节点返回前，每个孩子返回值都是其子树的正确最大深度。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：N叉树 · 递归 · 树高 · 后序遍历

易错点：叶节点 children 为空时最大孩子深度应按 0 处理。



## 222. 完全二叉树的节点个数

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 65/158  | 本章顺序 | 21/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:2284 |

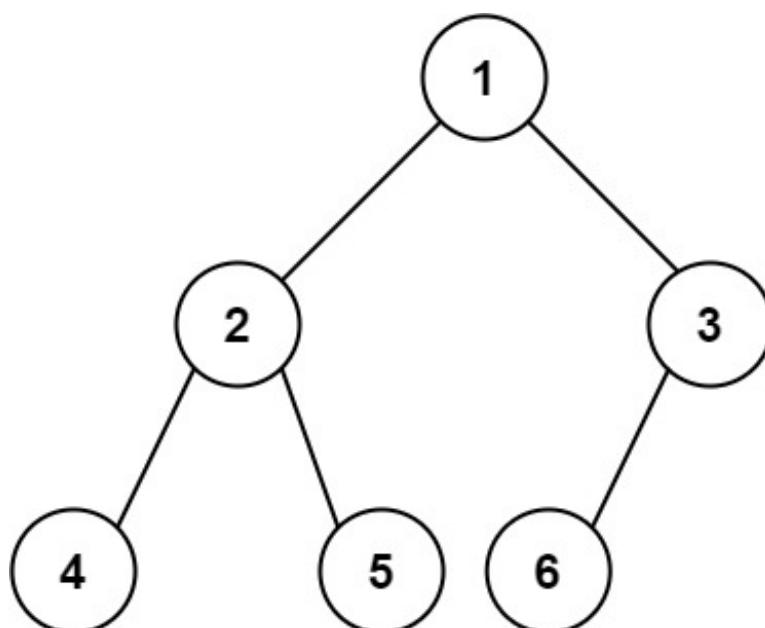
LeetCode : <https://leetcode.cn/problems/count-complete-tree-nodes/>

### 题目内容

给你一棵 完全二叉树 的根节点 `root`，求出该树的节点个数。

完全二叉树的定义如下：在完全二叉树中，除了最底层节点可能没填满外，其余每层节点数都达到最大值，并且最下面一层的节点都集中在该层最左边的若干位置。若最底层为第  $h$  层（从第 0 层开始），则该层包含  $1 \sim 2^h$  个节点。

示例 1：



输入：root = [1,2,3,4,5,6]  
输出：6

示例 2：

输入：root = []  
输出：0

示例 3：

输入：root = [1]  
输出：1

提示：

1 树中节点的数目范围是  $[0, 5 * 10^4]$

1  $0 \leq \text{Node.val} \leq 5 * 10^4$

1 题目数据保证输入的树是 完全二叉树

进阶：遍历树来统计节点是一种时间复杂度为  $O(n)$  的简单解决方案。你可以设计一个更快的算法吗？

我的题目思路与考察范围  
核心思路：比较最左链和最右链高度；相等则子树完美并直接算  $2^h-1$ ，否则递归两侧。  
关键不变量 / 状态：完全子树若左右边界高度相同，它必是满层完美二叉树。  
复杂度：时间  $O(\log^2 n)$ ，递归栈  $O(\log n)$ 。  
考察范围：完全二叉树 · 树高 · 递归 · 位运算  
易错点：高度与公式的边/节点计数必须统一，位移还要避免整数溢出。

110. 平衡二叉树

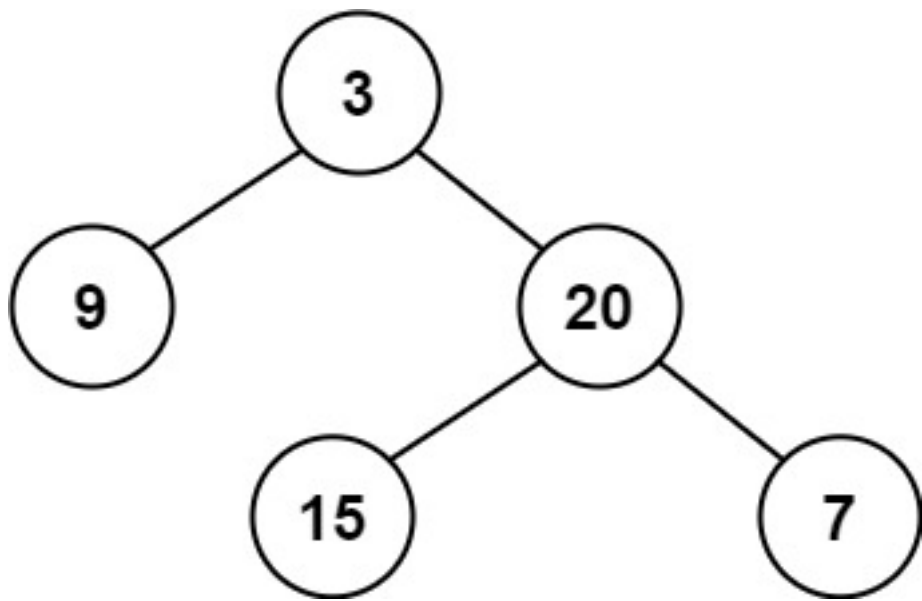
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 66/158  | 本章顺序 | 22/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:2538 |

LeetCode：https://leetcode.cn/problems/balanced-binary-tree/

题目内容

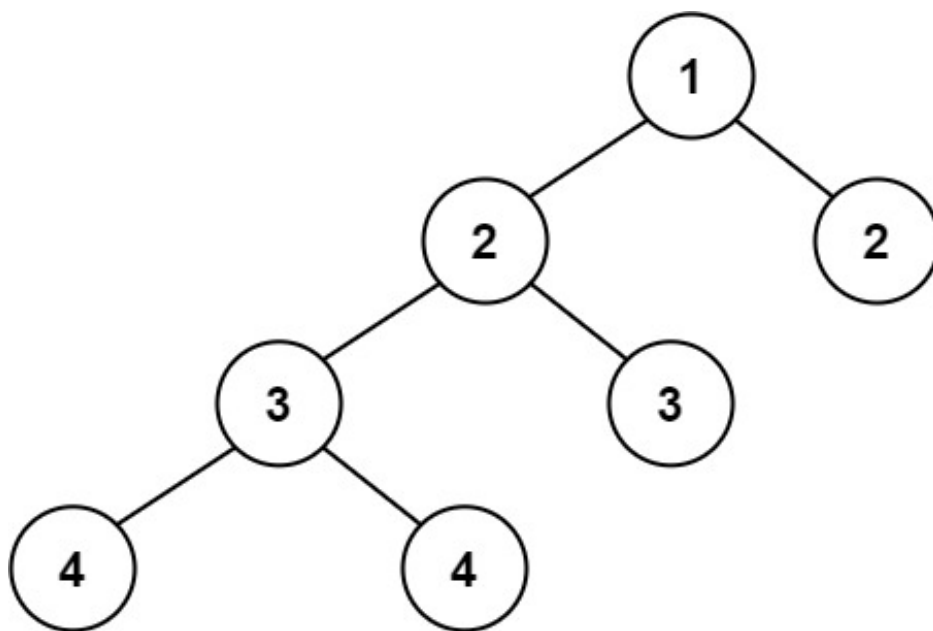
给定一个二叉树，判断它是否是 平衡二叉树

示例 1：



输入：root = [3,9,20,null,null,15,7]  
输出：true

示例 2：



输入：root = [1,2,2,3,3,null,null,4,4]  
输出：false

示例 3：

输入：root = []  
输出：true

提示：

1 树中的节点数在范围 [0, 5000] 内

1  $-10^4 \leq \text{Node.val} \leq 10^4$

我的题目思路与考察范围

核心思路：后序返回子树高度；发现任一节点高度差超过 1 就返回 -1 作为失衡哨兵。

关键不变量 / 状态：非 -1 返回值既表示子树平衡，也精确表示该子树高度。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉树 · 后序遍历 · 高度 · 剪枝

易错点：若对每个节点另算一次高度会退化到  $O(n^2)$ 。

## 257. 二叉树的所有路径

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 67/158  | 本章顺序 | 23/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:2879 |

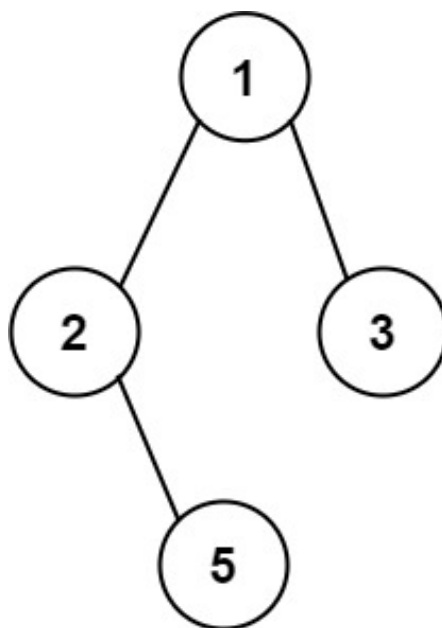
LeetCode：https://leetcode.cn/problems/binary-tree-paths/

### 题目内容

给你一个二叉树的根节点 root，按任意顺序，返回所有从根节点到叶子节点的路径。

叶子节点 是指没有子节点的节点。

示例 1：



输入：root = [1,2,3,null,5]  
输出：["1->2->5","1->3"]

示例 2：

输入：root = [1]  
输出：["1"]

提示：

1 树中节点的数目在范围 [1, 100] 内

1 -100 <= Node.val <= 100

我的题目思路与考察范围

核心思路：DFS 维护当前根到节点的路径，到叶子时序列化并收集，返回时回溯。

关键不变量 / 状态：进入节点后 path 恰好表示从根到当前节点的完整路径。

复杂度：时间  $O(nh)$ （含字符串复制），递归栈与路径  $O(h)$ 。

考察范围：二叉树 · DFS · 回溯 · 路径构造

易错点：只在叶子收集；递归返回前必须撤销当前节点或边的路径状态。

## 404. 左叶子之和

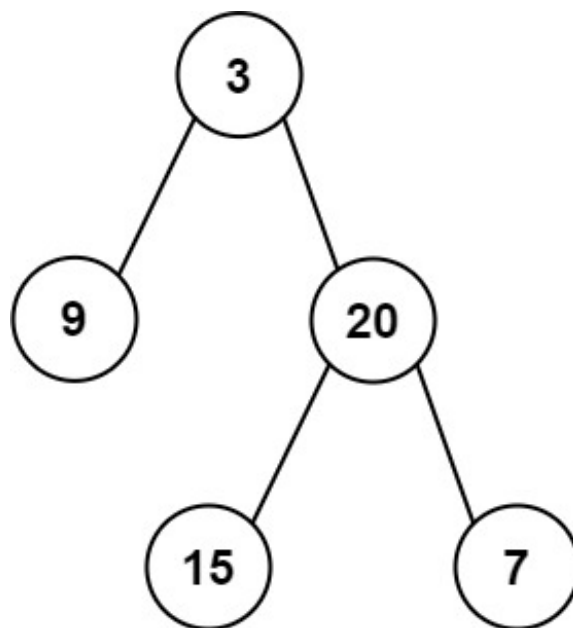
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 68/158  | 本章顺序 | 24/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:3515 |

LeetCode：https://leetcode.cn/problems/sum-of-left-leaves/

### 题目内容

给定二叉树的根节点 root，返回所有左叶子之和。

示例 1：



输入: root = [3,9,20,null,null,15,7]

输出: 24

解释: 在这个二叉树中, 有两个左叶子, 分别是 9 和 15, 所以返回 24

示例 2:

输入: root = [1]

输出: 0

提示:

1 节点数在 [1, 1000] 范围内

1  $-1000 \leq \text{Node.val} \leq 1000$

我的题目思路与考察范围

核心思路: 遍历父节点, 若其左孩子是叶子就累加, 否则继续递归。

关键不变量 / 状态: 一个值只在它同时满足“父节点左孩子”和“自身为叶子”时计入。

复杂度: 时间  $O(n)$ , 递归栈  $O(h)$ 。

考察范围: 二叉树 · DFS · 叶子判定 · 父子关系

易错点: 最左侧节点不等于左叶子, 根节点即使无孩子也不算左叶子。

## 513. 找树左下角的值

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 69/158  | 本章顺序 | 25/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:3679 |

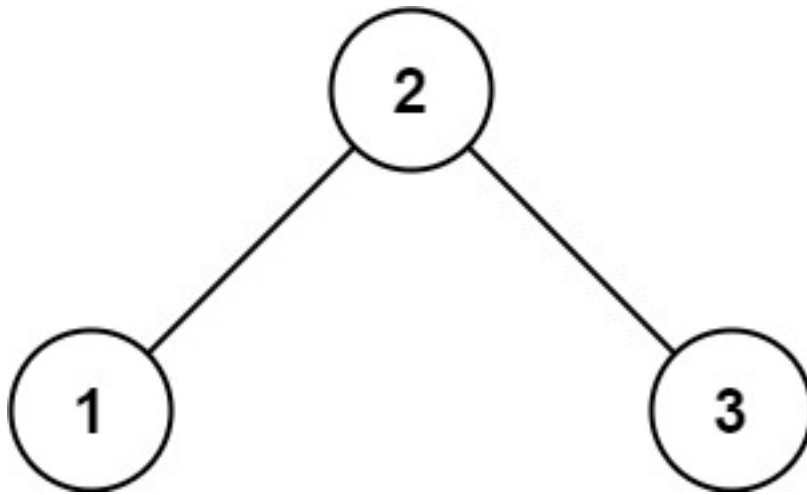
LeetCode : <https://leetcode.cn/problems/find-bottom-left-tree-value/>

### 题目内容

给定一个二叉树的 根节点 root , 请找出该二叉树的 最底层 最左边 节点的值。

假设二叉树中至少有一个节点。

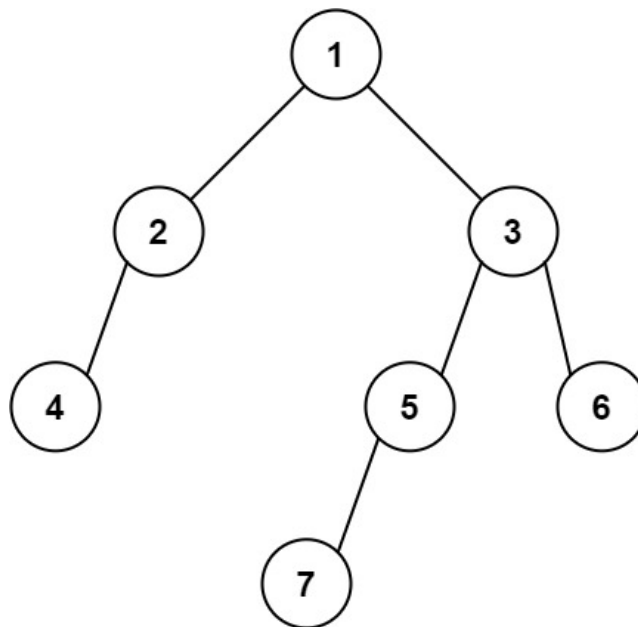
示例 1:



输入: root = [2,1,3]

输出: 1

示例 2:



输入: [1,2,3,4,null,5,6,null,null,7]

输出: 7

提示:

1 二叉树的节点个数的范围是  $[1, 10^4]$

1  $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

我的题目思路与考察范围

核心思路：按层 BFS，每层记录第一个节点；最后一层的第一个值即答案。

关键不变量 / 状态：同层按左到右入队，因此每轮首次出队节点是该层最左节点。

复杂度：时间  $O(n)$ ，空间  $O(w)$ 。

考察范围：二叉树 · BFS · 层序遍历 · 最深层

易错点：DFS 解法必须先左后右，且只在首次到达更深层时更新答案。

## 112. 路径总和

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 70/158  | 本章顺序 | 26/42                           |
| 难度   | 简单      | 题目类型 | 主讲题、章节题单                        |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:3876 |

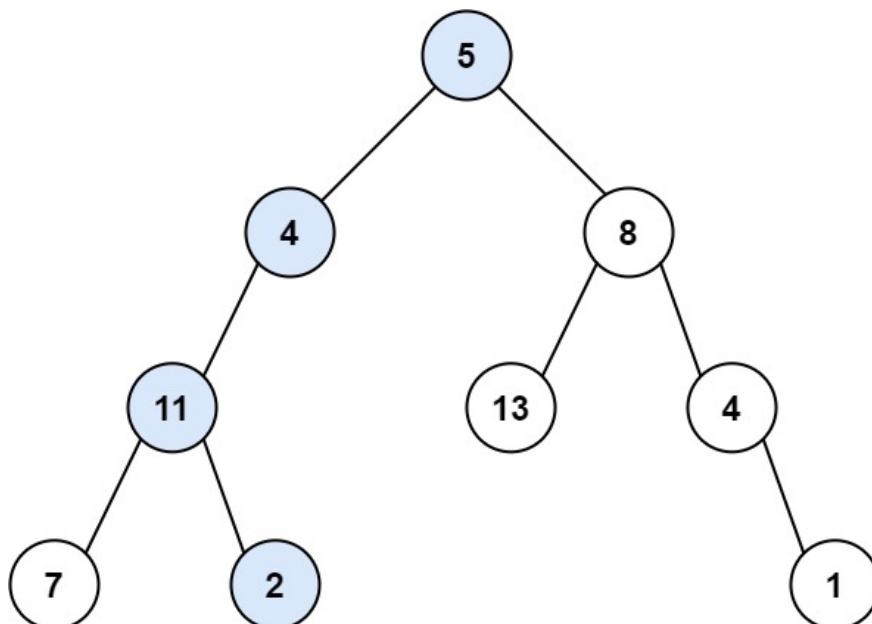
LeetCode : <https://leetcode.cn/problems/path-sum/>

### 题目内容

给你二叉树的根节点 `root` 和一个表示目标和的整数 `targetSum` 。判断该树中是否存在 根节点到叶子节点 的路径，这条路径上所有节点值相加等于目标和 `targetSum` 。如果存在，返回 `true` ；否则，返回 `false` 。

叶子节点 是指没有子节点的节点。

示例 1：

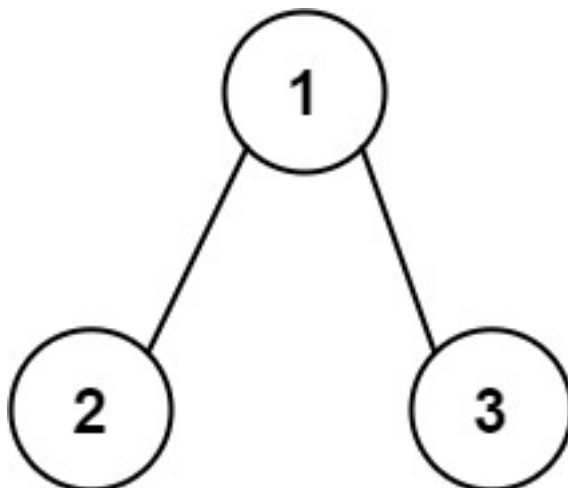


输入：root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22

输出：true

解释：等于目标和的根节点到叶节点路径如上图所示。

示例 2：



输入：root = [1,2,3], targetSum = 5  
 输出：false  
 解释：树中存在两条根节点到叶子节点的路径：  
 (1 --> 2): 和为 3  
 (1 --> 3): 和为 4  
 不存在 sum = 5 的根节点到叶子节点的路径。

示例 3：

输入：root = [], targetSum = 0  
 输出：false  
 解释：由于树是空的，所以不存在根节点到叶子节点的路径。

提示：

- 1 树中节点的数目在范围 [0, 5000] 内
- 1  $-1000 \leq \text{Node.val} \leq 1000$
- 1  $-1000 \leq \text{targetSum} \leq 1000$

我的题目思路与考察范围

核心思路：沿根到叶路径递减目标值，在叶子检查剩余值是否等于节点值。

关键不变量 / 状态：递归参数始终表示当前路径尚需补足的和。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉树 · DFS · 根到叶路径 · 目标和

易错点：不能在非叶节点提前以累计和相等判成功，路径必须终止于叶子。

## 113. 路径总和 II

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 71/158  | 本章顺序 | 27/42                           |
| 难度   | 中等      | 题目类型 | 章节题单、拓展/推荐                      |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:3899 |

LeetCode：https://leetcode.cn/problems/path-sum-ii/

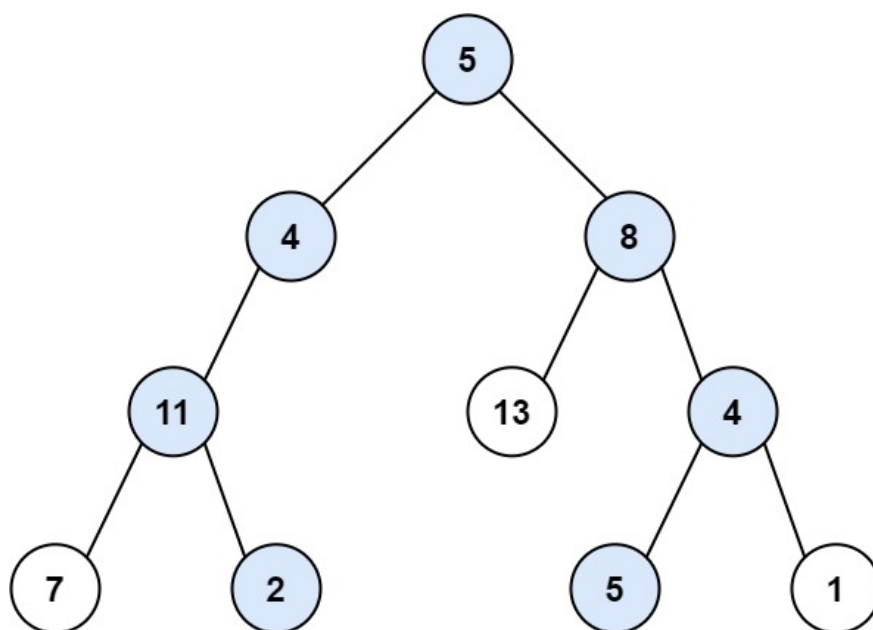
### 题目内容

给你二叉树的根节点 root 和一个整数目标和 targetSum，找出所有从根节点到叶子节点路径总和等于给定目标和的路径。

叶子节点是指没有子节点的节点。

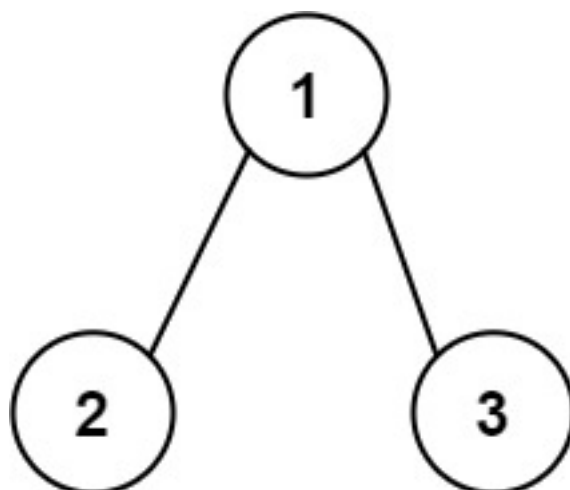
示例 1：





输入：root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22  
输出：[[5,4,11,2],[5,8,4,5]]

示例 2：



输入：root = [1,2,3], targetSum = 5  
输出：[]

示例 3：

输入：root = [1,2], targetSum = 0  
输出：[]

提示：

- 1 树中节点总数在范围 [0, 5000] 内
- 1  $-1000 \leq \text{Node.val} \leq 1000$
- 1  $-1000 \leq \text{targetSum} \leq 1000$

**我的题目思路与考察范围**

核心思路：DFS 同时维护剩余和与当前路径，叶子满足目标时复制路径到结果。

关键不变量 / 状态：进入节点后 path 与 remaining 分别对应同一条根到当前节点路径及其剩余和。

复杂度：遍历  $O(n)$ ，计入结果复制为  $O(nh)$ ，辅助空间  $O(h)$ 。

考察范围：二叉树 · DFS · 回溯 · 路径枚举

易错点：左右递归返回后都要 pop\_back，否则兄弟分支会共享脏路径。

## 106. 从中序与后序遍历序列构造二叉树

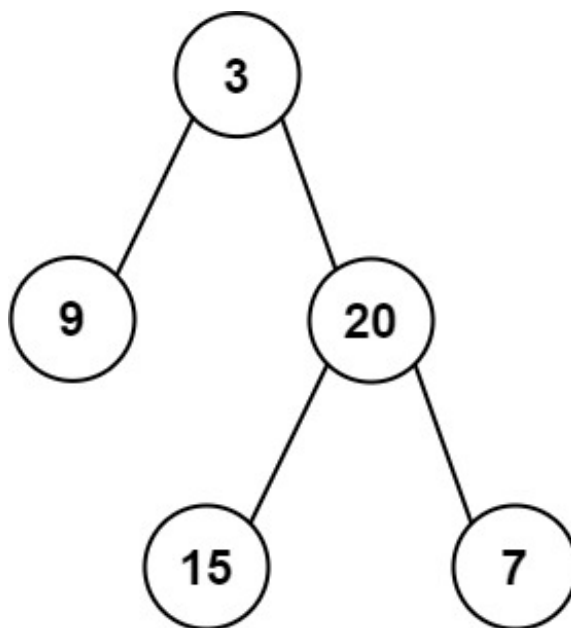
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 72/158  | 本章顺序 | 28/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:4169 |

LeetCode： <https://leetcode.cn/problems/construct-binary-tree-from-inorder-and-postorder-traversal/>

### 题目内容

给定两个整数数组 inorder 和 postorder，其中 inorder 是二叉树的中序遍历，postorder 是同一棵树的后序遍历，请你构造并返回这颗 二叉树。

示例 1:



输入：inorder = [9,3,15,20,7], postorder = [9,15,7,20,3]

输出：[3,9,20,null,null,15,7]

示例 2:

输入：inorder = [-1], postorder = [-1]

输出：[-1]

提示:

- 1 1 <= inorder.length <= 3000
- 1 postorder.length == inorder.length
- 1 -3000 <= inorder[i], postorder[i] <= 3000

- 1 inorder 和 postorder 都由 不同 的值组成
- 1 postorder 中每一个值都在 inorder 中
- 1 inorder 保证 是树的中序遍历
- 1 postorder 保证 是树的后序遍历

我的题目思路与考察范围

核心思路：后序末元素是根；用哈希定位其中序位置，再按左子树长度切分两个序列。

关键不变量 / 状态：每次递归的中序区间与后序区间包含完全相同的一组节点且长度相等。

复杂度：哈希定位后时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：二叉树构造 · 递归 · 中序遍历 · 后序遍历

易错点：后序右区间必须排除末尾根节点，区间开闭规则要始终一致。

## 105. 从前序与中序遍历序列构造二叉树

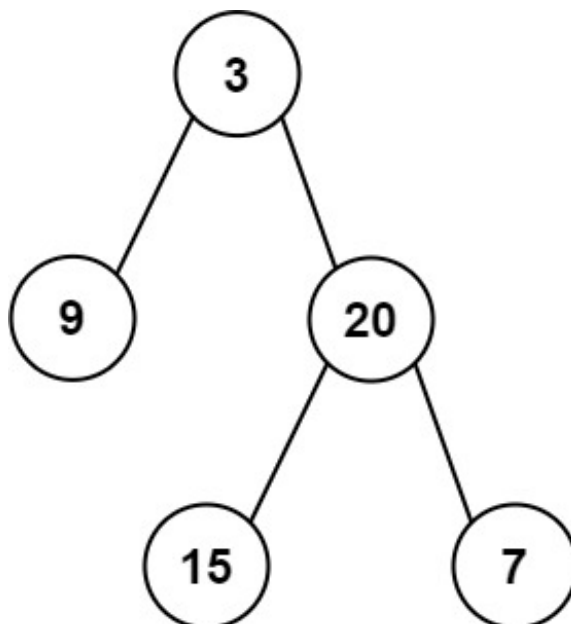
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 73/158  | 本章顺序 | 29/42                           |
| 难度   | 中等      | 题目类型 | 拓展/推荐                           |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:4512 |

LeetCode： <https://leetcode.cn/problems/construct-binary-tree-from-preorder-and-inorder-traversal/>

### 题目内容

给定两个整数数组 preorder 和 inorder，其中 preorder 是二叉树的 先序遍历，inorder 是同一棵树的 中序遍历，请构造二叉树并返回其根节点。

示例 1:



输入: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]

输出: [3,9,20,null,null,15,7]

示例 2:

输入: preorder = [-1], inorder = [-1]

输出: [-1]

提示:

- 1 1 <= preorder.length <= 3000
- 1 inorder.length == preorder.length
- 1 -3000 <= preorder[i], inorder[i] <= 3000
- 1 preorder 和 inorder 均 无重复 元素
- 1 inorder 均出现在 preorder
- 1 preorder 保证 为二叉树的前序遍历序列
- 1 inorder 保证 为二叉树的中序遍历序列

我的题目思路与考察范围

核心思路：前序首元素是根；在中序中定位根并用左子树长度划分左右区间。

关键不变量 / 状态：递归传入的前序与中序区间对应同一棵子树。

复杂度：哈希定位后时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：二叉树构造 · 递归 · 前序遍历 · 中序遍历

易错点：前序左区间长度必须等于中序根左侧元素个数，避免错一位。

## 654. 最大二叉树

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 74/158  | 本章顺序 | 30/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:4680 |

LeetCode : <https://leetcode.cn/problems/maximum-binary-tree/>

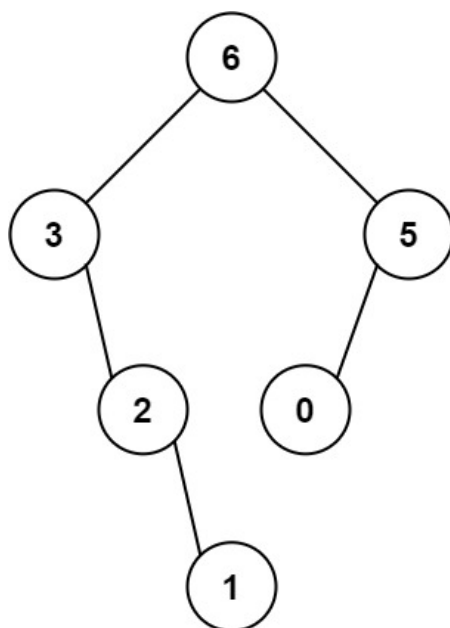
### 题目内容

给定一个不重复的整数数组 `nums` 。最大二叉树 可以用下面的算法从 `nums` 递归地构建:

- 1 创建一个根节点，其值为 `nums` 中的最大值。
- 2 递归地在最大值 左边 的 子数组前缀上 构建左子树。
- 3 递归地在最大值 右边 的 子数组后缀上 构建右子树。

返回 `nums` 构建的 最大二叉树 。

示例 1：



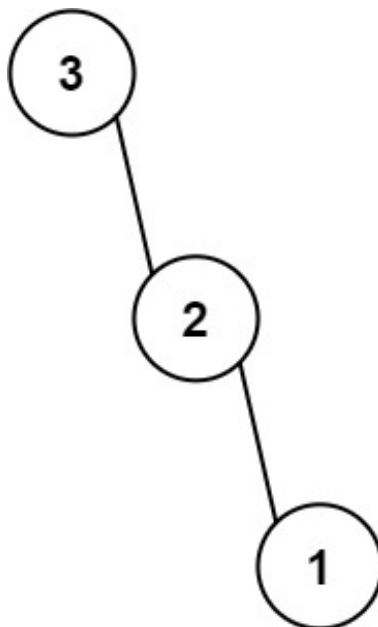
输入：nums = [3,2,1,6,0,5]

输出：[6,3,5,null,2,0,null,null,1]

解释：递归调用如下所示：

- [3,2,1,6,0,5] 中的最大值是 6，左边部分是 [3,2,1]，右边部分是 [0,5]。
- [3,2,1] 中的最大值是 3，左边部分是 []，右边部分是 [2,1]。
- 空数组，无子节点。
- [2,1] 中的最大值是 2，左边部分是 []，右边部分是 [1]。
- 空数组，无子节点。
- 只有一个元素，所以子节点是一个值为 1 的节点。
- [0,5] 中的最大值是 5，左边部分是 [0]，右边部分是 []。
- 只有一个元素，所以子节点是一个值为 0 的节点。
- 空数组，无子节点。

示例 2：



输入：nums = [3,2,1]

输出：[3,null,2,null,1]

提示：

1 1 <= nums.length <= 1000

1 0 <= nums[i] <= 1000

1 nums 中的所有整数 互不相同

我的题目思路与考察范围

核心思路：区间最大值作为根，最大值左、右两段递归构造左右子树。

关键不变量 / 状态：每个递归区间只生成该区间元素对应的唯一最大二叉树。

复杂度：朴素时间最坏  $O(n^2)$ 、栈  $O(n)$ ；单调栈可降至  $O(n)$ 。

考察范围：二叉树构造 · 分治 · 区间最大值 · 单调栈

易错点：递归切片频繁复制数组会增加开销，应传半开下标区间。

## 617. 合并二叉树

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 75/158  | 本章顺序 | 31/42                          |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:4970 |

LeetCode：https://leetcode.cn/problems/merge-two-binary-trees/

### 题目内容

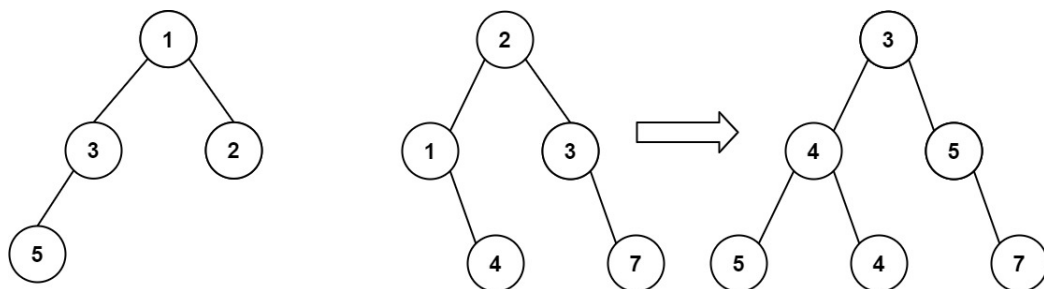
给你两棵二叉树：root1 和 root2。

想象一下，当你将其中一棵覆盖到另一棵之上时，两棵树上的一些节点将会重叠（而另一些不会）。你需要将这两棵树合并成一棵新二叉树。合并的规则是：如果两个节点重叠，那么将这两个节点的值相加作为合并后节点的新值；否则，不为 null 的节点将直接作为新二叉树的节点。

返回合并后的二叉树。

注意: 合并过程必须从两个树的根节点开始。

示例 1：



输入：root1 = [1,3,2,5], root2 = [2,1,3,null,4,null,7]  
输出：[3,4,5,5,4,null,7]

示例 2：

输入：root1 = [1], root2 = [1,2]  
输出：[2,2]

提示：

1 两棵树中的节点数目在范围 [0, 2000] 内

1  $-10^4 \leq \text{Node.val} \leq 10^4$

## 我的题目思路与考察范围

核心思路：同步递归两棵树；两节点都存在就求和值，只有一侧存在就沿用该子树。

关键不变量 / 状态：返回节点代表两棵输入树在同一位置合并后的完整子树。

复杂度：时间  $O(n+m)$ ，递归栈  $O(\max(h_1, h_2))$ 。

考察范围：二叉树 · 同步递归 · 结构合并 · 节点复用

易错点：若复用原节点会修改输入树；需要保留输入时必须新建节点。

## 700. 二叉搜索树中的搜索

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 76/158  | 本章顺序 | 32/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:5217 |

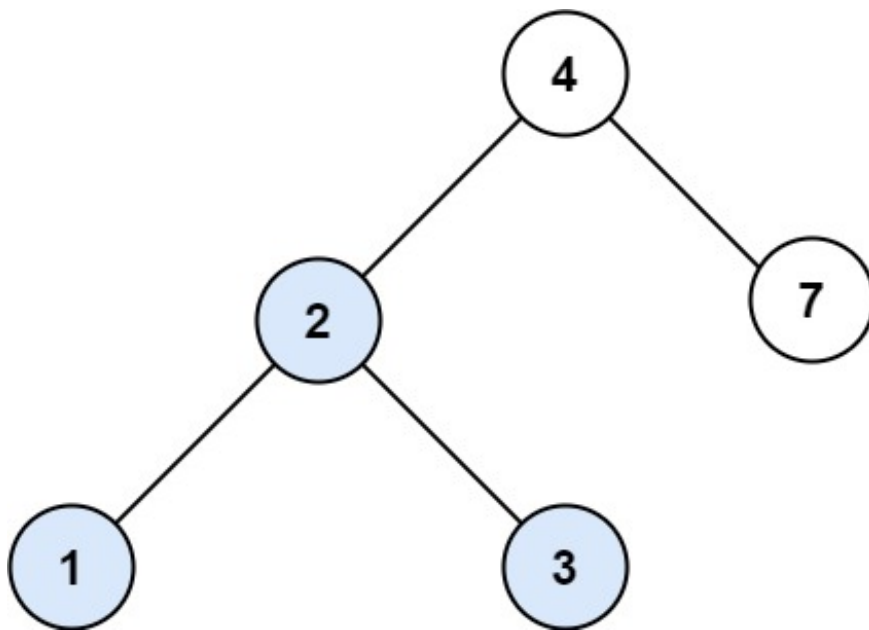
LeetCode : <https://leetcode.cn/problems/search-in-a-binary-search-tree/>

## 题目内容

给定二叉搜索树 (BST) 的根节点 root 和一个整数值 val。

你需要在 BST 中找到节点值等于 val 的节点。返回以该节点为根的子树。如果节点不存在，则返回 null。

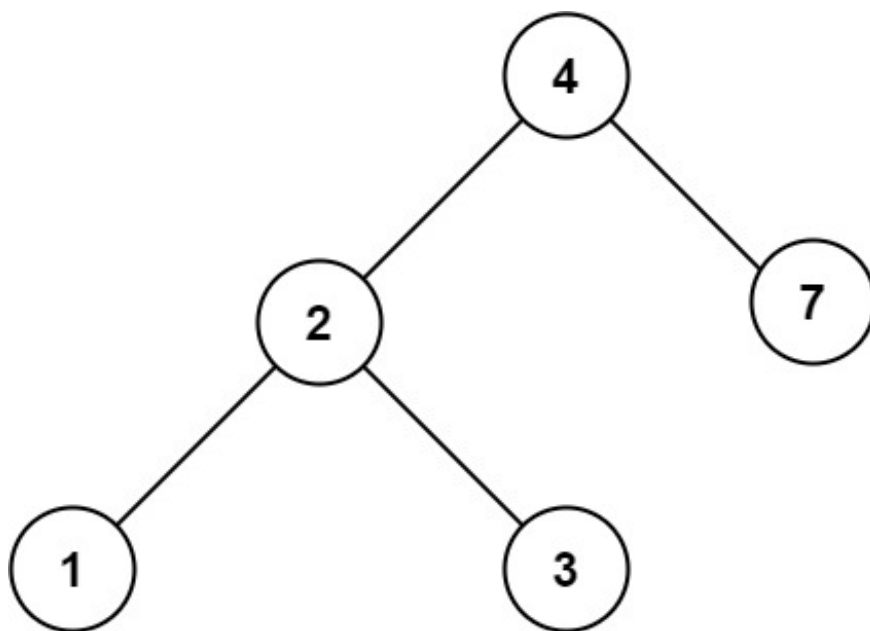
示例 1:



输入：root = [4,2,7,1,3], val = 2

输出：[2,1,3]

示例 2:



输入：root = [4,2,7,1,3], val = 5  
输出：[]

提示：

- 1 树中节点数在 [1, 5000] 范围内
- 1  $1 \leq \text{Node.val} \leq 10^7$
- 1 root 是二叉搜索树
- 1  $1 \leq \text{val} \leq 10^7$

我的题目思路与考察范围

核心思路：利用 BST 大小关系，每次只向可能包含目标的一侧下降。

关键不变量 / 状态：若目标存在，它始终位于当前节点代表的搜索子树中。

复杂度：时间  $O(h)$ ，额外空间迭代为  $O(1)$ 。

考察范围：二叉搜索树 · 有序性 · 查找 · 迭代

易错点：不要像普通二叉树一样同时搜索两侧，否则丢失 BST 的效率优势。

## 98. 验证二叉搜索树

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 77/158  | 本章顺序 | 33/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:5366 |

LeetCode：https://leetcode.cn/problems/validate-binary-search-tree/

### 题目内容

给你一个二叉树的根节点 root，判断其是否是一个有效的二叉搜索树。

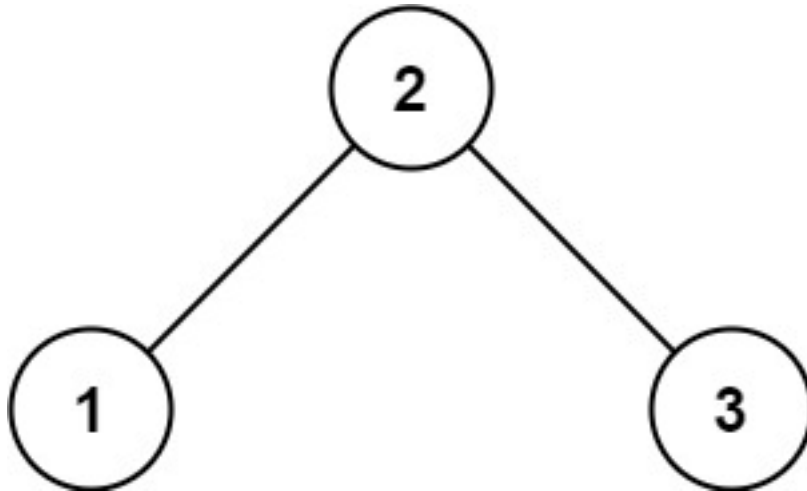
有效 二叉搜索树定义如下：

- 1 节点的左子树只包含严格小于当前节点的数。
- 1 节点的右子树只包含严格大于当前节点的数。



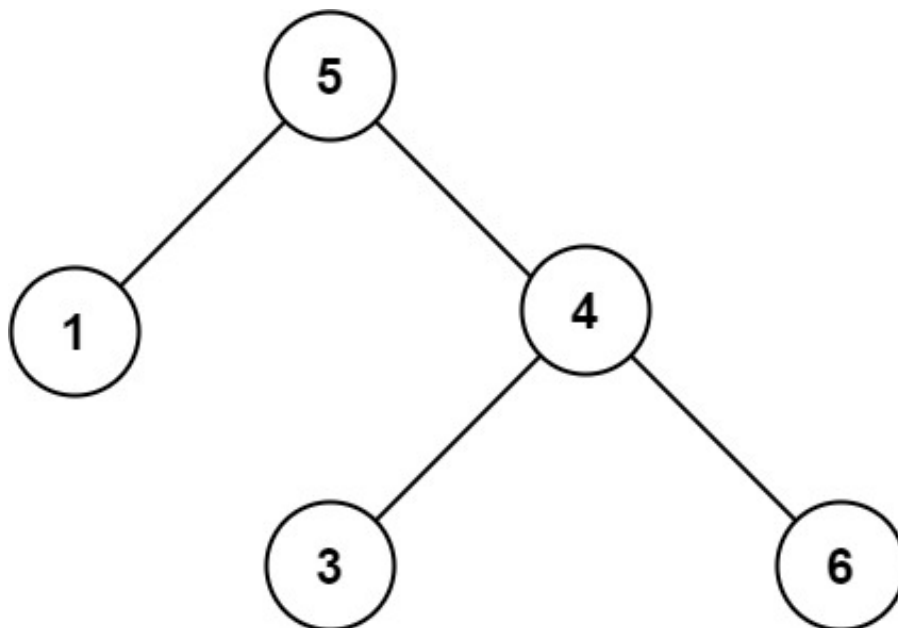
1 所有左子树和右子树自身必须也是二叉搜索树。

示例 1：



输入：root = [2,1,3]  
输出：true

示例 2：



输入：root = [5,1,4,null,null,3,6]  
输出：false  
解释：根节点的值是 5，但是右子节点的值是 4。

提示：

1 树中节点数目范围在  $[1, 10^4]$  内

1  $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

我的题目思路与考察范围

核心思路：中序遍历必须严格递增；保存前一个节点值逐项比较。

关键不变量 / 状态：处理当前节点前，prev 是中序序列中紧邻当前值的前驱。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉搜索树 · 中序遍历 · 有序性 · 边界

易错点：只比较父子关系不够；prev 应用 long 或节点指针处理极值与重复值。

## 530. 二叉搜索树的最小绝对差

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 78/158  | 本章顺序 | 34/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:5606 |

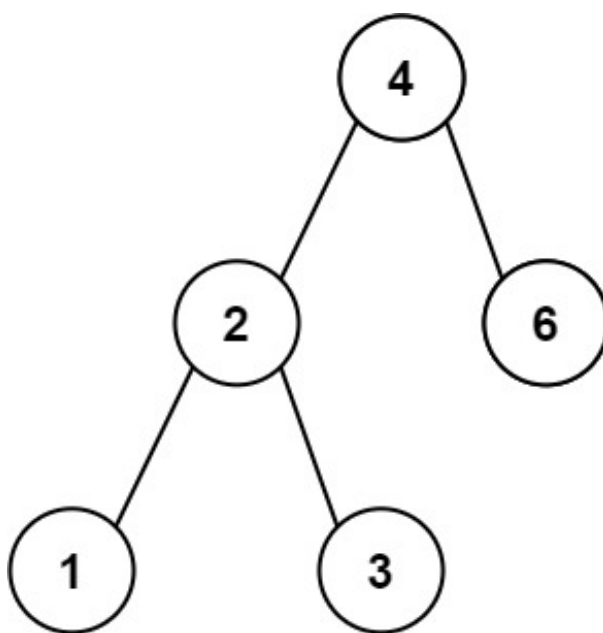
LeetCode : <https://leetcode.cn/problems/minimum-absolute-difference-in-bst/>

### 题目内容

给你一个二叉搜索树的根节点 `root`，返回 树中任意两不同节点值之间的最小差值。

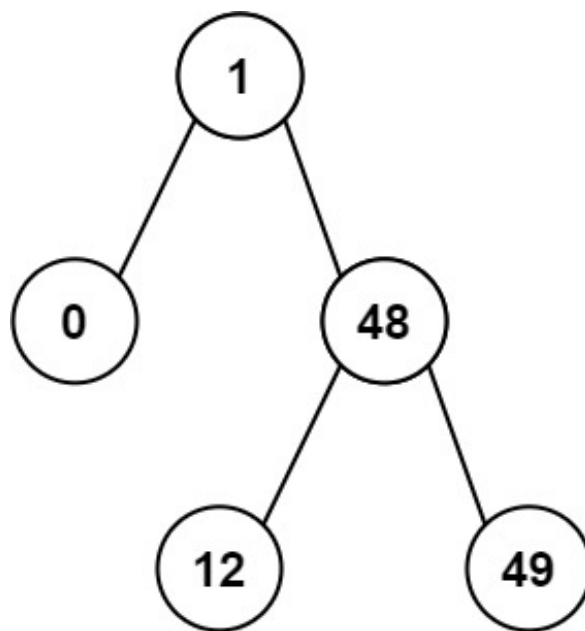
差值是一个正数，其数值等于两值之差的绝对值。

示例 1：



输入：root = [4,2,6,1,3]  
输出：1

示例 2：



输入：root = [1,0,48,null,null,12,49]  
输出：1

提示：

1 树中节点的数目范围是  $[2, 10^4]$

1  $0 \leq \text{Node.val} \leq 10^5$

注意：本题与 783 <https://leetcode.cn/problems/minimum-distance-between-bst-nodes/> 相同

我的题目思路与考察范围

核心思路：BST 中序序列有序，最小差必出现在相邻元素之间。

关键不变量 / 状态：访问当前值时，prev 是所有已访问值中最大的，因此也是最近前驱。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉搜索树 · 中序遍历 · 相邻差 · 有序序列

易错点：不要比较所有节点对；第一个中序节点没有前驱，需单独跳过。

## 501. 二叉搜索树中的众数

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 79/158  | 本章顺序 | 35/42                          |
| 难度   | 简单      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:5743 |

LeetCode： <https://leetcode.cn/problems/find-mode-in-binary-search-tree/>

### 题目内容

给你一个含重复值的二叉搜索树 (BST) 的根节点 root，找出并返回 BST 中的所有众数（即，出现频率最高的元素）。

如果树中有不止一个众数，可以按任意顺序返回。

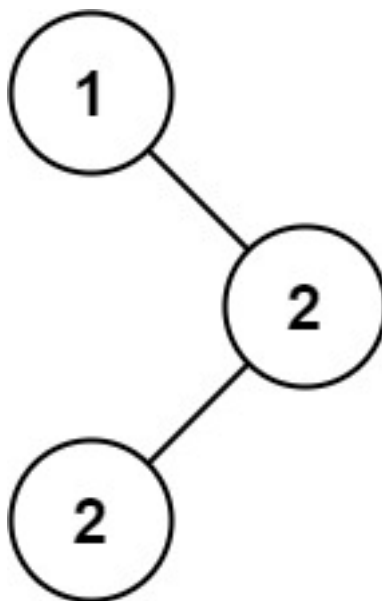
假定 BST 满足如下定义：

1 结点左子树中所含节点的值 小于等于 当前节点的值

1 结点右子树中所含节点的值 大于等于 当前节点的值

## 1 左子树和右子树都是二叉搜索树

示例 1：



输入：root = [1,null,2,2]

输出：[2]

示例 2：

输入：root = [0]

输出：[0]

提示：

1 树中节点的数目在范围  $[1, 10^4]$  内1  $-10^5 \leq \text{Node.val} \leq 10^5$ 

进阶：你可以不使用额外的空间吗？（假设由递归产生的隐式调用栈的开销不被计算在内）

我的题目思路与考察范围

核心思路：中序遍历把相同值聚成连续段，维护当前频次、最高频次和众数列表。

关键不变量 / 状态：访问当前节点后 count 等于其值在当前连续段中的出现次数。

复杂度：时间  $O(n)$ ，除递归栈和答案外额外  $O(1)$ 。

考察范围：二叉搜索树 · 中序遍历 · 频次统计 · 状态机

易错点：发现更高频次时必须清空旧答案；等于最高频次时才追加。

## 236. 二叉树的最近公共祖先

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 80/158  | 本章顺序 | 36/42                          |
| 难度   | 中等      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:6070 |

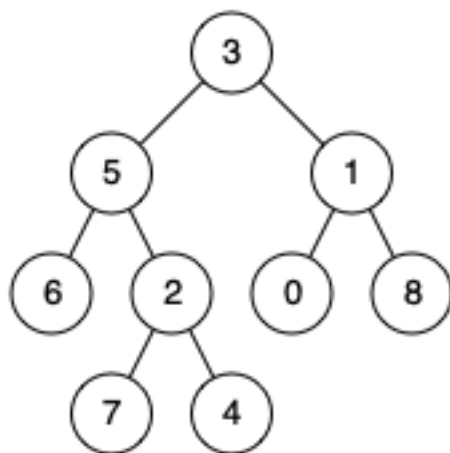
LeetCode：https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-tree/

## 题目内容

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。

百度百科 中最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

示例 1：

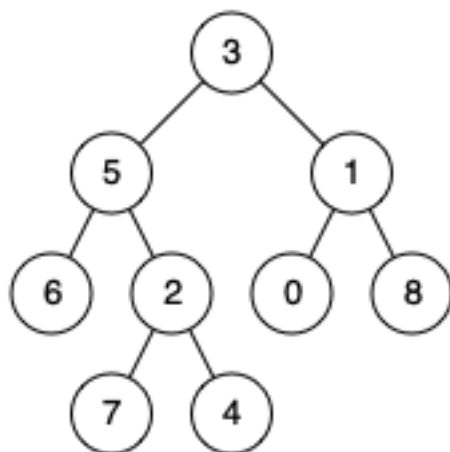


输入：root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

输出：3

解释：节点 5 和节点 1 的最近公共祖先是节点 3。

示例 2：



输入：root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4

输出：5

解释：节点 5 和节点 4 的最近公共祖先是节点 5。因为根据定义最近公共祖先节点可以为节点本身。

示例 3：

输入：root = [1,2], p = 1, q = 2

输出：1

提示：

- 1 树中节点数目在范围  $[2, 10^5]$  内。
- 1  $-10^9 \leq \text{Node.val} \leq 10^9$
- 1 所有  $\text{Node.val}$  互不相同。
- 1  $p \neq q$

1 p 和 q 均存在于给定的二叉树中。

我的题目思路与考察范围

核心思路：后序递归：命中 p/q 就返回自身；左右都非空时当前节点是最近公共祖先。

关键不变量 / 状态：递归返回值表示该子树中已找到的 p、q 或二者的最近汇合点。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉树 · 后序遍历 · 最近公共祖先 · 递归返回值

易错点：命中 p 或 q 要立即返回，因为祖先节点本身也可能是答案。

## 235. 二叉搜索树的最近公共祖先

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 81/158  | 本章顺序 | 37/42                          |
| 难度   | 中等      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:6408 |

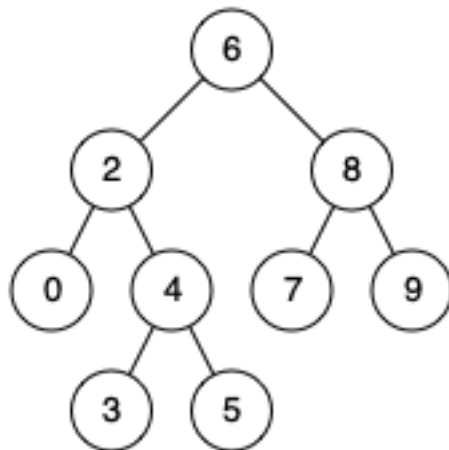
LeetCode：https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-search-tree/

### 题目内容

给定一个二叉搜索树，找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为：“对于有根树 T 的两个结点 p、q，最近公共祖先表示为一个结点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

例如，给定如下二叉搜索树: root = [6,2,8,0,4,7,9,null,null,3,5]



示例 1:

输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

输出: 6

解释: 节点 2 和节点 8 的最近公共祖先是 6。

示例 2:

输入: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

输出: 2

解释: 节点 2 和节点 4 的最近公共祖先是 2，因为根据定义最近公共祖先节点可以为节点本身。

说明:

1 所有节点的值都是唯一的。

1 p、q 为不同节点且均存在于给定的二叉搜索树中。

我的题目思路与考察范围

核心思路：利用 BST 有序性：两目标同小则向左，同大则向右，否则当前节点是分叉点。

关键不变量 / 状态：下降过程中 p、q 始终都位于当前节点的子树内。

复杂度：时间  $O(h)$ ，迭代额外空间  $O(1)$ 。

考察范围：二叉搜索树 · 最近公共祖先 · 有序性 · 迭代

易错点：比较时应覆盖当前节点等于 p 或 q 的情况，此时当前节点可直接返回。

## 701. 二叉搜索树中的插入操作

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 82/158  | 本章顺序 | 38/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:6634 |

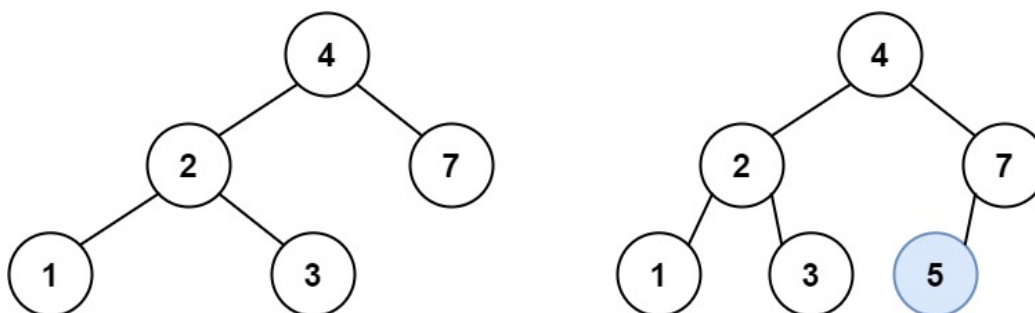
LeetCode：https://leetcode.cn/problems/insert-into-a-binary-search-tree/

### 题目内容

给定二叉搜索树 (BST) 的根节点 root 和要插入树中的值 value，将值插入二叉搜索树。返回插入后二叉搜索树的根节点。输入数据保证，新值和原始二叉搜索树中的任意节点值都不同。

注意，可能存在多种有效的插入方式，只要树在插入后仍保持为二叉搜索树即可。你可以返回任意有效的结果。

示例 1：



输入：root = [4,2,7,1,3], val = 5

输出：[4,2,7,1,3,5]

解释：另一个满足题目要求可以通过的树是：

示例 2：

输入：root = [40,20,60,10,30,50,70], val = 25

输出：[40,20,60,10,30,50,70,null,null,25]

示例 3：

输入：root = [4,2,7,1,3,null,null,null,null,null], val = 5

输出：[4,2,7,1,3,5]

提示：

1 树中的节点数将在  $[0, 10^4]$  的范围内。

1  $-10^8 \leq \text{Node.val} \leq 10^8$

1 所有值 Node.val 是独一无二的。

1  $-10^8 \leq \text{val} \leq 10^8$

## 1 保证 val 在原始BST中不存在。

我的题目思路与考察范围

核心思路：按值大小下降到首个空位置，将新节点接到对应父节点。

关键不变量 / 状态：搜索路径上的每次左右选择都保持 BST 的全局有序关系。

复杂度：时间  $O(h)$ ，迭代额外空间  $O(1)$ 。

考察范围：二叉搜索树 · 插入 · 递归 · 指针

易错点：递归写法必须把返回子树重新赋给 root.left 或 root.right。

## 450. 删除二叉搜索树中的节点

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 83/158  | 本章顺序 | 39/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:6833 |

LeetCode : <https://leetcode.cn/problems/delete-node-in-a-bst/>

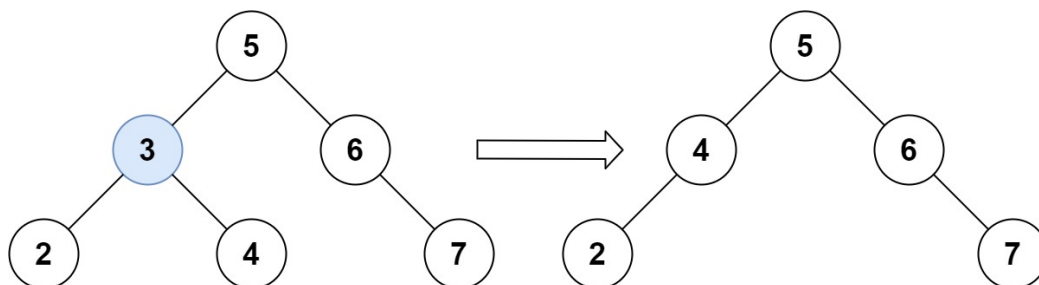
## 题目内容

给定一个二叉搜索树的根节点 root 和一个值 key，删除二叉搜索树中的 key 对应的节点，并保证二叉搜索树的性质不变。返回二叉搜索树（有可能被更新）的根节点的引用。

一般来说，删除节点可分为两个步骤：

- 1 首先找到需要删除的节点；
- 2 如果找到了，删除它。

示例 1:



输入：root = [5,3,6,2,4,null,7], key = 3

输出：[5,4,6,2,null,null,7]

解释：给定需要删除的节点值是 3，所以我们首先找到 3 这个节点，然后删除它。

一个正确的答案是 [5,4,6,2,null,null,7]，如下图所示。

另一个正确答案是 [5,2,6,null,4,null,7]。

示例 2:

输入：root = [5,3,6,2,4,null,7], key = 0

输出：[5,3,6,2,4,null,7]

解释：二叉树不包含值为 0 的节点

示例 3:

输入：root = [], key = 0

输出：[]

提示:

- 1 节点数的范围  $[0, 10^4]$ 。



1  $-10^5 \leq \text{Node.val} \leq 10^5$

1 节点值唯一

1 root 是合法的二叉搜索树

1  $-10^5 \leq \text{key} \leq 10^5$

进阶：要求算法时间复杂度为  $O(h)$ ， $h$  为树的高度。

我的题目思路与考察范围

核心思路：先定位节点；双子树时用中序后继替换，随后在右子树删除该后继。

关键不变量 / 状态：每次递归返回的子树都已删除目标且仍满足 BST 有序性。

复杂度：时间  $O(h)$ ，递归栈  $O(h)$ 。

考察范围：二叉搜索树 · 删除 · 中序后继 · 结构调整

易错点：双子树情形不能直接丢弃任一侧，且替换后还要删除重复后继节点。

## 669. 修剪二叉搜索树

|      |         |      |                                |
|------|---------|------|--------------------------------|
| 唯一题序 | 84/158  | 本章顺序 | 40/42                          |
| 难度   | 中等      | 题目类型 | 主讲题                            |
| 出现章节 | 第7章 二叉树 | 源位置  | 7.《代码随想录》二叉树 ( V3.0 ) .md:7098 |

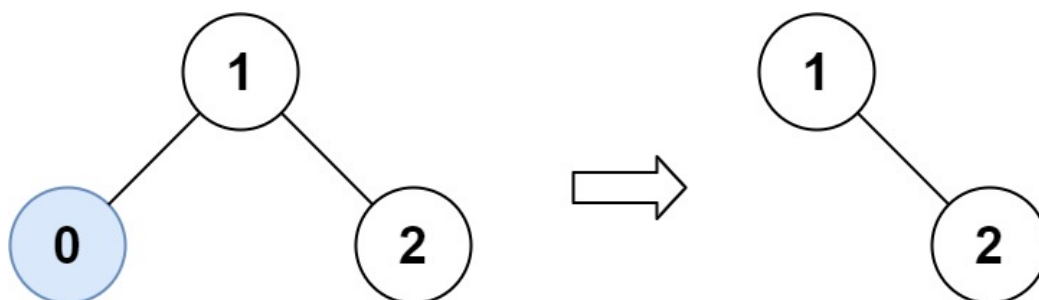
LeetCode：https://leetcode.cn/problems/trim-a-binary-search-tree/

### 题目内容

给你二叉搜索树的根节点 root，同时给定最小边界 low 和最大边界 high。通过修剪二叉搜索树，使得所有节点的值在  $[\text{low}, \text{high}]$  中。修剪树 不应该 改变保留在树中的元素的相对结构 (即，如果没有被移除，原有的父代子代关系都应当保留)。可以证明，存在 唯一 的答案。

所以结果应当返回修剪好的二叉搜索树的新的根节点。注意，根节点可能会根据给定的边界发生改变。

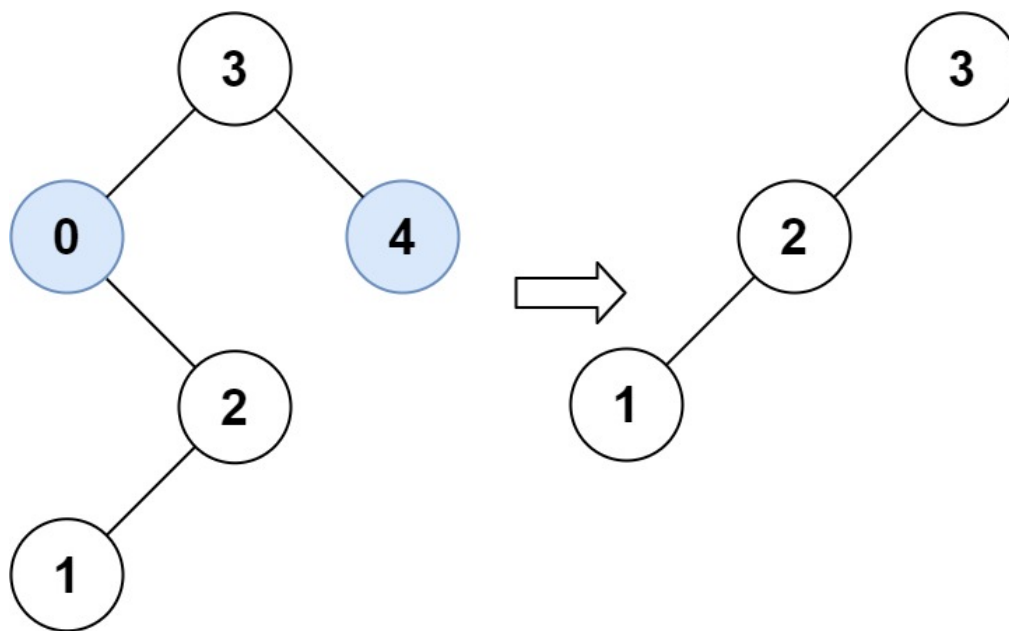
示例 1：



输入：root = [1,0,2], low = 1, high = 2

输出：[1,null,2]

示例 2：



输入：root = [3,0,4,null,2,null,null,1], low = 1, high = 3  
 输出：[3,2,null,1]

提示：

- 1 树中节点数在范围  $[1, 10^4]$  内
- 1  $0 \leq \text{Node.val} \leq 10^4$
- 1 树中每个节点的值都是唯一的
- 1 题目数据保证输入是一棵有效的二叉搜索树
- 1  $0 \leq \text{low} \leq \text{high} \leq 10^4$

我的题目思路与考察范围

核心思路：root 小于 low 时整棵左子树可丢弃并修剪右侧；大于 high 时对称处理。

关键不变量 / 状态：返回子树中的所有值均在区间内，并保持原有祖先关系与 BST 性质。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉搜索树 · 剪枝 · 区间 · 递归

易错点：越界根被删除后，其某个子树仍可能含合法节点，不能直接返回 null。

## 108. 将有序数组转换为二叉搜索树

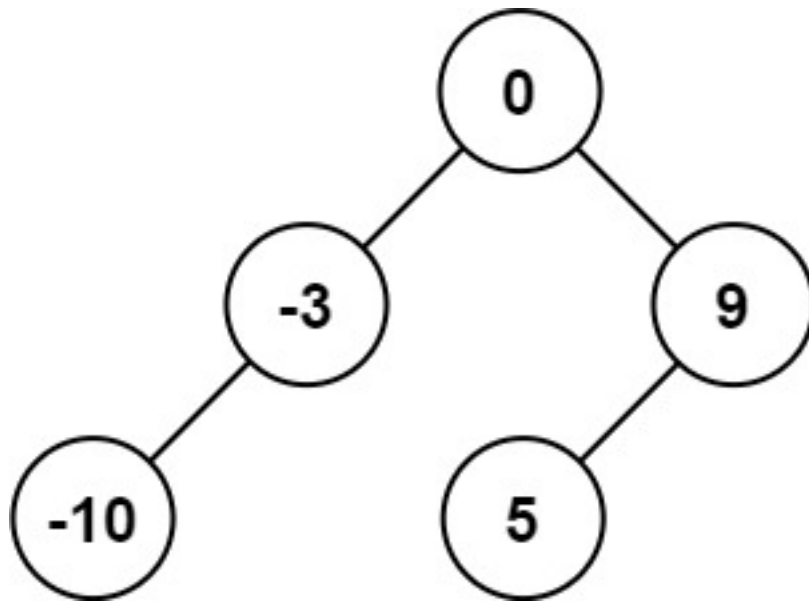
|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 85/158  | 本章顺序 | 41/42                           |
| 难度   | 简单      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:7336 |

LeetCode：https://leetcode.cn/problems/convert-sorted-array-to-binary-search-tree/

### 题目内容

给你一个整数数组 nums，其中元素已经按升序排列，请你将其转换为一棵平衡二叉搜索树。

示例 1：

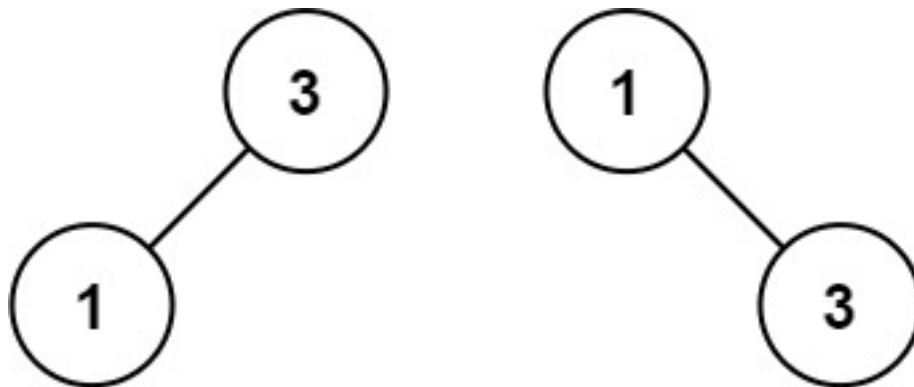


输入：nums = [-10,-3,0,5,9]

输出：[0,-3,9,-10,null,5]

解释：[0,-10,5,null,-3,null,9] 也将被视为正确答案：

示例 2：



输入：nums = [1,3]

输出：[3,1]

解释：[1,null,3] 和 [3,1] 都是高度平衡二叉搜索树。

提示：

1  $1 \leq \text{nums.length} \leq 10^4$

1  $-10^4 \leq \text{nums}[i] \leq 10^4$

1 nums 按严格递增顺序排列

我的题目思路与考察范围

核心思路：每次选择有序区间中点为根，左右半区递归生成平衡子树。

关键不变量 / 状态：递归区间有序且与生成子树一一对应，中点使两侧规模尽量接近。

复杂度：时间  $O(n)$ ，递归栈  $O(\log n)$ 。

考察范围：二叉搜索树 · 分治 · 平衡树 · 有序数组

易错点：统一使用半开或闭区间；混用边界会漏元素或无限递归。

# 538. 把二叉搜索树转换为累加树

|      |         |      |                                 |
|------|---------|------|---------------------------------|
| 唯一题序 | 86/158  | 本章顺序 | 42/42                           |
| 难度   | 中等      | 题目类型 | 主讲题                             |
| 出现章节 | 第7章 二叉树 | 源位置  | 7. 《代码随想录》二叉树 ( V3.0 ) .md:7543 |

LeetCode : <https://leetcode.cn/problems/convert-bst-to-greater-tree/>

## 题目内容

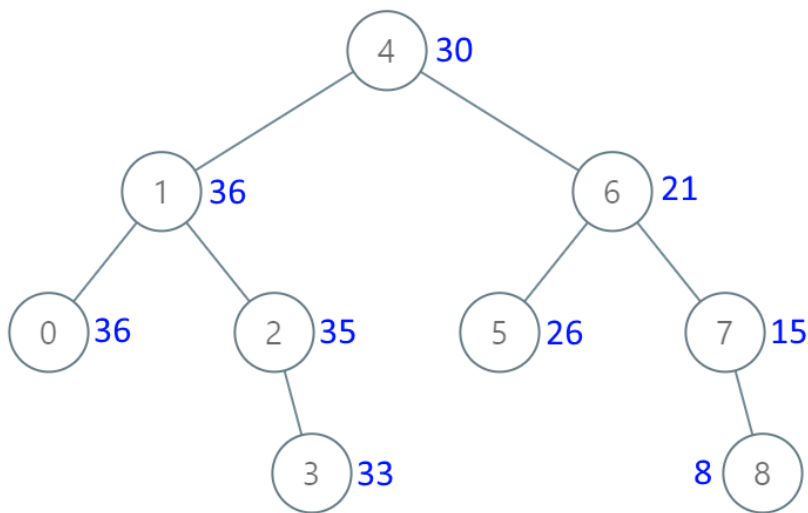
给出二叉搜索树的根节点 root，该树的节点值各不相同，请你将其转换为累加树 ( Greater Sum Tree )，将其转换为一个更大的树，使得原始二叉搜索树中的每个节点值都变为原本值加上原本二叉搜索树中所有比该节点值大的节点值的总和。

提醒一下，二叉搜索树满足下列约束条件：

- 1 节点的左子树仅包含键 小于 节点键的节点。
- 1 节点的右子树仅包含键 大于 节点键的节点。
- 1 左右子树也必须是二叉搜索树。

注意：本题和 1038: <https://leetcode.cn/problems/binary-search-tree-to-greater-sum-tree/> 相同

示例 1：



输入：[4,1,6,0,2,5,7,null,null,null,3,null,null,null,8]  
输出：[30,36,21,36,35,26,15,null,null,null,33,null,null,null,8]

示例 2：

输入：root = [0,null,1]  
输出：[1,null,1]

示例 3：

输入：root = [1,0,2]  
输出：[3,3,2]

示例 4：

输入：root = [3,2,4,1]  
输出：[7,9,4,10]

提示：

- 1 树中的节点数介于 0 和  $10^4$  之间。
- 1 每个节点的值介于  $-10^4$  和  $10^4$  之间。
- 1 树中的所有值 互不相同。
- 1 给定的树为二叉搜索树。

我的题目思路与考察范围

核心思路：按右-根-左逆中序遍历，用累计和更新当前节点。

关键不变量 / 状态：访问节点时 sum 已包含所有严格大于它的原始节点值。

复杂度：时间  $O(n)$ ，递归栈  $O(h)$ 。

考察范围：二叉搜索树 · 逆中序遍历 · 前缀和 · 原地修改

易错点：必须先累加原值再覆盖节点，并避免跨测试复用未清零的 sum。

## 第 8 章 回溯算法

本章 15 道去重题目，按源文件首次出现顺序排列。

### 77. 组合

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 87/158   | 本章顺序 | 1/15                           |
| 难度   | 中等       | 题目类型 | 主讲题、正文引用                       |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:180 |

LeetCode : <https://leetcode.cn/problems/combinations/>

#### 题目内容

给定两个整数  $n$  和  $k$ ，返回范围  $[1, n]$  中所有可能的  $k$  个数的组合。

你可以按 任何顺序 返回答案。

示例 1：

```
输入：n = 4, k = 2
输出：
[
  [2,4],
  [3,4],
  [2,3],
  [1,2],
  [1,3],
  [1,4],
]
```

示例 2：

```
输入：n = 1, k = 1
输出：[[1]]
```

提示：

1  $1 \leq n \leq 20$

1  $1 \leq k \leq n$

我的题目思路与考察范围

核心思路：用 `startIndex` 控制下一次只能选更大的数，路径长度到  $k$  时收集。

关键不变量 / 状态：path 严格递增且只包含已决定的前缀组合，不会产生排列重复。

复杂度：输出敏感，约  $O(C(n,k) \cdot k)$ ，递归栈  $O(k)$ 。

考察范围：回溯 · 组合 · 剪枝 · 树形搜索

易错点：循环上界应剪到  $n - (k - \text{path.size()}) + 1$ ，否则会搜索不可能凑满的分支。

### 216. 组合总和 III

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 88/158   | 本章顺序 | 2/15                           |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:628 |

LeetCode : <https://leetcode.cn/problems/combination-sum-iii/>

## 题目内容

找出所有相加之和为  $n$  的  $k$  个数的组合，且满足下列条件：

- 只使用数字1到9
- 每个数字 最多使用一次

返回 所有可能的有效组合的列表。该列表不能包含相同的组合两次，组合可以以任何顺序返回。

示例 1:

输入:  $k = 3, n = 7$   
 输出:  $[[1,2,4]]$   
 解释:  
 $1 + 2 + 4 = 7$   
 没有其他符合的组合了。

示例 2:

输入:  $k = 3, n = 9$   
 输出:  $[[1,2,6], [1,3,5], [2,3,4]]$   
 解释:  
 $1 + 2 + 6 = 9$   
 $1 + 3 + 5 = 9$   
 $2 + 3 + 4 = 9$   
 没有其他符合的组合了。

示例 3:

输入:  $k = 4, n = 1$   
 输出:  $[]$   
 解释: 不存在有效的组合。  
 在 $[1,9]$ 范围内使用4个不同的数字，我们可以得到的最小和是 $1+2+3+4 = 10$ ，因为 $10 > 1$ ，没有有效的组合。

提示:

- $2 \leq k \leq 9$
- $1 \leq n \leq 60$

我的题目思路与考察范围

核心思路：从 1..9 中递增选择  $k$  个不同数字，同时维护剩余目标和。

关键不变量 / 状态：path 无重复且递增，元素个数与累计和始终对应当前搜索路径。

复杂度：最多枚举  $C(9,k)$  个组合，输出复制  $O(k)$ 。

考察范围：回溯 · 固定长度组合 · 目标和 · 剪枝

易错点：既要按剩余个数剪枝，也要在和超过  $n$  时停止；数字不可重复使用。

## 17. 电话号码的字母组合

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 89/158   | 本章顺序 | 3/15                           |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:861 |

LeetCode : <https://leetcode.cn/problems/letter-combinations-of-a-phone-number/>

## 题目内容

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按 任意顺序 返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。



示例 1：

输入：digits = "23"  
输出：["ad","ae","af","bd","be","bf","cd","ce","cf"]

示例 2：

输入：digits = "2"  
输出：["a","b","c"]

提示：

1 1 <= digits.length <= 4

1 digits[i] 是范围 ['2', '9'] 的一个数字。

我的题目思路与考察范围

核心思路：每层消费一个数字，遍历其映射字母并递归到下一个位置。

关键不变量 / 状态：递归深度 index 等于 path 已为前 index 个数字各选一个字母。

复杂度：输出规模约  $O(4^n \cdot n)$ ，递归栈  $O(n)$ 。

考察范围：回溯 · 笛卡尔积 · 字符串 · 映射表

易错点：空输入应返回空列表；数字 7 和 9 各映射四个字母。

## 39. 组合总和

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 90/158   | 本章顺序 | 4/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:1185 |

LeetCode：https://leetcode.cn/problems/combination-sum/

### 题目内容

给你一个 无重复元素 的整数数组 candidates 和一个目标整数 target，找出 candidates 中可以使数字和为目标数 target 的所有不同组合，并以列表形式返回。你可以按任意顺序返回这些组合。



candidates 中的 同一个 数字可以 无限制重复被选取 。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 target 的不同组合数少于 150 个。

示例 1：

输入：candidates = [2,3,6,7], target = 7  
 输出：[[2,2,3],[7]]  
 解释：  
 2 和 3 可以形成一组候选， $2 + 2 + 3 = 7$ 。注意 2 可以使用多次。  
 7 也是一个候选， $7 = 7$ 。  
 仅有这两种组合。

示例 2：

输入: candidates = [2,3,5], target = 8  
 输出: [[2,2,2,2],[2,3,3],[3,5]]

示例 3：

输入: candidates = [2], target = 1  
 输出: []

提示：

- 1 1 <= candidates.length <= 30
- 1 2 <= candidates[i] <= 40
- 1 candidates 的所有元素 互不相同
- 1 1 <= target <= 40

我的题目思路与考察范围

核心思路：候选排序后回溯；递归仍传 i 允许当前数重复使用，剩余和为 0 时收集。

关键不变量 / 状态：path 非递减，remaining 等于 target 减去 path 元素和。

复杂度：最坏指数级，递归深度可达  $\text{target}/\min(\text{candidates})$ 。

考察范围：回溯 · 完全背包 · 组合 · 排序剪枝

易错点：重复使用应传 i 而非 i+1；候选必须为正数才能按剩余和安全剪枝。

## 40. 组合总和 II

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 91/158   | 本章顺序 | 5/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:1403 |

LeetCode：https://leetcode.cn/problems/combination-sum-ii/

### 题目内容

给定一个候选人编号的集合 candidates 和一个目标数 target，找出 candidates 中所有可以使数字和为 target 的组合。

candidates 中的每个数字在每个组合中只能使用 一次。

注意：解集不能包含重复的组合。

示例 1:

输入: candidates = [10,1,2,7,6,1,5], target = 8,  
 输出:  
 [  
 [1,1,6],

```
[1,2,5],
[1,7],
[2,6]
]
```

示例 2:

```
输入: candidates = [2,5,2,1,2], target = 5,
输出:
[
  [1,2,2],
  [5]
]
```

提示:

1 1 <= candidates.length <= 100

1 1 <= candidates[i] <= 50

1 1 <= target <= 30

我的题目思路与考察范围

核心思路：先排序；每个下标只用一次，并跳过同一树层中相同值的后续候选。

关键不变量 / 状态：同一层每个数值只作为一次分支起点，不同层仍可使用不同下标的同值。

复杂度：最坏  $O(2^n \cdot n)$ ，递归栈  $O(n)$ 。

考察范围：回溯 · 去重 · 排序 · 目标和

易错点：去重条件是  $i > \text{startIndex}$ ；若跨层也跳过相同值会漏掉合法组合。

## 131. 分割回文串

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 92/158   | 本章顺序 | 6/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:1664 |

LeetCode : <https://leetcode.cn/problems/palindrome-partitioning/>

### 题目内容

给你一个字符串  $s$ ，请你将  $s$  分割成一些子串，使每个子串都是回文串。返回  $s$  所有可能的分割方案。

示例 1：

```
输入: s = "aab"
输出: [["a","a","b"],["aa","b"]]
```

示例 2：

```
输入: s = "a"
输出: [["a"]]
```

提示：

1 1 <= s.length <= 16

1 s 仅由小写英文字母组成

## 我的题目思路与考察范围

核心思路：从 start 枚举切割终点，只对回文前缀继续递归，到字符串末尾收集分割。

关键不变量 / 状态：path 中每一段都是原串连续且已验证的回文子串，并恰好覆盖前缀 [0,start)。

复杂度：预处理回文  $O(n^2)$ ，枚举最坏  $O(n \cdot 2^n)$ 。

考察范围：回溯 · 字符串切割 · 回文 · 动态规划预处理

易错点：切割下标与 substring 长度易错；不要把子序列误当连续子串。

## 93. 复原 IP 地址

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 93/158   | 本章顺序 | 7/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:1976 |

LeetCode : <https://leetcode.cn/problems/restore-ip-addresses/>

## 题目内容

有效 IP 地址 正好由四个整数（每个整数位于 0 到 255 之间组成，且不能含有前导 0），整数之间用 '.' 分隔。

- 1 例如："0.1.2.201" 和 "192.168.1.1" 是有效 IP 地址，但是 "0.011.255.245"、"192.168.1.312" 和 "192.168@1.1" 是无效 IP 地址。

给定一个只包含数字的字符串 s，用以表示一个 IP 地址，返回所有可能的有效 IP 地址，这些地址可以通过在 s 中插入 '.' 来形成。你 不能 重新排序或删除 s 中的任何数字。你可以按 任何 顺序返回答案。

示例 1：

输入：s = "25525511135"  
输出：["255.255.11.135","255.255.111.35"]

示例 2：

输入：s = "0000"  
输出：["0.0.0.0"]

示例 3：

输入：s = "101023"  
输出：["1.0.10.23","1.0.102.3","10.1.0.23","10.10.2.3","101.0.2.3"]

提示：

- 1 1 <= s.length <= 20

- 1 s 仅由数字组成

## 我的题目思路与考察范围

核心思路：依次切出 4 段，每段长 1..3，验证数值范围和前导零后递归。

关键不变量 / 状态：已选 segments 都是合法 IP 段，并连续覆盖原串前缀。

复杂度：输入最长 12，搜索规模有常数上界；辅助空间  $O(1)$ 。

考察范围：回溯 · 字符串切割 · 合法性校验 · 剪枝

易错点：除单个 0 外不能有前导零；最终必须恰好四段并消耗全部字符。

## 78. 子集

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 94/158   | 本章顺序 | 8/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:2231 |

LeetCode : <https://leetcode.cn/problems/subsets/>

### 题目内容

给你一个整数数组 `nums`，数组中的元素互不相同。返回该数组所有可能的子集（幂集）。

解集不能包含重复的子集。你可以按任意顺序返回解集。

示例 1：

输入：nums = [1,2,3]  
输出：[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]

示例 2：

输入：nums = [0]  
输出：[[], [0]]

提示：

- 1 1 <= nums.length <= 10
- 1 -10 <= nums[i] <= 10
- 1 nums 中的所有元素互不相同

我的题目思路与考察范围

核心思路：每到一个搜索节点就记录当前 path，再从 startIndex 继续选择后续元素。

关键不变量 / 状态：path 表示一个唯一递增下标集合，搜索树每个节点对应一个子集。

复杂度：时间  $O(n \cdot 2^n)$ ，输出与递归空间  $O(n \cdot 2^n)$ 。

考察范围：回溯 · 子集 · 幂集 · 树形搜索

易错点：子集要收集所有中间节点，不是只在叶子或固定长度时收集。

## 90. 子集 II

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 95/158   | 本章顺序 | 9/15                            |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:2554 |

LeetCode : <https://leetcode.cn/problems/subsets-ii/>

### 题目内容

给你一个整数数组 `nums`，其中可能包含重复元素，请你返回该数组所有可能的子集（幂集）。

解集不能包含重复的子集。返回的解集中，子集可以按任意顺序排列。

示例 1：

输入：nums = [1,2,2]  
输出：[[], [1], [1,2], [1,2,2], [2], [2,2]]

示例 2：

输入：nums = [0]  
输出：[[],[0]]

提示：

1 1 <= nums.length <= 10

1 -10 <= nums[i] <= 10

我的题目思路与考察范围

核心思路：排序后枚举子集，并跳过同一层相邻的重复值。

关键不变量 / 状态：每个深度的循环中，同一数值只启动一个分支，path 仍可含多个不同下标的同值。

复杂度：最坏  $O(n \cdot 2^n)$ ，辅助递归栈  $O(n)$ 。

考察范围：回溯 · 子集 · 排序去重 · 重复元素

易错点：必须先排序；使用全局 set 虽可去重但掩盖树层去重逻辑且开销更大。

## 491. 非递减子序列

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 96/158   | 本章顺序 | 10/15                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:2709 |

LeetCode：https://leetcode.cn/problems/non-decreasing-subsequences/

### 题目内容

给你一个整数数组 nums，找出并返回所有该数组中不同的递增子序列，递增子序列中至少有两个元素。你可以按任意顺序返回答案。

数组中可能含有重复元素，如出现两个整数相等，也可以视作递增序列的一种特殊情况。

示例 1：

输入：nums = [4,6,7,7]  
输出：[[4,6],[4,6,7],[4,6,7,7],[4,7],[4,7,7],[6,7],[6,7,7],[7,7]]

示例 2：

输入：nums = [4,4,3,2,1]  
输出：[[4,4]]

提示：

1 1 <= nums.length <= 15

1 -100 <= nums[i] <= 100

我的题目思路与考察范围

核心思路：保持原数组顺序回溯，只追加不小于 path 末值的元素，并用层级 used 去重。

关键不变量 / 状态：path 始终非递减；同一递归层相同数值至多被选一次。

复杂度：最坏  $O(n \cdot 2^n)$ ，辅助空间  $O(n)$ 。

考察范围：回溯 · 子序列 · 层级去重 · 非递减

易错点：不能先排序，否则会改变子序列的原始相对顺序；结果长度至少为 2。

## 46. 全排列

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 97/158   | 本章顺序 | 11/15                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:2904 |

LeetCode : <https://leetcode.cn/problems/permutations/>

### 题目内容

给定一个不含重复数字的数组 `nums`，返回其 所有可能的全排列。你可以 按任意顺序 返回答案。

示例 1：

输入：nums = [1,2,3]  
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

示例 2：

输入：nums = [0,1]  
输出：[[0,1],[1,0]]

示例 3：

输入：nums = [1]  
输出：[[1]]

提示：

- 1 1 <= nums.length <= 6
- 1 -10 <= nums[i] <= 10
- 1 nums 中的所有整数 互不相同

我的题目思路与考察范围

核心思路：每层尝试所有尚未使用的下标，用 `used` 标记当前路径已选元素。

关键不变量 / 状态：path 中每个输入下标恰好使用一次，深度等于已确定的排列前缀长度。

复杂度：时间  $O(n \cdot n!)$ ，辅助空间  $O(n)$ 。

考察范围：回溯 · 排列 · `used`数组 · 全搜索

易错点：排列每层都从 0 枚举，不能使用组合题的 `startIndex`。

## 47. 全排列 II

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 98/158   | 本章顺序 | 12/15                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:3038 |

LeetCode : <https://leetcode.cn/problems/permutations-ii/>

### 题目内容

给定一个可包含重复数字的序列 `nums`，按任意顺序 返回所有不重复的全排列。

示例 1：

输入：nums = [1,1,2]  
输出：  
[[1,1,2],

```
[1,2,1],
[2,1,1]]
```

示例 2：

```
输入：nums = [1,2,3]
输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

提示：

```
1 1 <= nums.length <= 8
```

```
1 -10 <= nums[i] <= 10
```

我的题目思路与考察范围

核心思路：排序并使用 used；同层若前一个相同值尚未使用，则跳过当前值。

关键不变量 / 状态：相同值在同层按固定下标顺序被选择，从而每个唯一排列只生成一次。

复杂度：最坏  $O(n \cdot n!)$ ，辅助空间  $O(n)$ 。

考察范围：回溯 · 排列 · 排序去重 · used 数组

易错点：经典条件是  $nums[i] == nums[i-1]$  且  $!used[i-1]$ ；写反会改变去重层级。

## 332. 重新安排行程

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 99/158   | 本章顺序 | 13/15                           |
| 难度   | 困难       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:3544 |

LeetCode：https://leetcode.cn/problems/reconstruct-itinerary/

### 题目内容

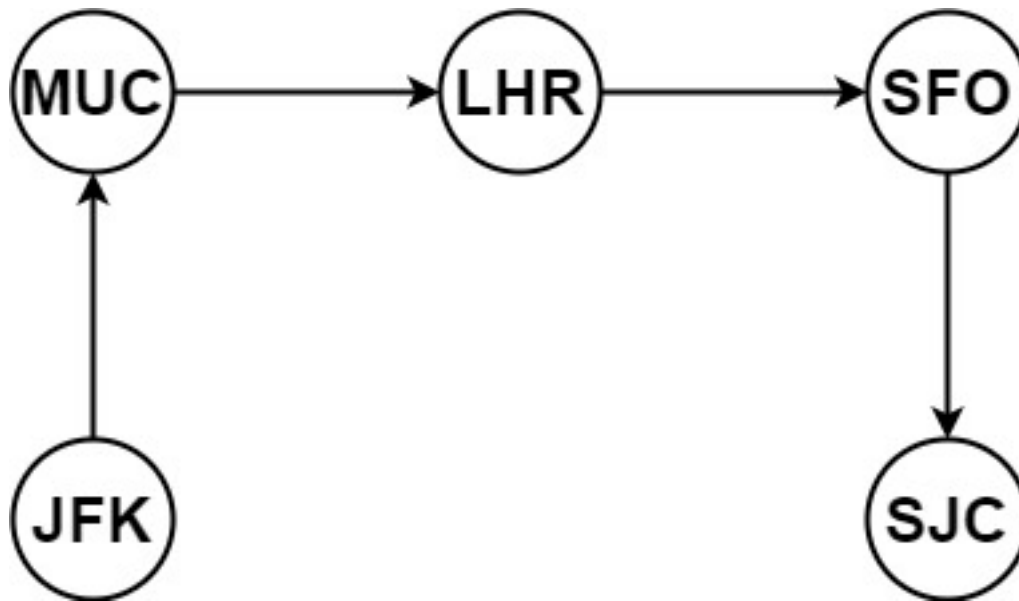
给你一份航线列表 tickets，其中  $tickets[i] = [from_i, to_i]$  表示飞机出发和降落的机场地点。请你对该行程进行重新规划排序。

所有这些机票都属于一个从 JFK（肯尼迪国际机场）出发的先生，所以该行程必须从 JFK 开始。如果存在多种有效的行程，请你按字典排序返回最小的行程组合。

1 例如，行程 ["JFK", "LGA"] 与 ["JFK", "LGB"] 相比就更小，排序更靠前。

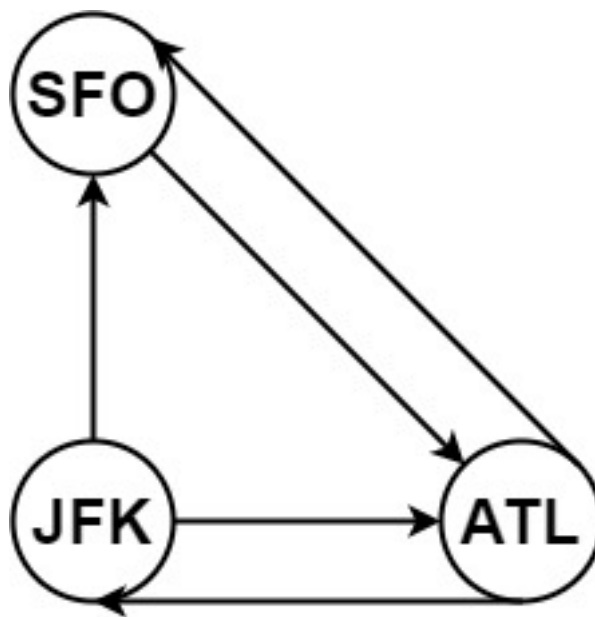
假定所有机票至少存在一种合理的行程。且所有的机票 必须都用一次 且 只能用一次。

示例 1：



输入 : tickets = [["MUC","LHR"],["JFK","MUC"],["SFO","SJC"],["LHR","SFO"]]  
 输出 : ["JFK","MUC","LHR","SFO","SJC"]

示例 2 :



输入 : tickets = [["JFK","SFO"],["JFK","ATL"],["SFO","ATL"],["ATL","JFK"],["ATL","SFO"]]  
 输出 : ["JFK","ATL","JFK","SFO","ATL","SFO"]  
 解释 : 另一种有效的行程是 ["JFK","SFO","ATL","JFK","ATL","SFO"] , 但是它字典排序更大更靠后。

提示 :

```

1  1 <= tickets.length <= 300
1  tickets[i].length == 2
1  from_i.length == 3
1  to_i.length == 3
1  from_i 和 to_i 由大写英文字母组成
1  from_i != to_i
  
```



## 我的题目思路与考察范围

核心思路：把机票建成按字典序取边的有向多重图，Hierholzer DFS 用尽出边后逆序加入机场。

关键不变量 / 状态：每张边被消费恰好一次；节点入答案时，从它出发的剩余行程已全部完成。

复杂度：排序或优先队列实现为  $O(E \log E)$ ，空间  $O(E)$ 。

考察范围：图 · 欧拉路径 · 字典序 · 多重边

易错点：重复机票必须按多重边计数；后序得到的路线需要反转。

## 51. N 皇后

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 100/158  | 本章顺序 | 14/15                           |
| 难度   | 困难       | 题目类型 | 主讲题                             |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8.《代码随想录》回溯算法 ( V3.0 ) .md:3812 |

LeetCode : <https://leetcode.cn/problems/n-queens/>

## 题目内容

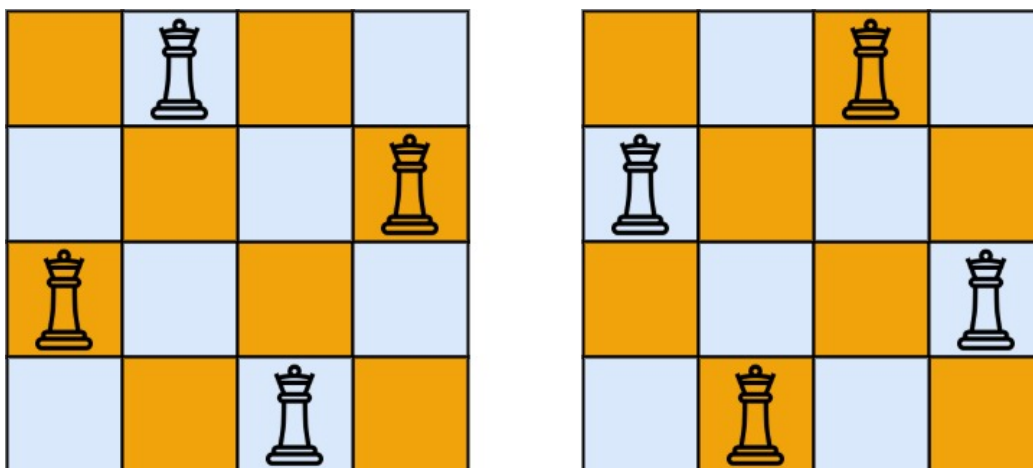
按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

$n$  皇后问题 研究的是如何将  $n$  个皇后放置在  $n \times n$  的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数  $n$ ，返回所有不同的  $n$  皇后问题 的解决方案。

每一种解法包含一个不同的  $n$  皇后问题 的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1：



输入： $n = 4$

输出：`[[".Q..","...Q","Q...", "..Q."],["..Q.", "Q...", "...Q", ".Q.."]]`

解释：如上图所示，4 皇后问题存在两个不同的解法。

示例 2：

输入： $n = 1$

输出：`[["Q"]]`

提示：

1  $1 \leq n \leq 9$

## 我的题目思路与考察范围

核心思路：按行放置皇后，用列、主对角线和副对角线占用状态剪枝。

关键不变量 / 状态：进入第 row 层时，前 row 行各有一个皇后且彼此不攻击。

复杂度：最坏约  $O(n!)$ ，状态与递归栈  $O(n)$ 。

考察范围：回溯 · 棋盘 · 约束满足 · 对角线

易错点：两类对角线可用 row-col 与 row+col 标识；回溯必须同步撤销三种占用。

## 37. 解数独

|      |          |      |                                  |
|------|----------|------|----------------------------------|
| 唯一题序 | 101/158  | 本章顺序 | 15/15                            |
| 难度   | 困难       | 题目类型 | 主讲题                              |
| 出现章节 | 第8章 回溯算法 | 源位置  | 8. 《代码随想录》回溯算法 ( V3.0 ) .md:4028 |

LeetCode : <https://leetcode.cn/problems/sudoku-solver/>

## 题目内容

编写一个程序，通过填充空格来解决数独问题。

数独的解法需遵循如下规则：

- 1 数字 1-9 在每一行只能出现一次。
- 2 数字 1-9 在每一列只能出现一次。
- 3 数字 1-9 在每一个以粗实线分隔的 3x3 宫内只能出现一次。（请参考示例图）

数独部分空格内已填入了数字，空白格用 '.' 表示。

示例 1：

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 |   |   | 7 |   |   |   |   |
| 6 |   |   | 1 | 9 | 5 |   |   |   |
|   | 9 | 8 |   |   |   |   | 6 |   |
| 8 |   |   |   | 6 |   |   |   | 3 |
| 4 |   |   | 8 |   | 3 |   |   | 1 |
| 7 |   |   |   | 2 |   |   |   | 6 |
|   | 6 |   |   |   |   | 2 | 8 |   |
|   |   |   | 4 | 1 | 9 |   |   | 5 |
|   |   |   |   | 8 |   |   | 7 | 9 |

输入：board = [["5","3",".", ".", "7",".", ".", ".", "."],["6",".", ".", "1","9","5",".", ".", "."],[".", "9","8",".", ".", ".", ".", "6","."],["8",".", ".", "6",".", ".", "3"],["4",".", ".", "3",".", ".", "1"],["7",".", ".", "2",".", ".", "6"],[".", "6",".", ".", "2","8","."],[".", ".", "4","1","9",".", ".", "5"],[".", ".", ".", "8",".", ".", "7","9"]]

输出：[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]

解释：输入的数独如上图所示，唯一有效的解决方案如下所示：

提示：

- 1 board.length == 9
- 1 board[i].length == 9
- 1 board[i][j] 是一位数字或者 '.'
- 1 题目数据 保证 输入数独仅有一个解

我的题目思路与考察范围

核心思路：选择一个空格，尝试不冲突的 1..9；失败就撤销，填满所有空格即结束。

关键不变量 / 状态：递归过程中所有已填单元都同时满足行、列和 3×3 宫约束。

复杂度：最坏指数级，位掩码校验空间  $O(1)$ ，递归深度至多 81。

考察范围：回溯 · 约束满足 · 位掩码 · 棋盘

易错点：找到完整解后要向上传播 true 立即停止；试填失败必须恢复为点号。

## 第 9 章 贪心算法

本章 17 道去重题目，按源文件首次出现顺序排列。

### 455. 分发饼干

|      |          |      |                               |
|------|----------|------|-------------------------------|
| 唯一题序 | 102/158  | 本章顺序 | 1/17                          |
| 难度   | 简单       | 题目类型 | 主讲题                           |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:99 |

LeetCode : <https://leetcode.cn/problems/assign-cookies/>

#### 题目内容

假设你是一位很棒的家长，想要给你的孩子们一些小饼干。但是，每个孩子最多只能给一块饼干。

对每个孩子  $i$ ，都有一个胃口值  $g[i]$ ，这是能让孩子们满足胃口的饼干的最小尺寸；并且每块饼干  $j$ ，都有一个尺寸  $s[j]$ 。如果  $s[j] \geq g[i]$ ，我们可以将这个饼干  $j$  分配给孩子  $i$ ，这个孩子会得到满足。你的目标是满足尽可能多的孩子，并输出这个最大数值。

示例 1:

输入:  $g = [1,2,3]$ ,  $s = [1,1]$   
 输出: 1  
 解释:  
 你有三个孩子和两块小饼干，3 个孩子的胃口值分别是：1,2,3。  
 虽然你有两块小饼干，由于他们的尺寸都是 1，你只能让胃口值是 1 的孩子满足。  
 所以你应该输出 1。

示例 2:

输入:  $g = [1,2]$ ,  $s = [1,2,3]$   
 输出: 2  
 解释:  
 你有两个孩子和三块小饼干，2 个孩子的胃口值分别是 1,2。  
 你拥有的饼干数量和尺寸都足以让所有孩子满足。  
 所以你应该输出 2。

提示：

```
1  1 <= g.length <= 3 * 10 ^4
1  0 <= s.length <= 3 * 10 ^4
1  1 <= g[i], s[j] <= 2 ^31 - 1
```

注意：本题与 2410. 运动员和训练师的最大匹配数 题相同。

#### 我的题目思路与考察范围

核心思路：将孩子胃口和饼干尺寸分别升序排序，用当前最小可满足的饼干喂当前胃口最小的孩子；若饼干太小就继续换更大的饼干。这样不会让大饼干被不必要地占用。

关键不变量 / 状态：已经匹配的孩子数量最大，未使用的饼干和未满足的孩子都不可能通过回头交换得到更多匹配。

复杂度：时间  $O(n \log n + m \log m)$ ，空间  $O(1)$ （不计排序栈）。

考察范围：贪心 · 排序 · 双指针 · 局部最优

易错点：不要为了满足胃口更大的孩子而跳过当前可满足的小孩子，否则可能浪费大尺寸饼干。

## 376. 摆动序列

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 103/158  | 本章顺序 | 2/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:236 |

LeetCode : <https://leetcode.cn/problems/wiggle-subsequence/>

### 题目内容

如果连续数字之间的差严格地在正数和负数之间交替，则数字序列称为 摆动序列 。第一个差（如果存在的话）可能是正数或负数。仅有一个元素或者含两个不等元素的序列也视作摆动序列。

1 例如，`[1, 7, 4, 9, 2, 5]` 是一个 摆动序列 ，因为差值 `(6, -3, 5, -7, 3)` 是正负交替出现的。

1 相反，`[1, 4, 7, 2, 5]` 和 `[1, 7, 4, 5, 5]` 不是摆动序列，第一个序列是因为它的前两个差值都是正数，第二个序列是因为它的最后一个差值为零。

子序列 可以通过从原始序列中删除一些（也可以不删除）元素来获得，剩下的元素保持其原始顺序。

给你一个整数数组 `nums` ，返回 `nums` 中作为 摆动序列 的 最长子序列的长度 。

示例 1：

输入：`nums = [1,7,4,9,2,5]`  
 输出：6  
 解释：整个序列均为摆动序列，各元素之间的差值为 `(6, -3, 5, -7, 3)`。

示例 2：

输入：`nums = [1,17,5,10,13,15,10,5,16,8]`  
 输出：7  
 解释：这个序列包含几个长度为 7 摆动序列。  
 其中一个是 `[1, 17, 10, 13, 10, 16, 8]`，各元素之间的差值为 `(16, -7, 3, -3, 6, -8)`。

示例 3：

输入：`nums = [1,2,3,4,5,6,7,8,9]`  
 输出：2

提示：

1 `1 <= nums.length <= 1000`

1 `0 <= nums[i] <= 1000`

进阶：你能否用  $O(n)$  时间复杂度完成此题？

我的题目思路与考察范围

核心思路：只保留能形成峰或谷的转折点：当前差值与前一个有效差值异号时，摆动长度加一。连续相等或同向坡度只保留最远端点；也可用 up/down 动态规划表达同一状态。

关键不变量 / 状态：扫描到当前位置时，保留的最后元素是当前单调坡的最优端点，且计数对应此前可获得的最长摆动子序列。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：贪心 · 动态规划 · 序列 · 峰谷

易错点：平坡不能重复计数；更新前一个有效差值时应只在真正发生方向变化后更新。

## 53. 最大子数组和

|      |                    |      |                                |
|------|--------------------|------|--------------------------------|
| 唯一题序 | 104/158            | 本章顺序 | 3/17                           |
| 难度   | 中等                 | 题目类型 | 主讲题                            |
| 出现章节 | 第9章 贪心算法、第10章 动态规划 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:502 |

LeetCode : <https://leetcode.cn/problems/maximum-subarray/>

### 题目内容

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

子数组 是数组中的一个连续部分。

示例 1：

输入：nums = [-2,1,-3,4,-1,2,1,-5,4]  
输出：6  
解释：连续子数组 [4,-1,2,1] 的和最大，为 6。

示例 2：

输入：nums = [1]  
输出：1

示例 3：

输入：nums = [5,4,-1,7,8]  
输出：23

提示：

1  $1 \leq \text{nums.length} \leq 10^5$

1  $-10^4 \leq \text{nums}[i] \leq 10^4$

进阶：如果你已经实现复杂度为  $O(n)$  的解法，尝试使用更为精妙的 分治法 求解。

我的题目思路与考察范围

核心思路：Kadane 算法令当前和表示必须以当前位置结尾的最大子数组和：若此前和为负就从当前元素重新开始，否则继续累加。它既可理解为负贡献立即丢弃的贪心，也可写成一维 DP。

关键不变量 / 状态：处理完 `nums[i]` 后，`current` 是所有以 `i` 结尾子数组的最大和，`answer` 是前缀 `[0,i]` 内的全局最大和。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：贪心 · 动态规划 · Kadane · 子数组

易错点：答案必须初始化为首元素或负无穷，不能初始化为 0，否则全负数组会出错。

## 122. 买卖股票的最佳时机 II

|      |                    |      |                                |
|------|--------------------|------|--------------------------------|
| 唯一题序 | 105/158            | 本章顺序 | 4/17                           |
| 难度   | 中等                 | 题目类型 | 主讲题                            |
| 出现章节 | 第9章 贪心算法、第10章 动态规划 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:760 |

LeetCode : <https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-ii/>

### 题目内容

给你一个整数数组 `prices`，其中 `prices[i]` 表示某支股票第 `i` 天的价格。

在每一天，你可以决定是否购买和/或出售股票。你在任何时候 最多 只能持有 一股 股票。然而，你可以在 同一天 多次买卖该股票，但要确保你持有的股票不超过一股。

返回 你能获得的 最大 利润。

示例 1：

输入：prices = [7,1,5,3,6,4]

输出：7

解释：在第 2 天（股票价格 = 1）的时候买入，在第 3 天（股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。  
随后，在第 4 天（股票价格 = 3）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，这笔交易所能获得利润 = 6 - 3 = 3。  
最大总利润为 4 + 3 = 7。

示例 2：

输入：prices = [1,2,3,4,5]

输出：4

解释：在第 1 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5 - 1 = 4。  
最大总利润为 4。

示例 3：

输入：prices = [7,6,4,3,1]

输出：0

解释：在这种情况下，交易无法获得正利润，所以不参与交易可以获得最大利润，最大利润为 0。

提示：

1 1 <= prices.length <= 3 \* 10 ^4

1 0 <= prices[i] <= 10 ^4

我的题目思路与考察范围

核心思路：无限次交易时，把每一段相邻上涨利润都收入囊中，利润为所有  $\max(\text{prices}[i]-\text{prices}[i-1], 0)$  之和；这等价于两状态股票 DP 的压缩形式。

关键不变量 / 状态：扫描到第  $i$  天时，已经获得前  $i$  天所有可分解上涨区间的最大总利润，且当天结束时不持有跨过下降边的股票。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：贪心 · 动态规划 · 股票 · 状态机

易错点：同一天可以卖出后再买入但不能同时持有多股；不要把本题误写成只能交易一次。

## 55. 跳跃游戏

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 106/158  | 本章顺序 | 5/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:895 |

LeetCode：https://leetcode.cn/problems/jump-game/

### 题目内容

给你一个非负整数数组 `nums`，你最初位于数组的 第一个下标。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果可以，返回 `true`；否则，返回 `false`。

示例 1：

输入：nums = [2,3,1,1,4]

输出：true

解释：可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

示例 2：

输入：nums = [3,2,1,0,4]

输出：false

解释：无论如何，总会到达下标为 3 的位置。但该下标的最大跳跃长度是 0，所以永远不可能到达最后一个下标。

提示：

1 1 <= nums.length <= 10 ^4

1 0 <= nums[i] <= 10 ^5

我的题目思路与考察范围

核心思路：从左到右维护当前能到达的最远下标 farthest；只要当前位置不超过 farthest，就用 i+nums[i] 扩展覆盖范围。一旦当前位置落在覆盖范围外就不可达。

关键不变量 / 状态：扫描到 i 前，区间 [0, farthest] 中的所有位置都可由起点到达。

复杂度：时间 O(n)，空间 O(1)。

考察范围：贪心 · 区间覆盖 · 可达性 · 数组

易错点：循环只能处理 i <= farthest 的位置，不能用一个本身不可达的位置继续扩大覆盖范围。

## 45. 跳跃游戏 II

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 107/158  | 本章顺序 | 6/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:981 |

LeetCode：https://leetcode.cn/problems/jump-game-ii/

### 题目内容

给定一个长度为 n 的 0 索引 整数数组 nums。初始位置在下标 0。

每个元素 nums[i] 表示从索引 i 向后跳转的最大长度。换句话说，如果你在索引 i 处，你可以跳转到任意 (i + j) 处：

1 0 <= j <= nums[i] 且

1 i + j < n

返回到达 n - 1 的最小跳跃次数。测试用例保证可以到达 n - 1。

示例 1:

输入: nums = [2,3,1,1,4]

输出: 2

解释: 跳到最后一个位置的最小跳跃数是 2。

从下标为 0 跳到下标为 1 的位置，跳 1 步，然后跳 3 步到达数组的最后一个位置。

示例 2:

输入: nums = [2,3,0,1,4]

输出: 2

提示:

1 1 <= nums.length <= 10 ^4

1 0 <= nums[i] <= 1000

1 题目保证可以到达 n - 1



## 我的题目思路与考察范围

核心思路：把一次跳跃能覆盖的范围看作 BFS 的一层，扫描当前层时维护下一层最远边界 `farthest`；到达 `currentEnd` 才增加步数并切换边界。

关键不变量 / 状态：在 `currentEnd` 之前的下标都能用 `jumps` 步到达，`farthest` 是再跳一步可达到的最远位置。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：贪心 · BFS 分层 · 区间覆盖 · 最少步数

易错点：到达终点后不要再增加步数；题目保证可达时才省略不可达分支。

## 1005. K 次取反后最大化的数组和

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 108/158  | 本章顺序 | 7/17                            |
| 难度   | 简单       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:1121 |

LeetCode : <https://leetcode.cn/problems/maximize-sum-of-array-after-k-negations/>

## 题目内容

给你一个整数数组 `nums` 和一个整数 `k`，按以下方法修改该数组：

- 1 选择某个下标 `i` 并将 `nums[i]` 替换为 `-nums[i]`。

重复这个过程恰好 `k` 次。可以多次选择同一个下标 `i`。

以这种方式修改数组后，返回数组 可能的最大和。

示例 1：

输入：`nums = [4,2,3]`, `k = 1`  
 输出：5  
 解释：选择下标 1，`nums` 变为 `[4,-2,3]`。

示例 2：

输入：`nums = [3,-1,0,2]`, `k = 3`  
 输出：6  
 解释：选择下标 (1, 2, 2)，`nums` 变为 `[3,1,0,2]`。

示例 3：

输入：`nums = [2,-3,-1,5,-4]`, `k = 2`  
 输出：13  
 解释：选择下标 (1, 4)，`nums` 变为 `[2,3,-1,5,4]`。

提示：

- 1  $1 \leq \text{nums.length} \leq 10^4$
- 1  $-100 \leq \text{nums}[i] \leq 100$
- 1  $1 \leq k \leq 10^4$

## 我的题目思路与考察范围

核心思路：按绝对值从大到小处理，优先翻转绝对值大的负数；负数用尽后若剩余次数为奇数，只需再翻转绝对值最小的元素。重复翻转同一元素只与剩余次数奇偶有关。

关键不变量 / 状态：处理过的较大绝对值元素已经取得对总和最有利的符号，未决策元素的绝对值不大于它们。

复杂度：时间  $O(n \log n)$ ，空间  $O(1)$ （不计排序栈）；值域有限时可用计数桶优化。

考察范围：贪心 · 排序 · 绝对值 · 奇偶性

易错点：翻完负数后必须按剩余次数的奇偶决定是否减去两倍最小绝对值。

## 134. 加油站

|      |          |      |                                  |
|------|----------|------|----------------------------------|
| 唯一题序 | 109/158  | 本章顺序 | 8/17                             |
| 难度   | 中等       | 题目类型 | 主讲题                              |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9. 《代码随想录》贪心算法 ( V3.0 ) .md:1312 |

LeetCode : <https://leetcode.cn/problems/gas-station/>

### 题目内容

在一条环路上有  $n$  个加油站，其中第  $i$  个加油站有汽油  $gas[i]$  升。

你有一辆油箱容量无限的汽车，从第  $i$  个加油站开往第  $i+1$  个加油站需要消耗汽油  $cost[i]$  升。你从其中的一个加油站出发，开始时油箱为空。

给定两个整数数组  $gas$  和  $cost$ ，如果你可以按顺序绕环路行驶一周，则返回出发时加油站的编号，否则返回  $-1$ 。如果存在解，则保证它是唯一的。

示例 1:

输入:  $gas = [1,2,3,4,5]$ ,  $cost = [3,4,5,1,2]$   
 输出: 3  
 解释:  
 从 3 号加油站(索引为 3 处)出发，可获得 4 升汽油。此时油箱有  $= 0 + 4 = 4$  升汽油  
 开往 4 号加油站，此时油箱有  $4 - 1 + 5 = 8$  升汽油  
 开往 0 号加油站，此时油箱有  $8 - 2 + 1 = 7$  升汽油  
 开往 1 号加油站，此时油箱有  $7 - 3 + 2 = 6$  升汽油  
 开往 2 号加油站，此时油箱有  $6 - 4 + 3 = 5$  升汽油  
 开往 3 号加油站，你需要消耗 5 升汽油，正好足够你返回到 3 号加油站。  
 因此，3 可为起始索引。

示例 2:

输入:  $gas = [2,3,4]$ ,  $cost = [3,4,3]$   
 输出: -1  
 解释:  
 你不能从 0 号或 1 号加油站出发，因为没有足够的汽油可以让你行驶到下一个加油站。  
 我们从 2 号加油站出发，可以获得 4 升汽油。此时油箱有  $= 0 + 4 = 4$  升汽油  
 开往 0 号加油站，此时油箱有  $4 - 3 + 2 = 3$  升汽油  
 开往 1 号加油站，此时油箱有  $3 - 3 + 3 = 3$  升汽油  
 你无法返回 2 号加油站，因为返程需要消耗 4 升汽油，但是你的油箱只有 3 升汽油。  
 因此，无论如何，你都不可能绕环路行驶一周。

提示:

```

1  n == gas.length == cost.length
1  1 <= n <= 10 ^5
1  0 <= gas[i], cost[i] <= 10 ^4
1  输入保证答案唯一。

```

我的题目思路与考察范围

核心思路：累计总油量差判断整体是否可行，同时维护从候选起点开始的区间和；区间和一旦为负，区间内任何站都不能作为起点，候选改为下一站。

关键不变量 / 状态：当前 candidate 到  $i$  的油量差始终非负；被跳过的所有起点都已证明无法越过首次负前缀。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：贪心 · 前缀和 · 环形数组 · 可行性

易错点：只有总差值非负才存在答案；重置候选点时区间和也必须清零。

## 135. 分发糖果

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 110/158  | 本章顺序 | 9/17                            |
| 难度   | 困难       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:1508 |

LeetCode : <https://leetcode.cn/problems/candy/>

## 题目内容

$n$  个孩子站成一排。

给你一个整数数组 ratings 表示每个孩子的评分。

你需要按照以下要求，给这些孩子分发糖果：

- 1 每个孩子 至少 分配到 1 个糖果。
- 1 相邻两个孩子中，评分 更高 的那个会获得更多的糖果。

请你给每个孩子分发糖果，计算并返回需要准备的 最少 糖果数目。

示例 1：

输入：ratings = [1,0,2]  
输出：5  
解释：你可以分别给第一个、第二个、第三个孩子分发 2、1、2 颗糖果。

示例 2：

输入：ratings = [1,2,2]  
输出：4  
解释：你可以分别给第一个、第二个、第三个孩子分发 1、2、1 颗糖果。  
第三个孩子只得到 1 颗糖果，这满足题面中的两个条件。

提示：

- 1  $1 \leq n == \text{ratings.length} \leq 5 * 10^4$
- 1  $0 \leq \text{ratings}[i] \leq 5 * 10^4$

我的题目思路与考察范围

核心思路：先从左到右保证评分上升时右孩子比左孩子多一颗，再从右到左保证评分下降时左孩子比右孩子多一颗，并取两个约束的最大值。

关键不变量 / 状态：第一遍满足所有左邻约束，第二遍在不破坏左邻约束的前提下补齐所有右邻约束。

复杂度：时间  $O(n)$ ，空间  $O(n)$ ；按坡度计数可进一步做到  $O(1)$  额外空间。

考察范围：贪心 · 双向扫描 · 局部约束 · 数组

易错点：第二遍不能直接覆盖第一遍结果，必须取 max，否则会破坏已经满足的另一侧约束。

## 860. 柠檬水找零

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 111/158  | 本章顺序 | 10/17                           |
| 难度   | 简单       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:1636 |

LeetCode : <https://leetcode.cn/problems/lemonade-change/>

## 题目内容

在柠檬水摊上，每一杯柠檬水的售价为 5 美元。顾客排队购买你的产品，（按账单 bills 支付的顺序）一次购买一杯。

每位顾客只买一杯柠檬水，然后向你付 5 美元、10 美元或 20 美元。你必须给每个顾客正确找零，也就是说净交易是每位顾客向你支付 5 美元。

注意，一开始你手头没有任何零钱。

给你一个整数数组 bills，其中 bills[i] 是第 i 位顾客付的账。如果你能给每位顾客正确找零，返回 true，否则返回 false。

示例 1：

输入：bills = [5,5,5,10,20]  
输出：true  
解释：  
前 3 位顾客那里，我们按顺序收取 3 张 5 美元的钞票。  
第 4 位顾客那里，我们收取一张 10 美元的钞票，并返还 5 美元。  
第 5 位顾客那里，我们找还一张 10 美元的钞票和一张 5 美元的钞票。  
由于所有客户都得到了正确的找零，所以我们输出 true。

示例 2：

输入：bills = [5,5,10,10,20]  
输出：false  
解释：  
前 2 位顾客那里，我们按顺序收取 2 张 5 美元的钞票。  
对于接下来的 2 位顾客，我们收取一张 10 美元的钞票，然后返还 5 美元。  
对于最后一位顾客，我们无法退回 15 美元，因为我们现在只有两张 10 美元的钞票。  
由于不是每位顾客都得到了正确的找零，所以答案是 false。

提示：

1 1 <= bills.length <= 10 ^5

1 bills[i] 不是 5 就是 10 或是 20

我的题目思路与考察范围

核心思路：只记录 5 元和 10 元钞票数量；收到 20 元时优先找一张 10 元加一张 5 元，因为 5 元更通用，无法这样找零时才用三张 5 元。

关键不变量 / 状态：处理完每位顾客后，手中钞票是此前所有合法找零选择中最保留 5 元灵活性的状态。

复杂度：时间 O(n)，空间 O(1)。

考察范围：贪心 · 模拟 · 计数 · 找零

易错点：给 20 元找零时应优先消耗 10+5，而不是优先消耗三张 5 元。

## 406. 根据身高重建队列

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 112/158  | 本章顺序 | 11/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:1760 |

LeetCode：https://leetcode.cn/problems/queue-reconstruction-by-height/

## 题目内容

假设有打乱顺序的一群人站成一个队列，数组 people 表示队列中一些人的属性（不一定按顺序）。每个 people[i] = [h<sub>i</sub>, k<sub>i</sub>] 表示第 i 个人的身高为 h<sub>i</sub>，前面正好有 k<sub>i</sub> 个身高大于或等于 h<sub>i</sub> 的人。

请你重新构造并返回输入数组 `people` 所表示的队列。返回的队列应该格式化为数组 `queue`，其中 `queue[j] = [h_j, k_j]` 是队列中第 `j` 个人的属性（`queue[0]` 是排在队列前面的人）。

示例 1：

输入：`people = [[7,0],[4,4],[7,1],[5,0],[6,1],[5,2]]`

输出：[[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]]

解释：

编号为 0 的人身高为 5，没有身高更高或者相同的人排在他前面。

编号为 1 的人身高为 7，没有身高更高或者相同的人排在他前面。

编号为 2 的人身高为 5，有 2 个身高更高或者相同的人排在他前面，即编号为 0 和 1 的人。

编号为 3 的人身高为 6，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。

编号为 4 的人身高为 4，有 4 个身高更高或者相同的人排在他前面，即编号为 0、1、2、3 的人。

编号为 5 的人身高为 7，有 1 个身高更高或者相同的人排在他前面，即编号为 1 的人。

因此 [[5,0],[7,0],[5,2],[6,1],[4,4],[7,1]] 是重新构造后的队列。

示例 2：

输入：`people = [[6,0],[5,0],[4,0],[3,2],[2,2],[1,4]]`

输出：[[4,0],[5,0],[2,2],[3,2],[1,4],[6,0]]

提示：

1 1 <= people.length <= 2000

1 0 <= h\_i <= 10^6

1 0 <= k\_i < people.length

1 题目数据确保队列可以被重建

我的题目思路与考察范围

核心思路：先按身高降序、同身高 `k` 升序排序，再把每个人插入结果的下标 `k`。插入时更高的人已经固定，矮人的后续插入不会改变当前人的前方高个数量。

关键不变量 / 状态：结果序列中的已插入者都不矮于当前人，且每个已插入者前方满足其 `k` 约束。

复杂度：排序  $O(n \log n)$ ，数组插入使总时间  $O(n^2)$ ，空间  $O(n)$ ；可用树状数组优化空位选择。

考察范围：贪心 · 排序 · 插入 · 二维约束

易错点：同身高必须按 `k` 升序，否则先插入较大的 `k` 可能没有足够位置或破坏顺序。

## 452. 用最少数量的箭引爆气球

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 113/158  | 本章顺序 | 12/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:2178 |

LeetCode：https://leetcode.cn/problems/minimum-number-of-arrows-to-burst-balloons/

### 题目内容

有一些球形气球贴在一堵用 `XY` 平面表示的墙面上。墙面上的气球记录在整数数组 `points`，其中 `points[i] = [x_start, x_end]` 表示水平直径在 `x_start` 和 `x_end` 之间的气球。你不知道气球的确切 `y` 坐标。

一支弓箭可以沿着 `x` 轴从不同点完全垂直地射出。在坐标 `x` 处射出一支箭，若有一个气球的直径的开始和结束坐标为 `x_start`，`x_end`，且满足 `x_start ≤ x ≤ x_end`，则该气球会被引爆。可以射出的弓箭的数量没有限制。弓箭一旦被射出之后，可以无限地前进。

给你一个数组 `points`，返回引爆所有气球所必须射出的最小弓箭数。

示例 1：

输入：points = [[10,16],[2,8],[1,6],[7,12]]  
 输出：2  
 解释：气球可以用2支箭来爆破：  
 -在x = 6处射出箭，击破气球[2,8]和[1,6]。  
 -在x = 11处发射箭，击破气球[10,16]和[7,12]。

示例 2：

输入：points = [[1,2],[3,4],[5,6],[7,8]]  
 输出：4  
 解释：每个气球需要射出一支箭，总共需要4支箭。

示例 3：

输入：points = [[1,2],[2,3],[3,4],[4,5]]  
 输出：2  
 解释：气球可以用2支箭来爆破：  
 - 在x = 2处发射箭，击破气球[1,2]和[2,3]。  
 - 在x = 4处射出箭，击破气球[3,4]和[4,5]。

提示：

```
1 1 <= points.length <= 10 ^5
1 points[i].length == 2
1 -2 ^31 <= x_start < x_end <= 2 ^31 - 1
```

我的题目思路与考察范围

核心思路：按区间右端点升序排序，在当前最早结束位置射箭；后续区间若起点不超过该位置就被同一箭覆盖，否则新开一箭。

关键不变量 / 状态：当前箭位于已处理且尚未分组区间的最小右端点，给未来区间留下最大的相交机会。

复杂度：时间  $O(n \log n)$ ，空间  $O(1)$ （不计排序栈）。

考察范围：贪心 · 区间 · 排序 · 最小覆盖

易错点：气球端点相等仍可被同一支箭射中，因此判断新箭应使用  $start > currentEnd$ 。

## 435. 无重叠区间

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 114/158  | 本章顺序 | 13/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:2314 |

LeetCode：https://leetcode.cn/problems/non-overlapping-intervals/

### 题目内容

给定一个区间的集合 intervals，其中  $intervals[i] = [start_i, end_i]$ 。返回需要移除区间的最小数量，使剩余区间互不重叠。

注意 只在一点上接触的区间是不重叠的。例如  $[1, 2]$  和  $[2, 3]$  是不重叠的。

示例 1:

输入: intervals = [[1,2],[2,3],[3,4],[1,3]]  
 输出: 1  
 解释: 移除 [1,3] 后，剩下的区间没有重叠。

示例 2:

输入: intervals = [ [1,2], [1,2], [1,2] ]  
 输出: 2

解释: 你需要移除两个 [1,2] 来使剩下的区间没有重叠。

示例 3:

输入: intervals = [ [1,2], [2,3] ]

输出: 0

解释: 你不需要移除任何区间, 因为它们已经是无重叠的了。

提示:

```
1 1 <= intervals.length <= 10 ^5
1 intervals[i].length == 2
1 -5 * 10 ^4 <= start_i < end_i <= 5 * 10 ^4
```

我的题目思路与考察范围

核心思路: 按右端点升序选择尽早结束的区间, 能接在当前结束点之后的就保留, 否则删除; 最少删除数等于总数减去最多可保留的不重叠区间数。

关键不变量 / 状态: 在已扫描区间中, 保留数量最大且最后一个保留区间的结束点尽可能小。

复杂度: 时间  $O(n \log n)$ , 空间  $O(1)$  ( 不计排序栈 )。

考察范围: 贪心 · 区间调度 · 排序 · 最少删除

易错点: 首尾相接不算重叠, 应允许  $\text{nextStart} \geq \text{currentEnd}$ 。

## 763. 划分字母区间

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 115/158  | 本章顺序 | 14/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:2512 |

LeetCode : <https://leetcode.cn/problems/partition-labels/>

### 题目内容

给你一个字符串  $s$ 。我们要把这个字符串划分为尽可能多的片段, 同一字母最多出现在一个片段中。例如, 字符串 "ababcc" 能够被分为 ["abab", "cc"], 但类似 ["aba", "bcc"] 或 ["ab", "ab", "cc"] 的划分是非法的。

注意, 划分结果需要满足: 将所有划分结果按顺序连接, 得到的字符串仍然是  $s$ 。

返回一个表示每个字符串片段的长度的列表。

示例 1:

输入:  $s = \text{"ababcbacadefegdehijhklij"}$

输出: [9,7,8]

解释:

划分结果为 "ababcbaca"、"defegde"、"hijhklij"。

每个字母最多出现在一个片段中。

像 "ababcbacadefegde", "hijhklij" 这样的划分是错误的, 因为划分的片段数较少。

示例 2:

输入:  $s = \text{"eccbbbbbdec"}$

输出: [10]

提示:

```
1 1 <= s.length <= 500
1 s 仅由小写英文字母组成
```

## 我的题目思路与考察范围

核心思路：先记录每个字符最后出现位置，扫描时把当前片段右边界更新为所见字符最后位置的最大值；当下标到达右边界即可安全切分。

关键不变量 / 状态：当前片段中所有已见字符的最后一次出现都不超过 end，只有  $i == \text{end}$  时片段才闭合。

复杂度：时间  $O(n)$ ，空间  $O(|\Sigma|)$ ，小写字母场景为  $O(1)$ 。

考察范围：贪心 · 区间合并 · 字符串 · 最后出现位置

易错点：边界必须取片段内所有字符最后位置的最大值，不能只看当前字符。

## 56. 合并区间

|      |          |      |                                  |
|------|----------|------|----------------------------------|
| 唯一题序 | 116/158  | 本章顺序 | 15/17                            |
| 难度   | 中等       | 题目类型 | 主讲题                              |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9. 《代码随想录》贪心算法 ( V3.0 ) .md:2642 |

LeetCode : <https://leetcode.cn/problems/merge-intervals/>

## 题目内容

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [start_i, end_i]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例 1：

输入：intervals = [[1,3],[2,6],[8,10],[15,18]]  
 输出：[[1,6],[8,10],[15,18]]  
 解释：区间 [1,3] 和 [2,6] 重叠，将它们合并为 [1,6]。

示例 2：

输入：intervals = [[1,4],[4,5]]  
 输出：[[1,5]]  
 解释：区间 [1,4] 和 [4,5] 可被视为重叠区间。

示例 3：

输入：intervals = [[4,7],[1,4]]  
 输出：[[1,7]]  
 解释：区间 [1,4] 和 [4,7] 可被视为重叠区间。

提示：

1  $1 \leq \text{intervals.length} \leq 10^4$

1  $\text{intervals}[i].\text{length} == 2$

1  $0 \leq \text{start}_i \leq \text{end}_i \leq 10^4$

## 我的题目思路与考察范围

核心思路：按起点升序排序，维护结果最后一个合并区间；若新区间起点不超过当前右端点就扩展右端点，否则开始新区间。

关键不变量 / 状态：结果中的区间互不重叠且覆盖所有已扫描区间，最后一个区间是唯一可能继续扩展的区间。

复杂度：时间  $O(n \log n)$ ，空间  $O(n)$ （输出空间）。

考察范围：排序 · 区间 · 贪心 · 扫描线

易错点：重叠时右端点要取  $\max(\text{currentEnd}, \text{newEnd})$ ，不能直接替换成新区间终点。



## 738. 单调递增的数字

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 117/158  | 本章顺序 | 16/17                           |
| 难度   | 中等       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:2819 |

LeetCode : <https://leetcode.cn/problems/monotone-increasing-digits/>

### 题目内容

当且仅当每个相邻位数上的数字  $x$  和  $y$  满足  $x \leq y$  时，我们称这个整数是 **单调递增** 的。

给定一个整数  $n$ ，返回 小于或等于  $n$  的最大数字，且数字呈 **单调递增**。

示例 1:

输入:  $n = 10$   
输出: 9

示例 2:

输入:  $n = 1234$   
输出: 1234

示例 3:

输入:  $n = 332$   
输出: 299

提示:

1  $0 \leq n \leq 10^9$

我的题目思路与考察范围

核心思路：从右向左找首次逆序，令左侧较大数字减一，并把它右边全部改为 9；继续向左检查可处理连续借位，最终得到不超过  $n$  的最大单调递增数字。

关键不变量 / 状态：扫描位置右侧已经确定为能使答案最大的合法后缀；一旦发生借位，标记位置后的数字都应为 9。

复杂度：时间  $O(d)$ ，空间  $O(d)$ ， $d$  为十进制位数。

考察范围：贪心 · 数字 · 字符串 · 从右向左

易错点：减一后可能制造新的左侧逆序，因此必须继续向左扫描，而不是只修复一次。

## 968. 监控二叉树

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 118/158  | 本章顺序 | 17/17                           |
| 难度   | 困难       | 题目类型 | 主讲题                             |
| 出现章节 | 第9章 贪心算法 | 源位置  | 9.《代码随想录》贪心算法 ( V3.0 ) .md:2937 |

LeetCode : <https://leetcode.cn/problems/binary-tree-cameras/>

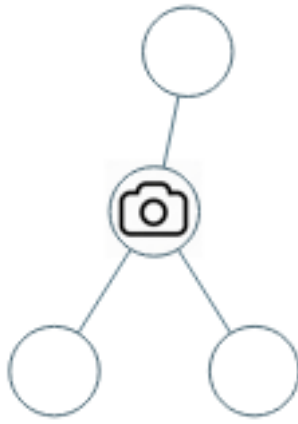
### 题目内容

给定一个二叉树，我们在树的节点上安装摄像头。

节点上的每个摄影头都可以监视 其父对象、自身及其直接子对象。

计算监控树的所有节点所需的最小摄像头数量。

示例 1：

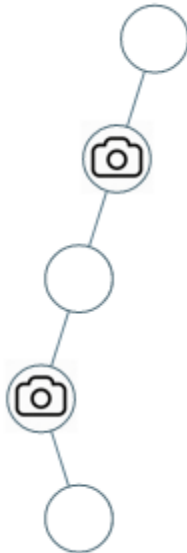


输入：[0,0,null,0,0]

输出：1

解释：如图所示，一台摄像头足以监控所有节点。

示例 2：



输入：[0,0,null,0,null,0,null,null,0]

输出：2

解释：需要至少两个摄像头来监视树的所有节点。上图显示了摄像头放置的有效位置之一。

提示：

- 1 给定树的节点数的范围是 [1, 1000]。
- 2 每个节点的值都是 0。

我的题目思路与考察范围

核心思路：后序遍历让每个节点返回未覆盖、有摄像头、已覆盖三种状态；孩子未覆盖时当前必须放相机，孩子有相机时当前已覆盖，否则当前暂不覆盖以让父节点放相机。根节点最后若未覆盖再补一台。

关键不变量 / 状态：处理一个节点后，其整棵子树除可能的当前节点外都已用最少摄像头覆盖，返回状态准确描述当前节点。

复杂度：时间  $O(n)$ ，递归栈空间  $O(h)$ 。

考察范围：贪心 · 二叉树 · 后序遍历 · 状态设计

易错点：空节点应视为已覆盖而不是有摄像头；遍历结束还要单独检查根节点状态。

# 第 10 章 动态规划

本章 37 道去重题目，按源文件首次出现顺序排列。

## 509. 斐波那契数

|      |           |      |                                 |
|------|-----------|------|---------------------------------|
| 唯一题序 | 119/158   | 本章顺序 | 1/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                             |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:141 |

LeetCode : <https://leetcode.cn/problems/fibonacci-number/>

### 题目内容

斐波那契数（通常用  $F(n)$  表示）形成的序列称为斐波那契数列。该数列由 0 和 1 开始，后面的每一项数字都是前面两项数字的和。也就是：

$$F(0) = 0, F(1) = 1$$
$$F(n) = F(n - 1) + F(n - 2), \text{ 其中 } n > 1$$

给定  $n$ ，请计算  $F(n)$ 。

示例 1：

输入： $n = 2$   
输出：1  
解释： $F(2) = F(1) + F(0) = 1 + 0 = 1$

示例 2：

输入： $n = 3$   
输出：2  
解释： $F(3) = F(2) + F(1) = 1 + 1 = 2$

示例 3：

输入： $n = 4$   
输出：3  
解释： $F(4) = F(3) + F(2) = 2 + 1 = 3$

提示：

$$0 \leq n \leq 30$$

我的题目思路与考察范围  
核心思路：只保存前两项并按  $F(i)=F(i-1)+F(i-2)$  向前滚动，无需完整 DP 数组；若追求更高阶性能可用矩阵快速幂或 fast doubling。  
关键不变量 / 状态：进入第  $i$  次计算时，两个变量分别等于  $F(i-2)$  与  $F(i-1)$ 。  
复杂度：迭代 DP 时间  $O(n)$ ，空间  $O(1)$ ；快速倍增可达  $O(\log n)$ 。  
考察范围：动态规划 · 滚动变量 · 递推 · 基础题  
易错点：先处理  $n=0$  和  $n=1$ ，滚动更新时不要覆盖仍需使用的旧值。

## 70. 爬楼梯

|      |           |      |                                 |
|------|-----------|------|---------------------------------|
| 唯一题序 | 120/158   | 本章顺序 | 2/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                             |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:298 |

LeetCode : <https://leetcode.cn/problems/climbing-stairs/>

### 题目内容

假设你正在爬楼梯。需要  $n$  阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

示例 1：

输入： $n = 2$   
 输出：2  
 解释：有两种方法可以爬到楼顶。  
 1. 1 阶 + 1 阶  
 2. 2 阶

示例 2：

输入： $n = 3$   
 输出：3  
 解释：有三种方法可以爬到楼顶。  
 1. 1 阶 + 1 阶 + 1 阶  
 2. 1 阶 + 2 阶  
 3. 2 阶 + 1 阶

提示：

1  $1 \leq n \leq 45$

我的题目思路与考察范围

核心思路：到第  $i$  阶的最后一步来自  $i-1$  或  $i-2$ ，因此  $ways[i]=ways[i-1]+ways[i-2]$ ，可压缩为两个变量。进阶为每次可走  $1..m$  阶时，它转化为求排列数的完全背包。

关键不变量 / 状态：滚动变量始终保存到达前两级台阶的方法数，二者覆盖到当前台阶的全部互斥最后一步。

复杂度：原题时间  $O(n)$ ，空间  $O(1)$ ； $1..m$  阶进阶版时间  $O(nm)$ 、空间  $O(n)$ 。

考察范围：动态规划 · 斐波那契 · 完全背包 · 排列数

易错点：原题与进阶完全背包的遍历语义不同；求排列时必须让容量在外、步数在内。

## 746. 使用最小花费爬楼梯

|      |           |      |                                 |
|------|-----------|------|---------------------------------|
| 唯一题序 | 121/158   | 本章顺序 | 3/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                             |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:499 |

LeetCode : <https://leetcode.cn/problems/min-cost-climbing-stairs/>

### 题目内容

给你一个整数数组  $cost$ ，其中  $cost[i]$  是从楼梯第  $i$  个台阶向上爬需要支付的费用。一旦你支付此费用，即可选择向上爬一个或者两个台阶。

你可以选择从下标为 0 或下标为 1 的台阶开始爬楼梯。

请你计算并返回达到楼梯顶部的最低花费。

示例 1：

输入：cost = [10,15,20]  
 输出：15  
 解释：你将从下标为 1 的台阶开始。  
 - 支付 15 ，向上爬两个台阶，到达楼梯顶部。  
 总花费为 15 。

示例 2：

输入：cost = [1,100,1,1,1,100,1,1,100,1]  
 输出：6  
 解释：你将从下标为 0 的台阶开始。  
 - 支付 1 ，向上爬两个台阶，到达下标为 2 的台阶。  
 - 支付 1 ，向上爬两个台阶，到达下标为 4 的台阶。  
 - 支付 1 ，向上爬两个台阶，到达下标为 6 的台阶。  
 - 支付 1 ，向上爬一个台阶，到达下标为 7 的台阶。  
 - 支付 1 ，向上爬两个台阶，到达下标为 9 的台阶。  
 - 支付 1 ，向上爬一个台阶，到达楼梯顶部。  
 总花费为 6 。

提示：

1 2 <= cost.length <= 1000

1 0 <= cost[i] <= 999

我的题目思路与考察范围

核心思路：把楼顶视为下标  $n$ ， $dp[i]$  表示到达  $i$  的最小花费，转移为  $\min(dp[i-1]+cost[i-1], dp[i-2]+cost[i-2])$ ；只需保存前两项。

关键不变量 / 状态：计算位置  $i$  时，前两项已经分别是到达  $i-1$ 、 $i-2$  的最小总花费。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 滚动数组 · 最小代价 · 线性DP

易错点：付费发生在离开某级台阶时，楼顶本身没有 cost，初始  $dp[0]=dp[1]=0$ 。

## 62. 不同路径

|      |           |      |                                 |
|------|-----------|------|---------------------------------|
| 唯一题序 | 122/158   | 本章顺序 | 4/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                             |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:825 |

LeetCode：https://leetcode.cn/problems/unique-paths/

### 题目内容

一个机器人位于一个  $m \times n$  网格的左上角（起始点在下图中标记为“Start”）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。

问总共有多少条不同的路径？

示例 1：



输入：m = 3, n = 7

输出：28

示例 2：

输入：m = 3, n = 2

输出：3

解释：

从左上角开始，总共有 3 条路径可以到达右下角。

1. 向右 -> 向下 -> 向下
2. 向下 -> 向下 -> 向右
3. 向下 -> 向右 -> 向下

示例 3：

输入：m = 7, n = 3

输出：28

示例 4：

输入：m = 3, n = 3

输出：6

提示：

1  $1 \leq m, n \leq 100$

1 题目数据保证答案小于等于  $2 * 10^9$

我的题目思路与考察范围

核心思路：机器人到达每个格子的路径数等于上方与左方路径数之和；首行和首列都只有一种走法，可用一维数组逐行累加。也可用组合数  $C(m+n-2, m-1)$ 。

关键不变量 / 状态：更新  $dp[j]$  前它表示上方格子的路径数，更新后表示当前格子且已加上左侧  $dp[j-1]$ 。

复杂度：DP 时间  $O(mn)$ ，空间  $O(n)$ ；组合公式可用  $O(\min(m, n))$  时间、 $O(1)$  空间。

考察范围：动态规划 · 网格 · 路径计数 · 组合数学

易错点：组合公式要避免中间乘法溢出；一维 DP 的首行初始化必须全为 1。

## 63. 不同路径 II

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 123/158   | 本章顺序 | 5/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:1062 |

LeetCode：https://leetcode.cn/problems/unique-paths-ii/

## 题目内容

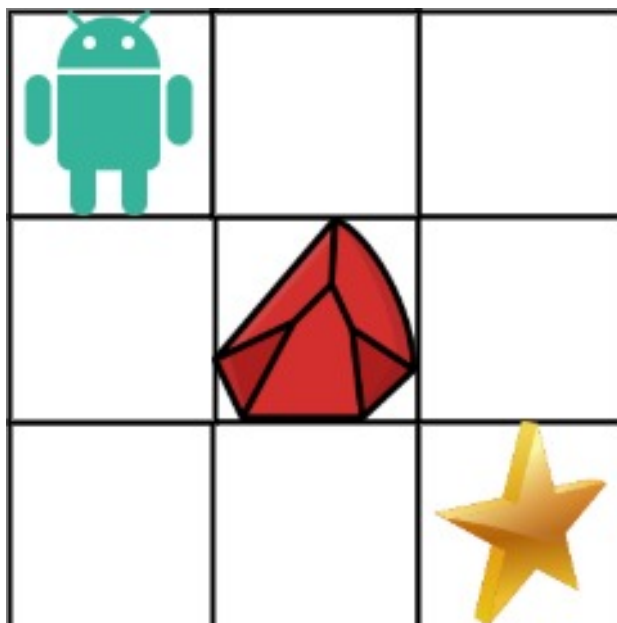
给定一个  $m \times n$  的整数数组 `grid`。一个机器人初始位于 左上角（即 `grid[0][0]`）。机器人尝试移动到 右下角（即 `grid[m - 1][n - 1]`）。机器人每次只能向下或者向右移动一步。

网格中的障碍物和空位置分别用 1 和 0 来表示。机器人的移动路径中不能包含任何有障碍物的方格。

返回机器人能够到达右下角的不同路径数量。

测试用例保证答案小于等于  $2 * 10^9$ 。

示例 1：



输入：obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]

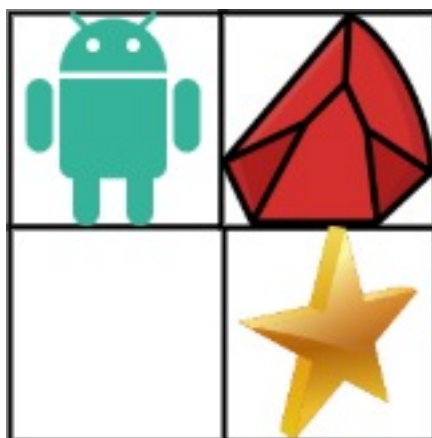
输出：2

解释：3x3 网格的正中间有一个障碍物。

从左上角到右下角一共有 2 条不同的路径：

1. 向右 -> 向右 -> 向下 -> 向下
2. 向下 -> 向下 -> 向右 -> 向右

示例 2：



输入：obstacleGrid = [[0,1],[0,0]]

输出：1

提示：

```
1 m == obstacleGrid.length
```

```

1  n == obstacleGrid[i].length
1  1 <= m, n <= 100
1  obstacleGrid[i][j] 为 0 或 1

```

我的题目思路与考察范围

核心思路：沿用网格路径 DP，但障碍格路径数强制置零；一维数组中非障碍格执行  $dp[j] += dp[j-1]$ ，让上方与左方贡献自然汇合。

关键不变量 / 状态：每次更新后  $dp[j]$  是从起点到当前格且不经过障碍的路径总数。

复杂度：时间  $O(mn)$ ，空间  $O(n)$ 。

考察范围：动态规划 · 网格 · 路径计数 · 障碍

易错点：起点或终点可能是障碍；遇障碍必须把  $dp[j]$  清零，不能仅跳过更新。

## 343. 整数拆分

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 124/158   | 本章顺序 | 6/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:1258 |

LeetCode：https://leetcode.cn/problems/integer-break/

### 题目内容

给定一个正整数  $n$ ，将其拆分为  $k$  个正整数的和 ( $k \geq 2$ )，并使这些整数的乘积最大化。

返回 你可以获得的最大乘积。

示例 1:

输入:  $n = 2$   
 输出: 1  
 解释:  $2 = 1 + 1, 1 \times 1 = 1$ 。

示例 2:

输入:  $n = 10$   
 输出: 36  
 解释:  $10 = 3 + 3 + 4, 3 \times 3 \times 4 = 36$ 。

提示:

```

1  2 <= n <= 58

```

我的题目思路与考察范围

核心思路：DP 枚举第一段  $j$ ，并比较  $j \times (i-j)$  与  $j \times dp[i-j]$ ；也可证明尽量拆成 3、余 1 时改成 2+2 的贪心最优。

关键不变量 / 状态：计算  $dp[i]$  时，所有更小整数的最优拆分乘积已经确定，枚举  $j$  覆盖所有第一刀位置。

复杂度：DP 时间  $O(n^2)$ 、空间  $O(n)$ ；数学贪心时间  $O(n)$  或用幂运算  $O(\log n)$ 、空间  $O(1)$ 。

考察范围：动态规划 · 贪心 · 数学 · 整数拆分

易错点：题目要求至少拆成两个正整数，不能把  $n$  本身当作合法答案。



## 96. 不同的二叉搜索树

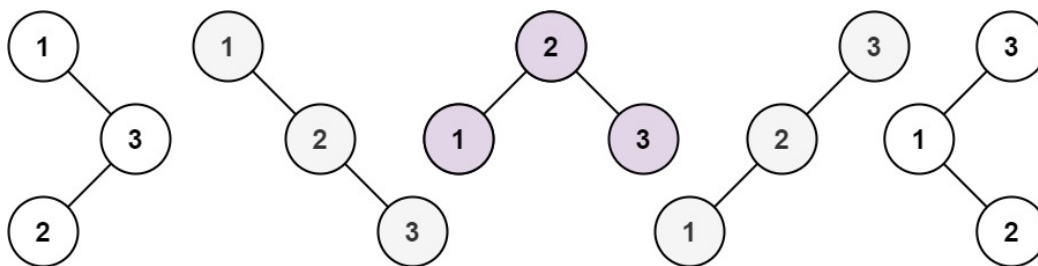
|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 125/158   | 本章顺序 | 7/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:1470 |

LeetCode : <https://leetcode.cn/problems/unique-binary-search-trees/>

### 题目内容

给你一个整数  $n$ ，求恰由  $n$  个节点组成且节点值从 1 到  $n$  互不相同的二叉搜索树有多少种？返回满足题意的二叉搜索树的种数。

示例 1：



输入： $n = 3$   
输出：5

示例 2：

输入： $n = 1$   
输出：1

提示：

1  $1 \leq n \leq 19$

我的题目思路与考察范围

核心思路：以  $j$  为根时，左侧  $j-1$  个节点和右侧  $i-j$  个节点的结构可独立组合，因此  $dp[i]$  累加  $dp[j-1]*dp[i-j]$ ，得到 Catalan 数。

关键不变量 / 状态： $dp[i]$  精确表示由  $i$  个有序节点可构成的不同 BST 数量，每个根节点划分互斥且完备。

复杂度：时间  $O(n^2)$ ，空间  $O(n)$ 。

考察范围：动态规划 · Catalan 数 · 二叉搜索树 · 计数

易错点：空子树必须计为一种结构，所以  $dp[0]=1$  是乘法转移成立的关键。

## 416. 分割等和子集

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 126/158   | 本章顺序 | 8/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:2236 |

LeetCode : <https://leetcode.cn/problems/partition-equal-subset-sum/>

## 题目内容

给你一个只包含正整数的非空数组 `nums`。请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

示例 1：

输入：`nums = [1,5,11,5]`  
 输出：`true`  
 解释：数组可以分割成 `[1, 5, 5]` 和 `[11]`。

示例 2：

输入：`nums = [1,2,3,5]`  
 输出：`false`  
 解释：数组不能分割成两个元素和相等的子集。

提示：

1 `1 <= nums.length <= 200`

1 `1 <= nums[i] <= 100`

我的题目思路与考察范围

核心思路：总和为奇数直接失败；否则把问题转成容量 `sum/2` 的 0/1 背包，判断是否能恰好选出目标和。容量必须倒序更新以保证每个数只用一次。

关键不变量 / 状态：处理前 `i` 个数后，`dp[j]` 表示容量 `j` 是否可由这些数构成，且当前数不会在同一轮被重复使用。

复杂度：时间  $O(nS)$ ，空间  $O(S)$ ， $S = \text{sum}/2$ 。

考察范围：动态规划 · 0/1 背包 · 子集和 · 可行性

易错点：容量正序会把 0/1 背包错误变成完全背包；必须先检查总和奇偶。

## 1049. 最后一块石头的重量 II

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 127/158   | 本章顺序 | 9/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:2427 |

LeetCode：https://leetcode.cn/problems/last-stone-weight-ii/

## 题目内容

有一堆石头，用整数数组 `stones` 表示。其中 `stones[i]` 表示第 `i` 块石头的重量。

每一回合，从中选出任意两块石头，然后将它们一起粉碎。假设石头的重量分别为 `x` 和 `y`，且  $x \leq y$ 。那么粉碎的可能结果如下：

1 如果  $x == y$ ，那么两块石头都会被完全粉碎；

1 如果  $x \neq y$ ，那么重量为 `x` 的石头将会完全粉碎，而重量为 `y` 的石头新重量为 `y-x`。

最后，最多只会剩下一块石头。返回此石头最小的可能重量。如果没有石头剩下，就返回 0。

示例 1：

输入：`stones = [2,7,4,1,8,1]`  
 输出：`1`  
 解释：  
 组合 2 和 4，得到 2，所以数组转化为 `[2,7,1,8,1]`，  
 组合 7 和 8，得到 1，所以数组转化为 `[2,1,1,1]`，  
 组合 2 和 1，得到 1，所以数组转化为 `[1,1,1]`，

组合 1 和 1，得到 0，所以数组转化为 [1]，这就是最优值。

示例 2：

输入：stones = [31,26,33,21,40]  
输出：5

提示：

1 1 <= stones.length <= 30

1 1 <= stones[i] <= 100

我的题目思路与考察范围

核心思路：把石头分成两组，使一组重量尽量接近总和的一半；用 0/1 背包求不超过 sum/2 的最大可达重量 best，答案是 sum-2\*best。

关键不变量 / 状态：处理若干石头后，dp[j] 是容量 j 下可装入的最大重量，每块石头至多使用一次。

复杂度：时间 O(nS)，空间 O(S)，S=floor(sum/2)。

考察范围：动态规划 · 0/1 背包 · 分组 · 最小差值

易错点：求的是最接近一半的可达重量，不是直接模拟每次粉碎最大的两块石头。

## 494. 目标和

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 128/158   | 本章顺序 | 10/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:2730 |

LeetCode：https://leetcode.cn/problems/target-sum/

### 题目内容

给你一个非负整数数组 nums 和一个整数 target。

向数组中的每个整数前添加 '+' 或 '-'，然后串联起所有整数，可以构造一个表达式：

1 例如，nums = [2, 1]，可以在 2 之前添加 '+'，在 1 之前添加 '-'，然后串联起来得到表达式 "+2-1"。

返回可以通过上述方法构造的、运算结果等于 target 的不同 表达式 的数目。

示例 1：

输入：nums = [1,1,1,1,1], target = 3  
输出：5  
解释：一共有 5 种方法让最终目标和为 3。  
-1 + 1 + 1 + 1 + 1 = 3  
+1 - 1 + 1 + 1 + 1 = 3  
+1 + 1 - 1 + 1 + 1 = 3  
+1 + 1 + 1 - 1 + 1 = 3  
+1 + 1 + 1 + 1 - 1 = 3

示例 2：

输入：nums = [1], target = 1  
输出：1

提示：

1 1 <= nums.length <= 20

1 0 <= nums[i] <= 1000

```
1 0 <= sum(nums[i]) <= 1000
```

```
1 -1000 <= target <= 1000
```

我的题目思路与考察范围

核心思路：令正号集合和为  $P$ ，则  $P - (\text{sum} - P) = \text{target}$ ，得到  $P = (\text{sum} + \text{target}) / 2$ ；问题变成统计和为  $P$  的 0/1 子集数量，容量倒序累加方案数。

关键不变量 / 状态：处理前  $i$  个数后， $\text{dp}[j]$  是用这些数恰好组成  $j$  的方案数，每个元素的正负选择只计一次。

复杂度：时间  $O(nP)$ ，空间  $O(P)$ 。

考察范围：动态规划 · 0/1 背包 · 方案计数 · 代数变换

易错点：若  $\text{abs}(\text{target}) > \text{sum}$  或  $\text{sum} + \text{target}$  为奇数必须返回 0； $\text{dp}[0] = 1$  才能正确处理零元素。

## 474. 一和零

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 129/158   | 本章顺序 | 11/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:2982 |

LeetCode：https://leetcode.cn/problems/ones-and-zeros/

### 题目内容

给你一个二进制字符串数组  $\text{strs}$  和两个整数  $m$  和  $n$ 。

请你找出并返回  $\text{strs}$  的最大子集的长度，该子集中最多有  $m$  个 0 和  $n$  个 1。

如果  $x$  的所有元素也是  $y$  的元素，集合  $x$  是集合  $y$  的子集。

示例 1：

输入： $\text{strs} = ["10", "0001", "111001", "1", "0"]$ ,  $m = 5$ ,  $n = 3$

输出：4

解释：最多有 5 个 0 和 3 个 1 的最大子集是  $\{"10", "0001", "1", "0"\}$ ，因此答案是 4。

其他满足题意但较小的子集包括  $\{"0001", "1"\}$  和  $\{"10", "1", "0"\}$ 。 $\{"111001"\}$  不满足题意，因为它含 4 个 1，大于  $n$  的值 3。

示例 2：

输入： $\text{strs} = ["10", "0", "1"]$ ,  $m = 1$ ,  $n = 1$

输出：2

解释：最大的子集是  $\{"0", "1"\}$ ，所以答案是 2。

提示：

```
1 1 <= str.length <= 600
```

```
1 1 <= str[i].length <= 100
```

```
1 str[i] 仅由 '0' 和 '1' 组成
```

```
1 1 <= m, n <= 100
```

我的题目思路与考察范围

核心思路：每个字符串消耗若干个 0 和 1，是有两个容量维度的 0/1 背包；统计 zero/one 后让两个容量都倒序，更新最多可选字符串数。

关键不变量 / 状态：处理过的字符串各至多选一次， $\text{dp}[i][j]$  是不超过  $i$  个 0、 $j$  个 1 时的最大子集大小。

复杂度：时间  $O(Lmn)$ ，空间  $O(mn)$ ， $L$  为字符串数量。

考察范围：动态规划 · 二维背包 · 0/1 背包 · 字符串计数

易错点：两个容量都必须倒序；漏掉任一维倒序都会让同一字符串被重复选择。

## 518. 零钱兑换 II

|      |           |      |                                   |
|------|-----------|------|-----------------------------------|
| 唯一题序 | 130/158   | 本章顺序 | 12/37                             |
| 难度   | 中等        | 题目类型 | 主讲题                               |
| 出现章节 | 第10章 动态规划 | 源位置  | 10. 《代码随想录》动态规划 ( V3.0 ) .md:3306 |

LeetCode : <https://leetcode.cn/problems/coin-change-ii/>

### 题目内容

给你一个整数数组 `coins` 表示不同面额的硬币，另给一个整数 `amount` 表示总金额。

请你计算并返回可以凑成总金额的硬币组合数。如果任何硬币组合都无法凑出总金额，返回 0。

假设每一种面额的硬币有无限个。

题目数据保证最终结果符合 32 位带符号整数。

示例 1：

```
输入：amount = 5, coins = [1, 2, 5]
输出：4
解释：有四种方式可以凑成总金额：
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1
```

示例 2：

```
输入：amount = 3, coins = [2]
输出：0
解释：只用面额 2 的硬币不能凑成总金额 3。
```

示例 3：

```
输入：amount = 10, coins = [10]
输出：1
```

提示：

- 1 `1 <= coins.length <= 300`
- 1 `1 <= coins[i] <= 5000`
- 1 `coins` 中的所有值 互不相同
- 1 `0 <= amount <= 5000`

我的题目思路与考察范围

核心思路：这是求组合数的完全背包：dp[0]=1，先枚举硬币，再让金额正序增长，使每种组合只按固定硬币顺序生成一次。

关键不变量 / 状态：处理到第 `i` 种硬币后，dp[a] 是只使用前 `i` 种硬币凑成金额 `a` 的组合数。

复杂度：时间  $O(nA)$ ，空间  $O(A)$ 。

考察范围：动态规划 · 完全背包 · 组合计数 · 零钱

易错点：交换两层循环会统计排列而非组合；完全背包的金额必须正序。

## 377. 组合总和 IV

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 131/158   | 本章顺序 | 13/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:3622 |

LeetCode : <https://leetcode.cn/problems/combination-sum-iv/>

### 题目内容

给你一个由 不同 整数组成的数组 `nums` ，和一个目标整数 `target` 。请你从 `nums` 中找出并返回总和为 `target` 的元素组合的个数。

题目数据保证答案符合 32 位整数范围。

示例 1：

```
输入：nums = [1,2,3], target = 4
输出：7
解释：
所有可能的组合为：
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
(2, 1, 1)
(2, 2)
(3, 1)
请注意，顺序不同的序列被视作不同的组合。
```

示例 2：

```
输入：nums = [9], target = 3
输出：0
```

提示：

- 1 `1 <= nums.length <= 200`
- 1 `1 <= nums[i] <= 1000`
- 1 `nums` 中的所有元素 互不相同
- 1 `1 <= target <= 1000`

进阶：如果给定的数组中含有负数会发生什么？问题会产生何种变化？如果允许负数出现，需要向题目中添加哪些限制条件？

#### 我的题目思路与考察范围

核心思路：不同顺序算不同方案，因此先枚举目标和 `i`，再枚举可作为最后一个数的 `nums[j]`，累加 `dp[i-nums[j]]`，本质是完全背包的排列计数。

关键不变量 / 状态：`dp[i]` 已统计所有总和为 `i` 的有序序列，每个序列按其最后一个元素被唯一归类。

复杂度：时间  $O(nT)$ ，空间  $O(T)$ 。

考察范围：动态规划 · 完全背包 · 排列计数 · 方案数

易错点：外层应遍历容量、内层遍历物品；方案数可能溢出，要遵守题目数值范围。

## 322. 零钱兑换

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 132/158   | 本章顺序 | 14/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:3867 |

LeetCode : <https://leetcode.cn/problems/coin-change/>

### 题目内容

给你一个整数数组 `coins`，表示不同面额的硬币；以及一个整数 `amount`，表示总金额。

计算并返回可以凑成总金额所需的 最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 `-1`。

你可以认为每种硬币的数量是无限的。

示例 1：

输入：coins = [1, 2, 5], amount = 11  
输出：3  
解释：11 = 5 + 5 + 1

示例 2：

输入：coins = [2], amount = 3  
输出：-1

示例 3：

输入：coins = [1], amount = 0  
输出：0

提示：

1 1 <= coins.length <= 12

1 1 <= coins[i] <= 2<sup>31</sup> - 1

1 0 <= amount <= 10<sup>4</sup>

我的题目思路与考察范围

核心思路：令 `dp[a]` 为凑成金额 `a` 的最少硬币数，完全背包正序更新 `dp[a]=min(dp[a],dp[a-coin]+1)`；无法到达的状态初始化为无穷。

关键不变量 / 状态：处理到当前硬币与金额时，`dp[a]` 是已允许硬币集合凑成 `a` 的最少数目。

复杂度：时间  $O(nA)$ ，空间  $O(A)$ 。

考察范围：动态规划 · 完全背包 · 最优化 · 零钱

易错点：不要从无穷状态加一；`dp[0]=0`，最终仍为无穷时返回 `-1`。

## 279. 完全平方数

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 133/158   | 本章顺序 | 15/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:4046 |

LeetCode : <https://leetcode.cn/problems/perfect-squares/>

## 题目内容

给你一个整数  $n$ ，返回和为  $n$  的完全平方数的最少数量。

完全平方数是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例 1：

输入： $n = 12$   
输出：3  
解释： $12 = 4 + 4 + 4$

示例 2：

输入： $n = 13$   
输出：2  
解释： $13 = 4 + 9$

提示：

1  $1 \leq n \leq 10^4$

我的题目思路与考察范围

核心思路：把每个平方数视为可无限使用的硬币， $dp[i]$  取  $dp[i-j^2]+1$  的最小值；也可将每层减去平方数做 BFS，首次到零的层数即答案。

关键不变量 / 状态：计算  $dp[i]$  时，所有更小金额的最少平方数个数已知，枚举最后一个平方数覆盖所有最优解。

复杂度：时间  $O(n\sqrt{n})$ ，空间  $O(n)$ 。

考察范围：动态规划 · 完全背包 · BFS · 数论

易错点：平方数可以重复使用，容量需正序；循环条件应为  $j^2 \leq i$ 。

## 139. 单词拆分

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 134/158   | 本章顺序 | 16/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:4387 |

LeetCode：https://leetcode.cn/problems/word-break/

## 题目内容

给你一个字符串  $s$  和一个字符串列表  $wordDict$  作为字典。如果可以利用字典中出现的一个或多个单词拼接出  $s$  则返回  $true$ 。

注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例 1：

输入： $s = \text{"leetcode"}$ ,  $wordDict = [\text{"leet"}, \text{"code"}]$   
输出： $true$   
解释：返回  $true$  因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

示例 2：

输入： $s = \text{"applepenapple"}$ ,  $wordDict = [\text{"apple"}, \text{"pen"}]$   
输出： $true$   
解释：返回  $true$  因为 "applepenapple" 可以由 "apple" "pen" "apple" 拼接成。  
注意，你可以重复使用字典中的单词。

示例 3：



输入: s = "catsandog", wordDict = ["cats", "dog", "sand", "and", "cat"]  
输出: false

提示：

- 1 1 <= s.length <= 300
- 1 1 <= wordDict.length <= 1000
- 1 1 <= wordDict[i].length <= 20
- 1 s 和 wordDict[i] 仅由小写英文字母组成
- 1 wordDict 中的所有字符串 互不相同

我的题目思路与考察范围

核心思路：dp[i] 表示前 i 个字符能否拆分，枚举切点 j；若 dp[j] 为真且 s[j:i] 在词典中，则 dp[i] 为真。可按最大单词长度限制 j 的范围。

关键不变量 / 状态：dp[i] 为真当且仅当前缀 s[0:i] 存在由词典词组成的完整分割。

复杂度：状态转移  $O(n^2)$ ，空间  $O(n+|dict|)$ ；子串构造成本依语言实现而定。

考察范围：动态规划 · 字符串 · 完全背包 · 可行性

易错点：dp[0] 必须为 true 表示空前缀可分割；不能只用贪心选择最长或最短单词。

## 198. 打家劫舍

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 135/158   | 本章顺序 | 17/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:4857 |

LeetCode：https://leetcode.cn/problems/house-robber/

### 题目内容

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你 不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

示例 1：

输入：[1,2,3,1]  
输出：4  
解释：偷窃 1 号房屋 (金额 = 1)，然后偷窃 3 号房屋 (金额 = 3)。  
偷窃到的最高金额 = 1 + 3 = 4。

示例 2：

输入：[2,7,9,3,1]  
输出：12  
解释：偷窃 1 号房屋 (金额 = 2)，偷窃 3 号房屋 (金额 = 9)，接着偷窃 5 号房屋 (金额 = 1)。  
偷窃到的最高金额 = 2 + 9 + 1 = 12。

提示：

- 1 1 <= nums.length <= 100
- 1 0 <= nums[i] <= 400

## 我的题目思路与考察范围

核心思路：对每间房比较不抢它的  $dp[i-1]$  与抢它的  $dp[i-2]+nums[i]$ ，取较大者；只保存前两项即可。

关键不变量 / 状态：处理到第  $i$  间房后，当前状态是前缀  $[0,i]$  在不抢相邻房约束下的最大金额。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 线性DP · 相邻约束 · 滚动变量

易错点：空数组和单元数组要单独兼容，滚动变量分别对应  $i-2$  与  $i-1$ 。

## 213. 打家劫舍 II

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 136/158   | 本章顺序 | 18/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:4967 |

LeetCode：https://leetcode.cn/problems/house-robber-ii/

## 题目内容

你是一个专业的小偷，计划偷窃沿街的房屋，每间房内都藏有一定的现金。这个地方所有的房屋都围成一圈，这意味着第一个房屋和最后一个房屋是紧挨着的。同时，相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。

给定一个代表每个房屋存放金额的非负整数数组，计算你 在不触动警报装置的情况下，今晚能够偷窃到的最高金额。

示例 1：

输入：nums = [2,3,2]

输出：3

解释：你不能先偷窃 1 号房屋（金额 = 2），然后偷窃 3 号房屋（金额 = 2），因为他们是相邻的。

示例 2：

输入：nums = [1,2,3,1]

输出：4

解释：你可以先偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3）。

偷窃到的最高金额 = 1 + 3 = 4。

示例 3：

输入：nums = [1,2,3]

输出：3

提示：

1 1 <= nums.length <= 100

1 0 <= nums[i] <= 1000

## 我的题目思路与考察范围

核心思路：首尾相邻不能同时抢，把环拆成两个线性问题：抢劫  $[0,n-2]$  或  $[1,n-1]$ ，分别运行打家劫舍 DP 后取最大值。

关键不变量 / 状态：每个线性子问题都排除环上的一个端点，因此其最优解天然满足首尾不同时出现。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 环形数组 · 分类讨论 · 相邻约束

易错点： $n=1$  时两个区间会失效，必须先直接返回唯一房屋金额。

## 337. 打家劫舍 III

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 137/158   | 本章顺序 | 19/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:5062 |

LeetCode : <https://leetcode.cn/problems/house-robber-iii/>

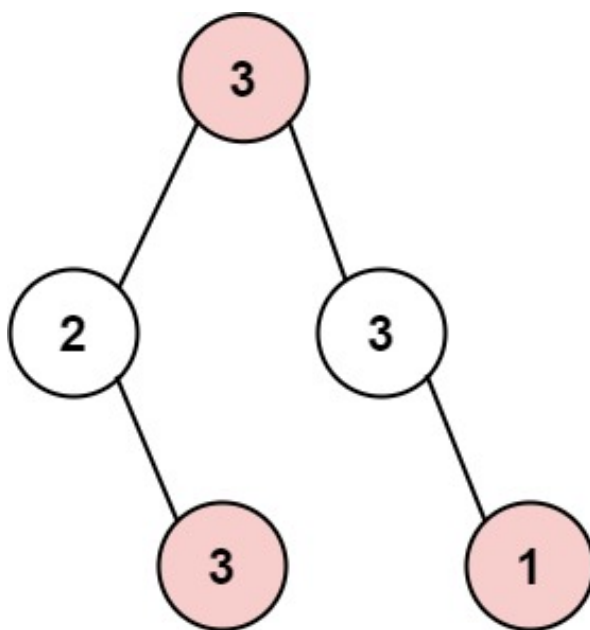
### 题目内容

小偷又发现了一个新的可行窃的地区。这个地区只有一个入口，我们称之为 root。

除了 root 之外，每栋房子有且只有一个“父”房子与之相连。一番侦察之后，聪明的小偷意识到“这个地方的所有房屋的排列类似于一棵二叉树”。如果两个直接相连的房子在同一天晚上被打劫，房屋将自动报警。

给定二叉树的 root。返回在不触动警报的情况下，小偷能够盗取的最高金额。

示例 1:

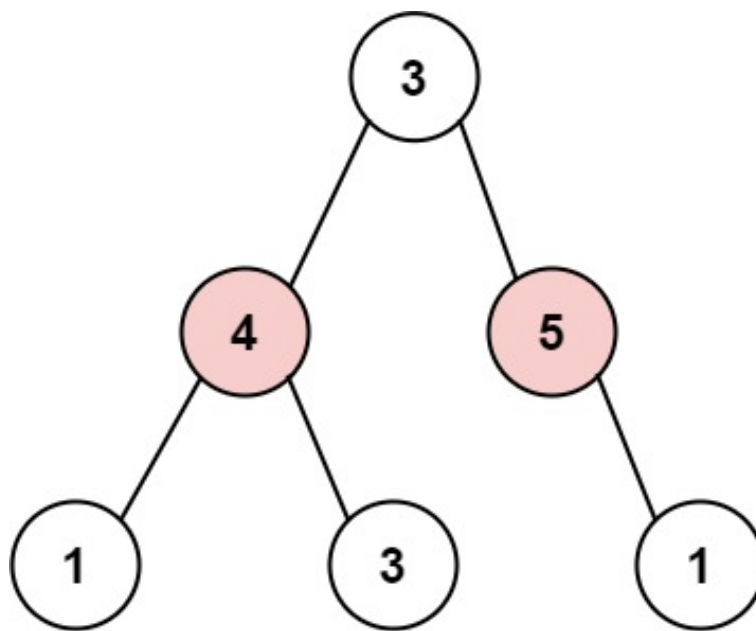


输入: root = [3,2,3,null,3,null,1]

输出: 7

解释: 小偷一晚能够盗取的最高金额  $3 + 3 + 1 = 7$

示例 2:



输入: root = [3,4,5,1,3,null,1]

输出: 9

解释: 小偷一晚能够盗取的最高金额  $4 + 5 = 9$

提示：

1 树的节点数在  $[1, 10^4]$  范围内

1  $0 \leq \text{Node.val} \leq 10^4$

我的题目思路与考察范围

核心思路：后序遍历每个节点返回两个值：不抢当前节点的最大金额与抢当前节点的最大金额；抢当前只能接孩子不抢状态，不抢当前可取孩子两状态最大值。

关键不变量 / 状态：每个节点返回值已经是其整棵子树在当前节点抢或不抢条件下的最优合法结果。

复杂度：时间  $O(n)$ ，递归栈空间  $O(h)$ 。

考察范围：动态规划 · 树形DP · 后序遍历 · 状态选择

易错点：不抢当前节点不代表必须抢孩子，应对每个孩子取抢与不抢的最大值。

## 121. 买卖股票的最佳时机

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 138/158   | 本章顺序 | 20/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:5274 |

LeetCode：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock/

### 题目内容

给定一个数组 `prices`，它的第  $i$  个元素 `prices[i]` 表示一支给定股票第  $i$  天的价格。

你只能选择 某一天 买入这只股票，并选择在 未来的某一个不同的日子 卖出该股票。设计一个算法来计算你能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 0。

示例 1：

输入：[7,1,5,3,6,4]

输出：5

解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5。

注意利润不能是 7-1 = 6，因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

示例 2：

输入：prices = [7,6,4,3,1]

输出：0

解释：在这种情况下，没有交易完成，所以最大利润为 0。

提示：

1 1 <= prices.length <= 10 ^5

1 0 <= prices[i] <= 10 ^4

我的题目思路与考察范围

核心思路：单次交易时扫描价格并维护此前最低买入价，用当天价格减最低价更新最大利润；也可写成持有/不持有两状态 DP。

关键不变量 / 状态：到第 i 天时 minPrice 是前缀最低价，answer 是在前缀内完成一次合法买卖的最大利润。

复杂度：时间 O(n)，空间 O(1)。

考察范围：贪心 · 动态规划 · 股票 · 前缀最值

易错点：必须先买后卖，因此最低价只能来自当天或更早，不能用未来价格。

## 122. 买卖股票的最佳时机 II ( 复练 )

本章顺序：21/37

LeetCode：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-ii/

完整收录：第 9 章 贪心算法

复练重点：本章改用动态规划视角复练：先定义状态，再写清转移、初始化与遍历顺序。

## 123. 买卖股票的最佳时机 III

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 139/158   | 本章顺序 | 22/37                            |
| 难度   | 困难        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:5894 |

LeetCode：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-iii/

### 题目内容

给定一个数组，它的第 i 个元素是一支给定的股票在第 i 天的价格。

设计一个算法来计算你能获取的最大利润。你最多可以完成 两笔 交易。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入：prices = [3,3,5,0,0,3,1,4]

输出：6

解释：在第 4 天（股票价格 = 0）的时候买入，在第 6 天（股票价格 = 3）的时候卖出，这笔交易所能获得利润 = 3-0 = 3。

随后，在第 7 天（股票价格 = 1）的时候买入，在第 8 天（股票价格 = 4）的时候卖出，这笔交易所能获得利润 = 4-1 = 3。

示例 2：

输入：prices = [1,2,3,4,5]

输出：4

解释：在第 1 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 5）的时候卖出，这笔交易所能获得利润 = 5-1 = 4。

注意你不能在第 1 天和第 2 天接连购买股票，之后再将它们卖出。

因为这样属于同时参与了多笔交易，你必须在再次购买前出售掉之前的股票。

示例 3：

输入：prices = [7,6,4,3,1]

输出：0

解释：在这个情况下，没有交易完成，所以最大利润为 0。

示例 4：

输入：prices = [1]

输出：0

提示：

1 1 <= prices.length <= 10 ^5

1 0 <= prices[i] <= 10 ^5

我的题目思路与考察范围

核心思路：维护第一次买入、第一次卖出、第二次买入、第二次卖出四个状态，每天按旧状态更新；最终第二次卖出状态包含至多两笔交易的最优利润。

关键不变量 / 状态：每个状态是在扫描到当天、完成对应次数操作并处于指定持仓状态时的最大现金。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 股票 · 状态机 · 有限交易

易错点：买入状态应初始化为 -prices[0]；“至多两次”允许第二笔交易不发生。

## 188. 买卖股票的最佳时机 IV

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 140/158   | 本章顺序 | 23/37                            |
| 难度   | 困难        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:6095 |

LeetCode：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-iv/

### 题目内容

给你一个整数数组 prices 和一个整数 k，其中 prices[i] 是某支给定的股票在第 i 天的价格。

设计一个算法来计算你能获取的最大利润。你最多可以完成 k 笔交易。也就是说，你最多可以买 k 次，卖 k 次。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1：

输入：k = 2, prices = [2,4,1]

输出：2

解释：在第 1 天（股票价格 = 2）的时候买入，在第 2 天（股票价格 = 4）的时候卖出，这笔交易所能获得利润 = 4-2 = 2。

示例 2：

输入：k = 2, prices = [3,2,6,5,0,3]

输出：7

解释：在第 2 天（股票价格 = 2）的时候买入，在第 3 天（股票价格 = 6）的时候卖出，这笔交易所能获得利润 = 6-2 = 4。

随后，在第 5 天（股票价格 = 0）的时候买入，在第 6 天（股票价格 = 3）的时候卖出，这笔交易所能获得利润 = 3-0 = 3。

提示：

```

1 1 <= k <= 100
1 1 <= prices.length <= 1000
1 0 <= prices[i] <= 1000

```

我的题目思路与考察范围

核心思路：用  $2k$  个买入/卖出状态滚动更新，第  $j$  次买入来自第  $j-1$  次卖出减当天价格，第  $j$  次卖出来自对应买入加价格。若  $k \geq n/2$ ，可退化为无限次交易。

关键不变量 / 状态：扫描到每天后，每个状态都表示不超过相应交易次数且处于指定持仓阶段的最大利润。

复杂度：时间  $O(nk)$ ，空间  $O(k)$ ；退化分支时间  $O(n)$ 。

考察范围：动态规划 · 股票 · 状态机 ·  $k$  次交易

易错点：一笔交易由一次买入和一次卖出组成； $k$  很大时若不走无限交易分支会浪费时间和内存。

## 309. 买卖股票的最佳时机含冷冻期

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 141/158   | 本章顺序 | 24/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:6257 |

LeetCode：https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-with-cooldown/

### 题目内容

给定一个整数数组 `prices`，其中第 `prices[i]` 表示第  $i$  天的股票价格。

设计一个算法计算出最大利润。在满足以下约束条件下，你可以尽可能地完成更多的交易（多次买卖一支股票）：

- 1 卖出股票后，你无法在第二天买入股票（即冷冻期为 1 天）。

注意：你不能同时参与多笔交易（你必须在再次购买前出售掉之前的股票）。

示例 1:

输入: `prices = [1,2,3,0,2]`  
 输出: 3  
 解释: 对应的交易状态为: [买入, 卖出, 冷冻期, 买入, 卖出]

示例 2:

输入: `prices = [1]`  
 输出: 0

提示：

```

1 1 <= prices.length <= 5000
1 0 <= prices[i] <= 1000

```

我的题目思路与考察范围

核心思路：区分持有、当天卖出、冷冻后可买等状态；买入只能从非冷冻可买状态转移，卖出由持有状态加当天价格得到。也可用前两天的不持有利润压缩转移。

关键不变量 / 状态：每个状态只汇总满足其持仓和冷冻条件的历史方案，非法的卖后立即买入路径不会进入持有状态。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 股票 · 状态机 · 冷冻期

易错点：卖出后的下一天不能买入，持有转移不能直接使用前一天刚卖出的利润。

## 714. 买卖股票的最佳时机含手续费

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 142/158   | 本章顺序 | 25/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:6615 |

LeetCode : <https://leetcode.cn/problems/best-time-to-buy-and-sell-stock-with-transaction-fee/>

### 题目内容

给定一个整数数组 `prices`，其中 `prices[i]` 表示第 `i` 天的股票价格；整数 `fee` 代表了交易股票的手续费用。

你可以无限次地完成交易，但是你每笔交易都需要付手续费。如果你已经购买了一个股票，在卖出它之前你就不能再继续购买股票了。

返回获得利润的最大值。

注意：这里的一笔交易指买入持有并卖出股票的整个过程，每笔交易你只需要为支付一次手续费。

示例 1：

```
输入：prices = [1, 3, 2, 8, 4, 9], fee = 2
输出：8
解释：能够达到的最大利润:
在此处买入 prices[0] = 1
在此处卖出 prices[3] = 8
在此处买入 prices[4] = 4
在此处卖出 prices[5] = 9
总利润: ((8 - 1) - 2) + ((9 - 4) - 2) = 8
```

示例 2：

```
输入：prices = [1,3,7,5,10,3], fee = 3
输出：6
```

提示：

```
1 1 <= prices.length <= 5 * 10 ^4
1 1 <= prices[i] < 5 * 10 ^4
1 0 <= fee < 5 * 10 ^4
```

我的题目思路与考察范围

核心思路：维护持有与不持有最大现金，卖出时统一扣一次手续费：`cash=max(cash,hold+price-fee)`，`hold=max(hold,cash-price)`。手续费也可等价地在买入时扣。

关键不变量 / 状态：每天结束后，`hold/cash` 分别是持有一股和不持股时的最大净利润，且每笔完整交易只扣一次费。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 贪心 · 股票 · 交易费

易错点：不要在买入和卖出两端都扣手续费；同日更新时要确保使用的状态语义一致。

## 300. 最长递增子序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 143/158   | 本章顺序 | 26/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7153 |



LeetCode : <https://leetcode.cn/problems/longest-increasing-subsequence/>

## 题目内容

给你一个整数数组 `nums`，找到其中最严格递增子序列的长度。

子序列 是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，`[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

示例 1：

输入：nums = [10,9,2,5,3,7,101,18]  
输出：4  
解释：最长递增子序列是 [2,3,7,101]，因此长度为 4。

示例 2：

输入：nums = [0,1,0,3,2,3]  
输出：4

示例 3：

输入：nums = [7,7,7,7,7,7,7]  
输出：1

提示：

1  $1 \leq \text{nums.length} \leq 2500$

1  $-10^4 \leq \text{nums}[i] \leq 10^4$

进阶：

1 你能将算法的时间复杂度降低到  $O(n \log(n))$  吗？

我的题目思路与考察范围

核心思路：维护 `tails[len-1]` 为长度 `len` 的递增子序列可取得的最小尾值，对每个数用二分找到第一个  $\geq$  它的位置替换；`tails` 长度就是 LIS 长度。经典 DP 枚举前驱为  $O(n^2)$ 。

关键不变量 / 状态：tails 严格递增，且每个长度对应的尾值在已扫描元素中最小，因此给未来延长留下最大空间。

复杂度：时间  $O(n \log n)$ ，空间  $O(n)$ 。

考察范围：动态规划 · 二分查找 · 耐心排序 · 子序列

易错点：严格递增必须使用 `lower_bound`（第一个  $\geq x$ ），若用 `upper_bound` 会错误允许相等元素延长。

## 674. 最长连续递增序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 144/158   | 本章顺序 | 27/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7269 |

LeetCode : <https://leetcode.cn/problems/longest-continuous-increasing-subsequence/>

## 题目内容

给定一个未经排序的整数数组，找到最长且 连续递增 的子序列，并返回该序列的长度。

连续递增的子序列 可以由两个下标 `l` 和 `r` ( $l < r$ ) 确定，如果对于每个  $l \leq i < r$ ，都有 `nums[i] < nums[i + 1]`，那么子序列 `[nums[l], nums[l + 1], ..., nums[r - 1], nums[r]]` 就是连续递增子序列。

示例 1：

输入：nums = [1,3,5,4,7]  
 输出：3  
 解释：最长连续递增序列是 [1,3,5]，长度为3。  
 尽管 [1,3,5,7] 也是升序的子序列，但它不是连续的，因为 5 和 7 在原数组里被 4 隔开。

示例 2：

输入：nums = [2,2,2,2,2]  
 输出：1  
 解释：最长连续递增序列是 [2]，长度为1。

提示：

1 1 <= nums.length <= 10 ^4

1 -10 ^9 <= nums[i] <= 10 ^9

我的题目思路与考察范围

核心思路：连续子数组只需比较相邻元素：若  $\text{nums}[i] > \text{nums}[i-1]$  就延长当前长度，否则重置为 1，并维护最大值。

关键不变量 / 状态：current 是必须以 i 结尾的最长连续递增段长度，answer 是已扫描前缀中的最大长度。

复杂度：时间  $O(n)$ ，空间  $O(1)$ 。

考察范围：动态规划 · 贪心 · 连续子数组 · 线性扫描

易错点：本题要求连续，不能像 LIS 那样跳过元素；遇到相等也必须重置。

## 718. 最长重复子数组

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 145/158   | 本章顺序 | 28/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7420 |

LeetCode：https://leetcode.cn/problems/maximum-length-of-repeated-subarray/

### 题目内容

给两个整数数组 nums1 和 nums2，返回两个数组中公共的、长度最长的子数组的长度。

示例 1：

输入：nums1 = [1,2,3,2,1], nums2 = [3,2,1,4,7]  
 输出：3  
 解释：长度最长的公共子数组是 [3,2,1]。

示例 2：

输入：nums1 = [0,0,0,0,0], nums2 = [0,0,0,0,0]  
 输出：5

提示：

1 1 <= nums1.length, nums2.length <= 1000

1 0 <= nums1[i], nums2[i] <= 100

我的题目思路与考察范围

核心思路：令  $\text{dp}[i][j]$  表示以  $\text{nums1}[i-1]$ 、 $\text{nums2}[j-1]$  结尾的公共连续段长度；元素相等时由左上角加一，否则归零。

一维压缩时 j 必须倒序。

关键不变量 / 状态：每个状态只描述以当前两个位置结尾的连续匹配，不相等时不能继承非连续结果。

复杂度：时间  $O(mn)$ ，空间  $O(\min(m,n))$ 。

考察范围：动态规划 · 连续子数组 · 二维DP · 滚动数组

易错点：不要写成最长公共子序列的 max 转移；一维更新若正序会污染左上角旧值。

## 1143. 最长公共子序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 146/158   | 本章顺序 | 29/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7655 |

LeetCode : <https://leetcode.cn/problems/longest-common-subsequence/>

### 题目内容

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 公共子序列 的长度。如果不存在 公共子序列，返回 0。

一个字符串的 子序列 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

1 例如，“ace” 是 “abcde” 的子序列，但 “aec” 不是 “abcde” 的子序列。

两个字符串的 公共子序列 是这两个字符串所共同拥有的子序列。

示例 1：

输入：text1 = "abcde", text2 = "ace"  
输出：3  
解释：最长公共子序列是 "ace"，它的长度为 3。

示例 2：

输入：text1 = "abc", text2 = "abc"  
输出：3  
解释：最长公共子序列是 "abc"，它的长度为 3。

示例 3：

输入：text1 = "abc", text2 = "def"  
输出：0  
解释：两个字符串没有公共子序列，返回 0。

提示：

1 `1 <= text1.length, text2.length <= 1000`

1 `text1` 和 `text2` 仅由小写英文字符组成。

我的题目思路与考察范围

核心思路：若两字符相等， $dp[i][j] = dp[i-1][j-1] + 1$ ；否则丢弃其中一个末字符，取  $dp[i-1][j]$  与  $dp[i][j-1]$  的最大值。可滚动压缩一维。

关键不变量 / 状态： $dp[i][j]$  是两个前缀 `text1[0:i]` 与 `text2[0:j]` 的最长公共子序列长度。

复杂度：时间  $O(mn)$ ，空间  $O(\min(m,n))$ 。

考察范围：动态规划 · 子序列 · 二维DP · 字符串

易错点：子序列不要求连续；字符相等时应取左上角加一，而不是当前行列的相邻最大值。

## 1035. 不相交的线

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 147/158   | 本章顺序 | 30/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7784 |

LeetCode : <https://leetcode.cn/problems/uncrossed-lines/>

## 题目内容

在两条独立的水平线上按给定的顺序写下 `nums1` 和 `nums2` 中的整数。

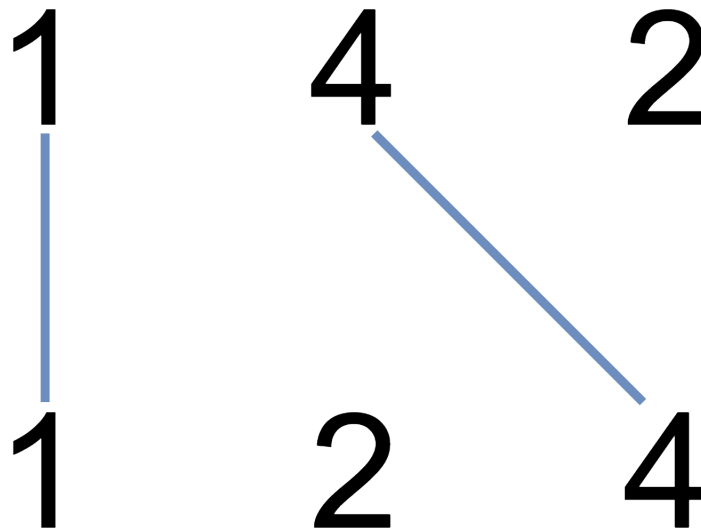
现在，可以绘制一些连接两个数字 `nums1[i]` 和 `nums2[j]` 的直线，这些直线需要同时满足：

- 1 `nums1[i] == nums2[j]`
- 1 且绘制的直线不与任何其他连线（非水平线）相交。

请注意，连线即使在端点也不能相交：每个数字只能属于一条连线。

以这种方法绘制线条，并返回可以绘制的最大连线数。

示例 1：



输入：`nums1 = [1,4,2]`, `nums2 = [1,2,4]`

输出：2

解释：可以画出两条不交叉的线，如上图所示。

但无法画出第三条不相交的直线，因为从 `nums1[1]=4` 到 `nums2[2]=4` 的直线将与从 `nums1[2]=2` 到 `nums2[1]=2` 的直线相交。

示例 2：

输入：`nums1 = [2,5,1,2,5]`, `nums2 = [10,5,2,1,5,2]`

输出：3

示例 3：

输入：`nums1 = [1,3,7,1,7,5]`, `nums2 = [1,9,2,5,1]`

输出：2

提示：

1 `1 <= nums1.length, nums2.length <= 500`

1 `1 <= nums1[i], nums2[j] <= 2000`

我的题目思路与考察范围

核心思路：不相交连线保持两数组中的相对顺序，因此问题与最长公共子序列完全等价，直接使用 LCS 转移。

关键不变量 / 状态：`dp[i][j]` 是两个前缀间最多的不相交连线数，任何合法连线集合都对应一个公共子序列。

复杂度：时间  $O(mn)$ ，空间  $O(\min(m,n))$ 。

考察范围：动态规划 · LCS · 序列 · 模型转换

易错点：不要按数值贪心匹配最近位置；局部匹配可能阻断后续更多连线。

## 53. 最大子数组和 ( 复练 )

本章顺序：31/37

LeetCode：https://leetcode.cn/problems/maximum-subarray/

完整收录：第 9 章 贪心算法

复练重点：本章改用动态规划视角复练：先定义状态，再写清转移、初始化与遍历顺序。

## 392. 判断子序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 148/158   | 本章顺序 | 32/37                            |
| 难度   | 简单        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:7947 |

LeetCode：https://leetcode.cn/problems/is-subsequence/

### 题目内容

给定字符串  $s$  和  $t$ ，判断  $s$  是否为  $t$  的子序列。

字符串的一个子序列是原始字符串删除一些（也可以不删除）字符而不改变剩余字符相对位置形成的新字符串。（例如，“ace”是“abcde”的一个子序列，而“aec”不是）。

进阶：

如果有大量输入的  $S$ ，称作  $S_1, S_2, \dots, S_k$  其中  $k \geq 10$  亿，你需要依次检查它们是否为  $T$  的子序列。在这种情况下，你会怎样改变代码？

致谢：

特别感谢 @pbrother 添加此问题并且创建所有测试用例。

示例 1：

输入：s = "abc", t = "ahbgdc"  
输出：true

示例 2：

输入：s = "axc", t = "ahbgdc"  
输出：false

提示：

- 1  $0 \leq s.length \leq 100$
- 1  $0 \leq t.length \leq 10^4$
- 1 两个字符串都只由小写字符组成。

我的题目思路与考察范围

核心思路：用双指针扫描  $s$  与  $t$ ，字符相等时推进  $s$  指针， $t$  指针始终前进；扫描结束看  $s$  是否全部匹配。若对同一  $t$  有大量查询，可预处理位置表并二分。

关键不变量 / 状态： $t$  扫描到当前位置时， $s$  指针表示此前前缀能按顺序匹配的最长前缀长度。

复杂度：单次判断时间  $O(|t|)$ ，空间  $O(1)$ ；多查询预处理方案可将每次查询降为  $O(|s| \log |t|)$ 。

考察范围：双指针 · 动态规划 · 子序列 · 字符串

易错点：只能保持相对顺序向前匹配，不能在  $t$  中回退或把问题当作子串查找。

## 115. 不同的子序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 149/158   | 本章顺序 | 33/37                            |
| 难度   | 困难        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:8077 |

LeetCode : <https://leetcode.cn/problems/distinct-subsequences/>

### 题目内容

给你两个字符串  $s$  和  $t$ ，统计并返回在  $s$  的子序列中  $t$  出现的个数。

测试用例保证结果在 32 位有符号整数范围内。

示例 1：

```
输入：s = "rabbbit", t = "rabbit"
输出：3
解释：
如下所示，有 3 种可以从 s 中得到 "rabbit" 的方案。
rabbbit
rabbbit
rabbbit
```

示例 2：

```
输入：s = "babgbag", t = "bag"
输出：5
解释：
如下所示，有 5 种可以从 s 中得到 "bag" 的方案。
babgbag
babgbag
babgbag
babgbag
babgbag
```

提示：

1  $1 \leq s.length, t.length \leq 1000$

1  $s$  和  $t$  由英文字母组成

我的题目思路与考察范围

核心思路： $dp[i][j]$  表示  $s$  的前  $i$  个字符构成  $t$  的前  $j$  个字符的方案数；字符相等时可选或不选当前  $s$  字符，方案为左上加上方，否则只能不选。

关键不变量 / 状态：每种方案按是否使用当前  $s$  字符被唯一拆分，所有保持顺序的选择都被计入一次。

复杂度：时间  $O(mn)$ ，空间  $O(n)$ ；一维压缩时目标下标需倒序。

考察范围：动态规划 · 子序列 · 方案计数 · 字符串

易错点：空目标串有一种构造方式，应初始化  $dp[*][0]=1$ ；计数类型要防止中间结果溢出。

## 583. 两个字符串的删除操作

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 150/158   | 本章顺序 | 34/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:8220 |

LeetCode : <https://leetcode.cn/problems/delete-operation-for-two-strings/>

## 题目内容

给定两个单词 word1 和 word2，返回使得 word1 和 word2 相同所需的最小步数。

每步可以删除任意一个字符串中的一个字符。

示例 1：

输入: word1 = "sea", word2 = "eat"  
输出: 2  
解释: 第一步将 "sea" 变为 "ea"，第二步将 "eat" 变为 "ea"

示例 2:

输入: word1 = "leetcode", word2 = "etco"  
输出: 4

提示：

1 1 <= word1.length, word2.length <= 500

1 word1 和 word2 只包含小写英文字母

我的题目思路与考察范围

核心思路：保留两字符串的最长公共子序列，其余字符全部删除，最少删除数为  $m+n-2*LCS$ ；也可直接定义删除操作 DP。

关键不变量 / 状态：任何最终相同字符串都是公共子序列，保留最长者等价于删除总数最少。

复杂度：时间  $O(mn)$ ，空间  $O(\min(m,n))$ 。

考察范围：动态规划 · LCS · 编辑距离 · 字符串

易错点：这里仅允许删除，不能直接套含替换操作的编辑距离而不修改代价。

## 72. 编辑距离

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 151/158   | 本章顺序 | 35/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:8344 |

LeetCode：https://leetcode.cn/problems/edit-distance/

## 题目内容

给你两个单词 word1 和 word2，请返回将 word1 转换成 word2 所使用的最少操作数。

你可以对一个单词进行如下三种操作：

1 插入一个字符

1 删除一个字符

1 替换一个字符

示例 1：

输入: word1 = "horse", word2 = "ros"  
输出: 3  
解释：  
horse -> rorse (将 'h' 替换为 'r')  
rorse -> rose (删除 'r')  
rose -> ros (删除 'e')

示例 2：

输入：word1 = "intention", word2 = "execution"  
 输出：5  
 解释：  
 intention -> inention (删除 't')  
 inention -> enention (将 'i' 替换为 'e')  
 enention -> exention (将 'n' 替换为 'x')  
 exention -> exection (将 'n' 替换为 'c')  
 exection -> execution (插入 'u')

提示：

1 0 <= word1.length, word2.length <= 500

1 word1 和 word2 由小写英文字母组成

我的题目思路与考察范围

核心思路：dp[i][j] 表示 word1 前 i 个字符变为 word2 前 j 个字符的最少操作；末字符相等取左上，否则在删除、插入、替换三种前驱中取最小再加一。

关键不变量 / 状态：每个状态已覆盖最后一次操作的全部可能，且其前驱都是更短前缀的最优解。

复杂度：时间  $O(mn)$ ，空间  $O(\min(m,n))$ 。

考察范围：动态规划 · 编辑距离 · 字符串 · 二维DP

易错点：首行和首列分别是空串插入或删除的代价，必须初始化为连续下标。

## 647. 回文子串

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 152/158   | 本章顺序 | 36/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:8709 |

LeetCode：https://leetcode.cn/problems/palindromic-substrings/

### 题目内容

给你一个字符串 s，请你统计并返回这个字符串中回文子串的数目。

回文字符串 是正着读和倒过来读一样的字符串。

子字符串 是字符串中的由连续字符组成的一个序列。

示例 1：

输入：s = "abc"  
 输出：3  
 解释：三个回文子串："a", "b", "c"

示例 2：

输入：s = "aaa"  
 输出：6  
 解释：6个回文子串："a", "a", "a", "aa", "aa", "aaa"

提示：

1 1 <= s.length <= 1000

1 s 由小写英文字母组成



## 我的题目思路与考察范围

核心思路：以每个字符和每对相邻字符为中心向两侧扩展，只要字符相等就得到一个新的回文子串；区间 DP 也能在  $O(n^2)$  时间完成但需更多空间。

关键不变量 / 状态：一次中心扩展中，当前  $[l, r]$  始终是回文，下一层是否成立只取决于两端字符是否相等。

复杂度：时间  $O(n^2)$ ，空间  $O(1)$ 。

考察范围：双指针 · 中心扩展 · 动态规划 · 回文

易错点：奇数长度与偶数长度有两类中心，漏掉任一类都会少计回文子串。

## 516. 最长回文子序列

|      |           |      |                                  |
|------|-----------|------|----------------------------------|
| 唯一题序 | 153/158   | 本章顺序 | 37/37                            |
| 难度   | 中等        | 题目类型 | 主讲题                              |
| 出现章节 | 第10章 动态规划 | 源位置  | 10.《代码随想录》动态规划 ( V3.0 ) .md:8929 |

LeetCode : <https://leetcode.cn/problems/longest-palindromic-subsequence/>

## 题目内容

给你一个字符串  $s$ ，找出其中最长的回文子序列，并返回该序列的长度。

子序列定义为：不改变剩余字符顺序的情况下，删除某些字符或者不删除任何字符形成的一个序列。

示例 1：

输入： $s = "bbbab"$   
 输出：4  
 解释：一个可能的最长回文子序列为 "bbbb"。

示例 2：

输入： $s = "cbbd"$   
 输出：2  
 解释：一个可能的最长回文子序列为 "bb"。

提示：

- $1 \leq s.length \leq 1000$
- $s$  仅由小写英文字母组成

## 我的题目思路与考察范围

核心思路：区间 DP 令  $dp[i][j]$  为  $s[i:j+1]$  的最长回文子序列；两端相等时取  $dp[i+1][j-1]+2$ ，否则取去掉任一端后的最大值。

关键不变量 / 状态：按区间长度递增或  $i$  逆序计算时，当前状态依赖的更短区间都已完成。

复杂度：时间  $O(n^2)$ ，空间  $O(n^2)$ ，可滚动压缩到  $O(n)$ 。

考察范围：动态规划 · 区间DP · 子序列 · 回文

易错点：子序列不要求连续；单字符区间必须初始化为 1，遍历顺序要保证  $i+1$  状态已知。

## 第 11 章 单调栈

本章 5 道去重题目，按源文件首次出现顺序排列。

### 739. 每日温度

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 154/158  | 本章顺序 | 1/5                            |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第11章 单调栈 | 源位置  | 11. 《代码随想录》单调栈 ( V3.0 ) .md:19 |

LeetCode : <https://leetcode.cn/problems/daily-temperatures/>

#### 题目内容

给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 0 来代替。

示例 1:

输入: `temperatures = [73,74,75,71,69,72,76,73]`  
输出: `[1,1,4,2,1,1,0,0]`

示例 2:

输入: `temperatures = [30,40,50,60]`  
输出: `[1,1,1,0]`

示例 3:

输入: `temperatures = [30,60,90]`  
输出: `[1,1,0]`

提示：

1 `1 <= temperatures.length <= 10 ^5`

1 `30 <= temperatures[i] <= 100`

我的题目思路与考察范围

核心思路：维护存放下标的单调递减栈；当前温度更高时不断弹出较冷日期，并用下标差填写其等待天数，最后把当前下标入栈。

关键不变量 / 状态：栈中下标对应的温度从栈底到栈顶非递增，且这些日期尚未找到右侧第一个更高温度。

复杂度：时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：单调栈 · 下一个更大元素 · 数组 · 下标距离

易错点：栈里必须存下标而不是温度，才能计算距离；相等温度不能触发弹栈。

### 496. 下一个更大元素 I

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 155/158  | 本章顺序 | 2/5                             |
| 难度   | 简单       | 题目类型 | 主讲题                             |
| 出现章节 | 第11章 单调栈 | 源位置  | 11. 《代码随想录》单调栈 ( V3.0 ) .md:194 |

LeetCode : <https://leetcode.cn/problems/next-greater-element-i/>

## 题目内容

nums1 中数字 x 的下一个更大元素 是指 x 在 nums2 中对应位置 右侧 的 第一个 比 x 大的元素。

给你两个 没有重复元素 的数组 nums1 和 nums2 ，下标从 0 开始计数，其中 nums1 是 nums2 的子集。

对于每个  $0 \leq i < \text{nums1.length}$  ，找出满足  $\text{nums1}[i] == \text{nums2}[j]$  的下标 j ，并且在 nums2 确定  $\text{nums2}[j]$  的 下一个更大元素 。如果不存在下一个更大元素，那么本次查询的答案是 -1 。

返回一个长度为  $\text{nums1.length}$  的数组 ans 作为答案，满足  $\text{ans}[i]$  是如上所述的 下一个更大元素 。

示例 1：

输入：nums1 = [4,1,2], nums2 = [1,3,4,2].  
 输出：[-1,3,-1]  
 解释：nums1 中每个值的下一个更大元素如下所述：  
 - 4 ，用加粗斜体标识，nums2 = [1,3,4,2]。不存在下一个更大元素，所以答案是 -1 。  
 - 1 ，用加粗斜体标识，nums2 = [1,3,4,2]。下一个更大元素是 3 。  
 - 2 ，用加粗斜体标识，nums2 = [1,3,4,2]。不存在下一个更大元素，所以答案是 -1 。

示例 2：

输入：nums1 = [2,4], nums2 = [1,2,3,4].  
 输出：[3,-1]  
 解释：nums1 中每个值的下一个更大元素如下所述：  
 - 2 ，用加粗斜体标识，nums2 = [1,2,3,4]。下一个更大元素是 3 。  
 - 4 ，用加粗斜体标识，nums2 = [1,2,3,4]。不存在下一个更大元素，所以答案是 -1 。

提示：

- 1  $1 \leq \text{nums1.length} \leq \text{nums2.length} \leq 1000$
- 1  $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^4$
- 1 nums1 和 nums2 中所有整数 互不相同
- 1 nums1 中的所有整数同样出现在 nums2 中

进阶：你可以设计一个时间复杂度为  $O(\text{nums1.length} + \text{nums2.length})$  的解决方案吗？

我的题目思路与考察范围

核心思路：扫描 nums2 并用单调递减栈求每个值右侧第一个更大元素，弹栈时写入哈希表；再按 nums1 的顺序查询结果。

关键不变量 / 状态：栈中元素尚未找到更大值且从底到顶递减，当前值是所有被弹元素遇到的第一个更大值。

复杂度：时间  $O(n+m)$ ，空间  $O(n)$ 。

考察范围：单调栈 · 哈希表 · 下一个更大元素 · 子集查询

易错点：答案顺序由 nums1 决定，不能直接返回扫描 nums2 得到的顺序；题设值唯一才可直接以值作键。

## 503. 下一个更大元素 II

|      |          |      |                                |
|------|----------|------|--------------------------------|
| 唯一题序 | 156/158  | 本章顺序 | 3/5                            |
| 难度   | 中等       | 题目类型 | 主讲题                            |
| 出现章节 | 第11章 单调栈 | 源位置  | 11.《代码随想录》单调栈 ( V3.0 ) .md:372 |

LeetCode：https://leetcode.cn/problems/next-greater-element-ii/

## 题目内容

给定一个循环数组 nums ( nums[nums.length - 1] 的下一个元素是 nums[0] )，返回 nums 中每个元素的 下一个更大元素 。

数字  $x$  的下一个更大的元素是按数组遍历顺序，这个数字之后的第一个比它更大的数，这意味着你应该循环地搜索它的下一个更大的数。如果不存在，则输出  $-1$ 。

示例 1:

输入:  $\text{nums} = [1,2,1]$   
 输出:  $[2,-1,2]$   
 解释: 第一个  $1$  的下一个更大的数是  $2$ ；  
 数字  $2$  找不到下一个更大的数；  
 第二个  $1$  的下一个最大的数需要循环搜索，结果也是  $2$ 。

示例 2:

输入:  $\text{nums} = [1,2,3,4,3]$   
 输出:  $[2,3,4,-1,4]$

提示:

1  $1 \leq \text{nums.length} \leq 10^4$

1  $-10^9 \leq \text{nums}[i] \leq 10^9$

我的题目思路与考察范围

核心思路：把环形数组视为扫描两遍，用  $i\%n$  访问元素并维护单调递减下标栈；第二遍只负责结算未找到答案的下标，通常只在第一遍压栈。

关键不变量 / 状态：栈中下标对应元素仍未在其右侧一圈范围内遇到更大值，当前元素是被弹下标的首个更大值。

复杂度：时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：单调栈 · 环形数组 · 取模 · 下一个更大元素

易错点：若两遍都无条件压栈会产生重复下标；比较必须严格大于，相等值不是答案。

## 42. 接雨水

|      |          |      |                                 |
|------|----------|------|---------------------------------|
| 唯一题序 | 157/158  | 本章顺序 | 4/5                             |
| 难度   | 困难       | 题目类型 | 主讲题                             |
| 出现章节 | 第11章 单调栈 | 源位置  | 11. 《代码随想录》单调栈 ( V3.0 ) .md:502 |

LeetCode：https://leetcode.cn/problems/trapping-rain-water/

### 题目内容

给定  $n$  个非负整数表示每个宽度为  $1$  的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：



输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出：6

解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2：

输入：height = [4,2,0,3,2,5]

输出：9

提示：

1 n == height.length

1 1 <= n <= 2 \* 10 ^4

1 0 <= height[i] <= 10 ^5

我的题目思路与考察范围

核心思路：单调递减栈遇到更高柱时，以弹出的柱为凹槽底、当前柱和新栈顶为左右边界，按宽度乘有效高度累加雨水。双指针维护左右最大值可把额外空间降为  $O(1)$ 。

关键不变量 / 状态：栈中柱高从底到顶非递增，弹栈时左右第一个更高边界已确定，因此该层凹槽水量可一次结算。

复杂度：单调栈时间  $O(n)$ 、空间  $O(n)$ ；双指针时间  $O(n)$ 、空间  $O(1)$ 。

考察范围：单调栈 · 双指针 · 面积 · 凹槽模型

易错点：弹出槽底后若栈为空就没有左边界；宽度应为  $\text{right-left}-1$ ，高度要减去槽底高度。

## 84. 柱状图中最大的矩形

|      |          |      |                            |
|------|----------|------|----------------------------|
| 唯一题序 | 158/158  | 本章顺序 | 5/5                        |
| 难度   | 困难       | 题目类型 | 主讲题                        |
| 出现章节 | 第11章 单调栈 | 源位置  | 11.《代码随想录》单调栈（V3.0）.md:869 |

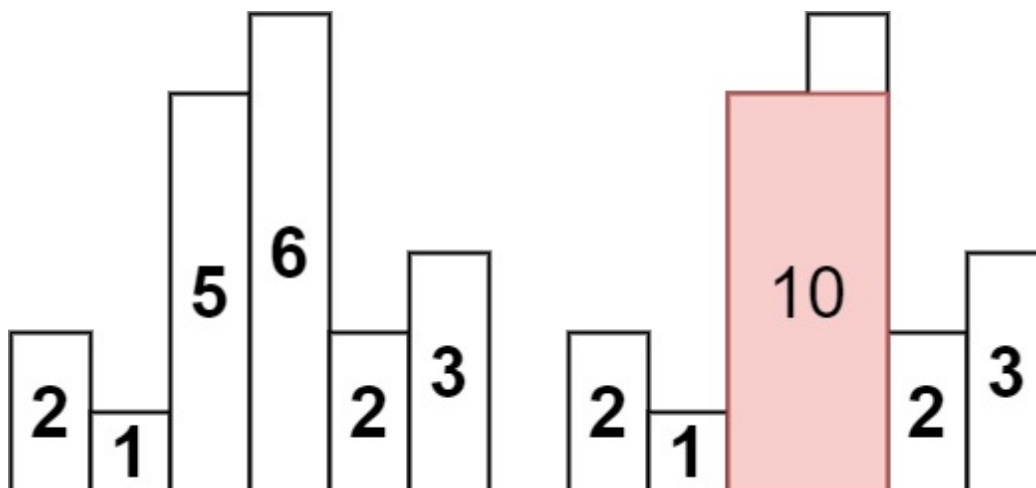
LeetCode：https://leetcode.cn/problems/largest-rectangle-in-histogram/

### 题目内容

给定  $n$  个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。

求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1：

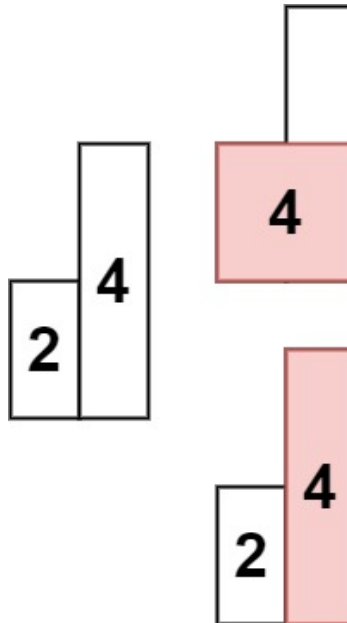


输入：heights = [2,1,5,6,2,3]

输出：10

解释：最大的矩形为图中红色区域，面积为 10

示例 2：



输入：heights = [2,4]

输出：4

提示：

1 1 <= heights.length <= 10 ^5

1 0 <= heights[i] <= 10 ^4

我的题目思路与考察范围

核心思路：在高度数组两端加入 0 哨兵，维护单调递增下标栈；遇到更矮柱时，弹出的柱高作为矩形高度，新栈顶与当前下标给出其能延伸的左右边界。

关键不变量 / 状态：栈中高度单调递增，未弹出的每根柱还没有遇到其右侧第一个更矮柱。

复杂度：时间  $O(n)$ ，空间  $O(n)$ 。

考察范围：单调栈 · 柱状图 · 最大面积 · 哨兵

易错点：宽度是  $\text{currentIndex} - \text{newTop} - 1$ ；尾部哨兵用于强制结算所有剩余柱，等高柱要采用一致的处理策略。

## 附录：校验口径

章节顺序按源目录 README 与文件数字前缀确定。

仅收录指向 LeetCode /problems/<slug>/ 的语义化 Markdown 链接；programmercarl、Bilibili、GitHub 与卡码网链接不计入题目。

同章按首次出现顺序去重；跨章复练保留章节位置。

题干为生成日获取的公开题面；若 LeetCode 后续修订，以题目链接为准。